

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Công - Nguyễn Hưng

XÂY DỰNG HỆ THỐNG KIỂM THỬ TỰ
ĐỘNG
ỨNG DỤNG WEB

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

Tp. Hồ Chí Minh, tháng 07/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Chí Công - 22120038

Nguyễn Hưng - 22120122

**XÂY DỰNG HỆ THỐNG KIỂM THỬ TỰ
ĐỘNG
ỨNG DỤNG WEB**

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN

GIẢNG VIÊN HƯỚNG DẪN

PGS. TS. Nguyễn Văn Vũ

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan rằng khóa luận tốt nghiệp “Xây dựng hệ thống kiểm thử tự động ứng dụng Web” là công trình nghiên cứu độc lập của chúng tôi, được thực hiện dưới sự hướng dẫn khoa học của PGS.TS. Nguyễn Văn Vũ, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh.

Toàn bộ nội dung trình bày trong khóa luận này, bao gồm các phân tích, thiết kế, cài đặt và kết quả thực nghiệm, đều do chúng tôi trực tiếp thực hiện và chưa từng được công bố trong bất kỳ công trình nào khác. Các tài liệu, số liệu và kết quả trích dẫn từ nguồn bên ngoài đều được chú thích rõ ràng theo quy định. Chúng tôi hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của những nội dung được trình bày.

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Nguyễn Chí Công

Nguyễn Hưng

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Để hoàn thành khóa luận này, chúng tôi đã nhận được sự hỗ trợ và đồng hành của nhiều cá nhân và tập thể mà chúng tôi trân trọng ghi nhận.

Trước hết, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS. Nguyễn Văn Vũ, người đã tận tình định hướng nghiên cứu, góp ý về phương pháp tiếp cận và dành thời gian đọc, nhận xét qua từng giai đoạn của quá trình thực hiện. Những phản hồi khoa học và kinh nghiệm thực tiễn của thầy là nền tảng quan trọng giúp chúng tôi hình thành tư duy hệ thống và giải quyết các vấn đề kỹ thuật phát sinh trong đề tài.

Chúng tôi cũng xin gửi lời cảm ơn chân thành đến quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, đã trang bị cho chúng tôi nền tảng kiến thức và kỹ năng trong suốt bốn năm học. Môi trường học thuật nghiêm túc và cởi mở của Khoa chính là điều kiện để chúng tôi có thể theo đuổi và hoàn thiện đề tài này.

Lời cảm ơn đặc biệt xin được gửi đến gia đình, những người luôn là chỗ dựa tinh thần vững chắc trong những thời điểm khó khăn nhất của quá trình nghiên cứu. Sự tin tưởng và khích lệ của gia đình là động lực lớn để chúng tôi kiên trì đến cuối.

Mặc dù đã cố gắng hết sức, khóa luận chắc chắn còn những điểm chưa hoàn thiện. Chúng tôi rất mong nhận được sự góp ý từ hội đồng phản biện và các bạn đọc để tiếp tục cải thiện trong những công trình tiếp theo.

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Nguyễn Chí Công

Nguyễn Hưng

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó (*phải có chữ ký của giảng viên hướng dẫn*).

Mục lục

Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vii
Mục lục	viii
Danh sách hình	xiv
Danh sách bảng	xv
Danh mục từ viết tắt	xvii
Tóm tắt	xvii
1 Tổng quan đề tài	1
1.1 Bối cảnh và vấn đề nghiên cứu	1
1.2 Mục tiêu đề tài	2
1.3 Phạm vi đề tài	3
1.4 Giải pháp đề xuất	4
1.5 Kết quả đạt được	5

1.6	Cấu trúc báo cáo	6
2	Nền tảng kỹ thuật và các giải pháp liên quan	8
2.1	Kiểm thử web và kiểm thử tự động	8
2.2	Một số công cụ kiểm thử web liên quan	9
2.3	Mô hình ngôn ngữ lớn trong hệ thống kiểm thử web	11
2.4	AI agent và browser automation	12
2.5	Định vị giải pháp đề xuất	13
3	Phân tích yêu cầu hệ thống	15
3.1	Tổng quan hệ thống	15
3.2	Nhóm người dùng	17
3.3	Yêu cầu chức năng	18
3.4	Yêu cầu phi chức năng	20
3.5	Sơ đồ use case	21
3.6	Đặc tả use case chính	25
3.7	Kết chương	27
4	Kiến trúc và thiết kế hệ thống	29
4.1	Kiến trúc tổng thể hệ thống	29
4.1.1	Kiến trúc component cấp hệ thống	30
4.1.2	Công nghệ sử dụng	31
4.1.3	Luồng giao tiếp tổng quan	32
4.2	Thiết kế dữ liệu và lưu trữ	33
4.2.1	Mô hình dữ liệu cốt lõi	34
4.2.2	Các bảng hỗ trợ và phạm vi trình bày	36
4.2.3	Các quyết định schema đáng chú ý	39
4.3	Thiết kế các luồng xử lý cốt lõi	40
4.3.1	Luồng tạo project, scan/crawl website và sinh test case	40
4.3.2	Luồng chạy bằng goal-based agent	41
4.3.3	Luồng chạy bằng replay script	43
4.3.4	Luồng sinh report và evidence	44

4.4	Thiết kế agent-worker và cơ chế thực thi	45
4.5	Thiết kế cơ chế hỗ trợ bảo trì kiểm thử	47
4.5.1	Object Repository và State Anchors	47
4.5.2	Quản lý test suite và dữ liệu kiểm thử	48
4.6	Thiết kế bảo mật và kiểm soát vận hành	48
4.7	Cơ sở lựa chọn thiết kế và các đánh đổi	50
4.8	Kết chương	52
5	Xây dựng và tích hợp hệ thống	53
5.1	Môi trường và công nghệ sử dụng	53
5.2	Xây dựng các thành phần chính	55
5.2.1	Giao diện người dùng	55
5.2.2	Backend API	55
5.2.3	Agent-worker	56
5.2.4	Cơ sở dữ liệu	56
5.3	Tích hợp các công nghệ lõi	57
5.3.1	Tích hợp Browser-use	57
5.3.2	Tích hợp Playwright	58
5.3.3	Tích hợp Google Gemini Flash	58
5.4	Xây dựng các chức năng kiểm thử	59
5.4.1	Scan/crawl website	59
5.4.2	Sinh test case	59
5.4.3	Chạy test bằng agent	59
5.4.4	Replay script	60
5.4.5	Quản lý test data và test suite	60
5.4.6	Report và evidence	61
5.5	Xây dựng cơ chế hỗ trợ	61
5.5.1	Object repository và locator fallback	61
5.5.2	AI analysis và report	62
5.6	Bảo mật và xử lý dữ liệu nhạy cảm	62
5.7	Khó khăn trong quá trình xây dựng	63

5.8	Kết chương	64
6	Thực nghiệm và đánh giá hệ thống	65
6.1	Mục tiêu đánh giá	65
6.2	Môi trường và dữ liệu thực nghiệm	66
6.2.1	Môi trường thực nghiệm	66
6.2.2	Nguồn dữ liệu đánh giá	68
6.2.3	Quy trình thực nghiệm	69
6.3	Case study: OrangeHRM	70
6.4	Thiết kế kịch bản và chỉ số đánh giá	71
6.4.1	Các kịch bản đánh giá	71
6.4.2	Các chỉ số đánh giá	73
6.5	Kết quả thực nghiệm từ log hệ thống	74
6.5.1	Tổng quan kết quả thực nghiệm	74
6.5.2	Kết quả theo nhóm kịch bản	75
6.5.3	Kết quả sinh test case	76
6.5.4	Kết quả sinh dataset	77
6.6	So sánh chế độ agent và replay script	78
6.7	Phân tích lỗi trong quá trình thực nghiệm	81
6.7.1	Phân loại lỗi theo failure type	81
6.7.2	Phân tích lỗi theo step thất bại	82
6.8	Đánh giá các cơ chế hỗ trợ	83
6.8.1	Object repository và self-healing	83
6.8.2	Evidence, log và report	84
6.8.3	Phân tích lỗi bằng mô hình ngôn ngữ lớn	84
6.9	Đánh giá trải nghiệm người dùng	85
6.9.1	Mục tiêu khảo sát	85
6.9.2	Đối tượng tham gia khảo sát	85
6.9.3	Kết quả khảo sát theo nhóm tiêu chí	86
6.9.4	Mức độ chỉnh sửa test case sinh ra	87
6.10	Nhận xét tổng hợp	88

6.11	Các yếu tố ảnh hưởng và giới hạn thực nghiệm	90
6.12	Kết chương	91
7	Tổ chức thực hiện đề tài	93
7.1	Phân chia công việc trong nhóm	93
7.1.1	Vai trò của từng thành viên	93
7.1.2	Phạm vi công việc của từng thành viên	94
7.1.3	Cách phối hợp giữa các thành viên	96
7.2	Quy trình làm việc	96
7.2.1	Quy trình phát triển hệ thống	96
7.2.2	Quy trình quản lý mã nguồn	97
7.2.3	Quy trình kiểm tra và tích hợp chức năng	97
7.3	Kế hoạch thực hiện	98
7.3.1	Các giai đoạn thực hiện đề tài	98
7.3.2	Tiến độ thực hiện	99
7.3.3	Kết quả đạt được theo từng giai đoạn	99
7.4	Công cụ hỗ trợ thực hiện	100
7.4.1	Công cụ quản lý mã nguồn	100
7.4.2	Công cụ phát triển và kiểm thử	100
7.4.3	Công cụ triển khai và theo dõi hệ thống	100
7.4.4	Công cụ hỗ trợ trao đổi trong nhóm	101
7.5	Khó khăn và cách xử lý	101
7.5.1	Khó khăn về kỹ thuật	101
7.5.2	Khó khăn về tích hợp hệ thống	101
7.5.3	Khó khăn trong quá trình thực nghiệm	102
7.5.4	Cách xử lý và bài học rút ra	102
7.6	Kết chương	103
8	Kết luận và hướng phát triển	104
8.1	Kết quả đạt được	104
8.1.1	Về mặt nghiên cứu và kiến thức nền tảng	104
8.1.2	Về mặt xây dựng hệ thống	105

8.1.3 Về mặt thực nghiệm và đánh giá	106
8.2 Hạn chế của hệ thống	106
8.3 Bài học kinh nghiệm	107
8.4 Hướng phát triển trong tương lai	108
8.5 Kết luận chung	109
Danh mục công trình của tác giả	110
Tài liệu tham khảo	111
A Ngữ pháp tiếng Việt	113
B Ngữ pháp tiếng Nôm	114

Danh sách hình

3.1	Use case quản lý tài khoản và project	22
3.2	Use case tạo test case và chạy bằng agent	23
3.3	Use case replay script, test data và test suite	24
3.4	Use case evidence, báo cáo và phân tích lỗi	25
4.1	Kiến trúc tổng thể của hệ thống kiểm thử tự động ứng dụng web	31
4.2	Mô hình dữ liệu cốt lõi phục vụ vòng đời kiểm thử	34
4.3	Luồng tạo project và sinh test case	41
4.4	Luồng chạy test bằng goal-based agent	42
4.5	Luồng chạy test bằng replay script	43
4.6	Luồng lưu evidence và sinh báo cáo	44
4.7	Thiết kế bên trong agent-worker	46

Danh sách bảng

3.1	Các thành phần logic chính của hệ thống	16
3.2	Danh sách yêu cầu chức năng của hệ thống	18
3.3	Danh sách yêu cầu phi chức năng của hệ thống	20
3.4	Đặc tả rút gọn các use case chính	26
4.1	Công nghệ sử dụng theo thành phần	32
4.2	Các bảng cốt lõi trong thiết kế dữ liệu	35
4.3	Các nhóm bảng hỗ trợ trong hệ thống	37
4.4	Các quyết định schema đáng chú ý	39
4.5	Các nhóm kiểm soát bảo mật và vận hành	49
4.6	Tổng hợp các quyết định kiến trúc lớn của Chương 4	51
5.1	Môi trường và công nghệ sử dụng	54
6.1	Môi trường thực nghiệm hệ thống	67
6.2	Nguồn dữ liệu sử dụng trong đánh giá	69
6.3	Các kịch bản đánh giá trên OrangeHRM	72
6.4	Các chỉ số đánh giá hệ thống	74
6.5	Tổng hợp kết quả thực nghiệm trên OrangeHRM	75
6.6	Tỷ lệ thực thi thành công theo nhóm kịch bản chính	76
6.7	Kết quả sinh test case từ prompt	77
6.8	Kết quả sinh dataset	78
6.9	So sánh kết quả thực thi giữa agent và replay script	78
6.10	So sánh thời gian thực thi theo kết quả chạy	79

6.11 Ví dụ so sánh thời gian giữa lần chạy agent và replay thành công	80
6.12 Phân loại lỗi trong quá trình thực nghiệm	81
6.13 Tỷ lệ step thất bại theo loại hành động	82
6.14 Nhóm người tham gia khảo sát	85
6.15 Kinh nghiệm của người tham gia với kiểm thử tự động Web	86
6.16 Tổng hợp kết quả khảo sát theo nhóm tiêu chí	86
6.17 Mức độ chỉnh sửa test case sinh ra trước khi sử dụng . . .	87
6.18 So sánh agent và replay script	88
6.19 Tổng hợp mức độ đáp ứng các mục tiêu đánh giá	89
7.1 Vai trò của từng thành viên trong nhóm	94
7.2 Phân chia phạm vi công việc	95
7.3 Các giai đoạn thực hiện đề tài	98
7.4 Tiến độ thực hiện đề tài	99

Tóm tắt

Giới hạn 200-300 từ, sử dụng văn phong học thuật, ngắn gọn, khách quan, viết liền mạch, không gạch đầu dòng.

Hướng dẫn trình bày tóm tắt:

1. Giới thiệu vấn đề/ngữ cảnh của vấn đề (khoảng 1 - 3 câu): Trình bày bối cảnh và vấn đề thực tế đang tồn tại/ khoảng trống nghiên cứu, tầm quan trọng của vấn đề nghiên cứu, lý do cần thực hiện đề tài.
2. Mục tiêu đề tài (1 câu): nêu cụ thể đề tài giải quyết/phát triển điều gì.
3. Phương pháp và giải pháp (1 - 3 câu): Trình bày phương pháp, công nghệ, công cụ, mô hình, giải pháp được đề xuất trong đề tài. Nêu những cải tiến nổi bật (nếu có). Nêu phương pháp thử nghiệm, đánh giá, bộ dữ liệu được sử dụng để đánh giá (nếu có)
4. Kết quả chính (1 - 2 câu): Trình bày ngắn gọn các kết quả nổi bật, có thể đưa số liệu cụ thể (độ chính xác, tốc độ xử lý, dung lượng dữ liệu, ...)
5. Kết luận (1 - 2 câu): Nêu ý nghĩa khoa học và thực tiễn (nếu có) của phương pháp/giải pháp hay kết quả của đề tài, khả năng áp dụng vào thực tế/đóng góp cho doanh nghiệp/người dùng/nghiên cứu sau này, ...

Chương 1

Tổng quan đề tài

1.1 Bối cảnh và vấn đề nghiên cứu

Ứng dụng web hiện nay được sử dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, quản trị nội bộ, giáo dục trực tuyến, chăm sóc khách hàng và các hệ thống doanh nghiệp. Cùng với sự phát triển đó, giao diện web ngày càng phức tạp hơn do có nhiều trạng thái tương tác, dữ liệu bất đồng bộ, thành phần động và luồng nghiệp vụ thay đổi thường xuyên.

Những thay đổi liên tục của ứng dụng web đặt ra yêu cầu cao đối với hoạt động kiểm thử. Mỗi lần cập nhật giao diện, sửa lỗi hoặc bổ sung chức năng đều có thể làm phát sinh lỗi hồi quy. Nếu các lỗi này không được phát hiện kịp thời, chúng có thể ảnh hưởng đến trải nghiệm người dùng, độ tin cậy của hệ thống và chất lượng sản phẩm. Các nghiên cứu về chất lượng phần mềm cũng cho thấy chi phí xử lý lỗi thường tăng lên khi lỗi được phát hiện ở giai đoạn muộn [1].

Kiểm thử thủ công vẫn cần thiết vì con người có khả năng đánh giá ngữ cảnh và phát hiện các bất thường về trải nghiệm. Tuy nhiên, khi số lượng chức năng tăng lên và chu kỳ phát hành ngắn hơn, việc lặp lại nhiều kịch bản kiểm thử bằng thao tác thủ công trở nên tốn thời gian và dễ sai sót. Vì vậy, kiểm thử tự động là hướng tiếp cận quan trọng để hỗ trợ chạy lại các kịch bản kiểm thử và phát hiện lỗi sớm hơn.

Hiện nay, các công cụ như Selenium và Playwright đã hỗ trợ tốt việc tự động hóa trình duyệt [2], [3]. Tuy nhiên, các công cụ này thường yêu cầu người dùng viết script, khai báo selector, XPath hoặc hiểu cấu trúc DOM của trang web. Khi giao diện thay đổi, selector có thể không còn phù hợp, khiến kịch bản kiểm thử thất bại dù chức năng nghiệp vụ vẫn hoạt động đúng.

Từ thực tế trên, đề tài tập trung vào bài toán xây dựng một hệ thống hỗ trợ kiểm thử tự động ứng dụng web, trong đó người dùng có thể mô tả mục tiêu kiểm thử bằng ngôn ngữ tự nhiên. Hệ thống hướng đến việc giảm rào cản viết script kiểm thử, hỗ trợ thực thi kiểm thử trên trình duyệt, lưu lại bằng chứng thực thi và phục vụ việc chạy lại các kịch bản kiểm thử trong những lần sau.

1.2 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống kiểm thử tự động ứng dụng web có khả năng hỗ trợ người dùng tạo, thực thi, lưu vết và đánh giá các ca kiểm thử thông qua một nền tảng web.

Các mục tiêu cụ thể bao gồm:

- Xây dựng nền tảng web cho phép người dùng tạo project kiểm thử và khai báo website mục tiêu.
- Hỗ trợ tạo test case từ prompt hoặc goal bằng ngôn ngữ tự nhiên.
- Tích hợp mô hình ngôn ngữ lớn để hỗ trợ sinh test case, hỗ trợ agent trong quá trình chạy kiểm thử và gợi ý nguyên nhân khi test thất bại.
- Tích hợp Browser-use agent và Playwright để thực thi kiểm thử trên trình duyệt.

- Ghi nhận log, screenshot, action history, verdict và evidence sau mỗi lần chạy test.
- Sinh và chạy lại replay script nhằm hỗ trợ kiểm thử hồi quy và giảm sự phụ thuộc vào mô hình ngôn ngữ lớn trong các lần chạy sau.
- Hỗ trợ quản lý test data, test suite, object repository và báo cáo kết quả kiểm thử.

1.3 Phạm vi đề tài

Về phạm vi chức năng, hệ thống tập trung vào kiểm thử các luồng thao tác trên giao diện người dùng của ứng dụng web, chẳng hạn như mở trang, đăng nhập, nhập dữ liệu, nhấn nút, điều hướng và kiểm tra nội dung hiển thị.

Hệ thống phù hợp nhất với môi trường staging hoặc môi trường kiểm thử có kiểm soát. Đây là môi trường an toàn hơn để chạy các kịch bản tự động, sử dụng tài khoản test và ghi nhận bằng chứng kiểm thử. Website production không phải là phạm vi chính của đề tài do có nhiều rủi ro liên quan đến dữ liệu thật, quyền truy cập và tác động ngoài ý muốn.

Về phạm vi kỹ thuật, đề tài tập trung vào kiểm thử giao diện web trên trình duyệt máy tính để bàn thông qua Browser-use và Playwright. Đề tài chưa đi sâu vào kiểm thử hiệu năng, kiểm thử bảo mật chuyên sâu, kiểm thử API độc lập, kiểm thử ứng dụng di động, kiểm thử chịu tải hoặc kiểm thử tương thích trình duyệt ở quy mô lớn.

Hệ thống cũng không nhằm tự động xử lý mọi tình huống phức tạp trên website. Các trường hợp như CAPTCHA, OTP, xác thực đa yếu tố, cơ chế chống bot mạnh hoặc thao tác liên quan đến thanh toán thật không thuộc trọng tâm xử lý của đề tài. Các cơ chế như locator fallback và self-healing chỉ hỗ trợ một số thay đổi nhỏ của giao diện; khi website thay đổi mạnh về luồng nghiệp vụ, người dùng vẫn cần xem lại test case hoặc replay script.

1.4 Giải pháp đề xuất

Để đạt được các mục tiêu trên trong phạm vi đã xác định, đề tài đề xuất xây dựng một nền tảng web hỗ trợ kiểm thử tự động cho ứng dụng web, đặc biệt trên môi trường staging hoặc môi trường kiểm thử có kiểm soát. Thay vì yêu cầu người dùng trực tiếp viết script, hệ thống cho phép người dùng nhập prompt hoặc goal bằng ngôn ngữ tự nhiên để mô tả mục tiêu kiểm thử.

Quy trình tổng quát của hệ thống gồm các bước chính: người dùng tạo project kiểm thử, khai báo website mục tiêu thông qua base URL, nhập mô tả kiểm thử, sau đó hệ thống sử dụng mô hình ngôn ngữ lớn để sinh test case có cấu trúc. Trong phạm vi đề tài, Google Gemini Flash được sử dụng để hỗ trợ xử lý ngôn ngữ tự nhiên, sinh test case, hỗ trợ agent suy luận hành động và phân tích lỗi [4].

Sau khi test case được tạo, Browser-use agent nhận test case làm đầu vào và tương tác với website thông qua trình duyệt [5]. Playwright được sử dụng làm lớp điều khiển trình duyệt để mở trang, nhập liệu, nhấn nút, kiểm tra trạng thái và chạy lại script [3]. Trong quá trình chạy, hệ thống ghi nhận log, screenshot, action history, verdict và evidence để phục vụ việc xem lại kết quả.

Điểm trọng tâm của giải pháp là kết hợp giữa chế độ chạy linh hoạt bằng agent và chế độ chạy lại bằng replay script. Agent được dùng để khám phá và thực thi luồng kiểm thử ban đầu. Khi phiên chạy thành công, hệ thống chuẩn hóa lịch sử thao tác thành replay script để sử dụng cho các lần kiểm thử sau. Cách tiếp cận này giúp tận dụng khả năng linh hoạt của mô hình ngôn ngữ lớn và agent, đồng thời giảm chi phí và tăng tính ổn định khi kiểm thử hồi quy.

1.5 Kết quả đạt được

Sau quá trình thực hiện, đề tài đã xây dựng được một hệ thống web hỗ trợ kiểm thử tự động ứng dụng web với các chức năng chính phục vụ vòng đời kiểm thử từ tạo test case, chạy kiểm thử, lưu vết thực thi đến xem báo cáo kết quả.

Các kết quả chính đạt được bao gồm:

- Xây dựng giao diện web cho phép người dùng quản lý project, test case, test run, replay script, test data, test suite, object repository, evidence và report.
- Xây dựng backend API để quản lý dữ liệu nghiệp vụ, kiểm tra dữ liệu đầu vào, điều phối yêu cầu kiểm thử và lưu trữ kết quả thực thi.
- Tích hợp mô hình ngôn ngữ lớn vào quy trình kiểm thử để hỗ trợ sinh test case từ mô tả tự nhiên, sinh dữ liệu kiểm thử, hỗ trợ agent trong quá trình thực thi và gợi ý nguyên nhân khi test thất bại.
- Tích hợp Browser-use agent để thực thi kiểm thử theo goal hoặc test case trên trình duyệt.
- Tích hợp Playwright để điều khiển trình duyệt và chạy lại replay script cho các lần kiểm thử hồi quy.
- Lưu lại log, screenshot, action history, verdict, evidence và report nhằm hỗ trợ người dùng xem lại quá trình thực thi, truy vết kết quả và phân tích lỗi.
- Thiết kế quy trình kết hợp giữa agent-based execution và replay-based execution, trong đó agent được sử dụng cho lần chạy linh hoạt ban đầu, còn replay script được sử dụng cho các lần kiểm thử hồi quy sau.

- Xây dựng các cơ chế hỗ trợ bảo trì kiểm thử như object repository, locator fallback và state anchors ở mức phù hợp với phạm vi đề tài.

Nhìn chung, hệ thống đã hiện thực được luồng kiểm thử tự động cốt lõi: từ mô tả mục tiêu kiểm thử bằng ngôn ngữ tự nhiên, chuyển thành test case, thực thi trên trình duyệt, ghi nhận bằng chứng và hỗ trợ chạy lại trong các lần kiểm thử sau. Kết quả này là cơ sở để đánh giá khả năng ứng dụng của việc kết hợp mô hình ngôn ngữ lớn, agent và replay script trong bài toán kiểm thử tự động ứng dụng web.

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành tám chương, trình bày lần lượt từ bối cảnh đề tài, nền tảng kỹ thuật, phân tích yêu cầu, thiết kế hệ thống, hiện thực, đánh giá kết quả, tổ chức thực hiện cho đến kết luận và hướng phát triển.

Chương 1 trình bày tổng quan đề tài, bao gồm bối cảnh thực tiễn, vấn đề cần giải quyết, mục tiêu, phạm vi, giải pháp đề xuất và kết quả đạt được của khóa luận.

Chương 2 trình bày nền tảng kỹ thuật và các giải pháp liên quan, bao gồm kiểm thử phần mềm, kiểm thử tự động ứng dụng web, mô hình ngôn ngữ lớn, AI agent và các công cụ browser automation.

Chương 3 phân tích tổng quan hệ thống, nhóm người dùng, yêu cầu chức năng, yêu cầu phi chức năng và các use case chính của hệ thống.

Chương 4 trình bày kiến trúc và thiết kế hệ thống, bao gồm kiến trúc tổng thể, thiết kế dữ liệu ở mức cốt lõi, các luồng xử lý chính, cơ chế hỗ trợ bảo trì kiểm thử và các quyết định thiết kế quan trọng.

Chương 5 trình bày quá trình xây dựng và tích hợp hệ thống, bao gồm cách hiện thực frontend, backend API, cơ sở dữ liệu, agent-worker, cơ chế chạy test bằng agent, replay script, object repository, evidence và report.

Chương 6 trình bày thực nghiệm và đánh giá hệ thống. Nội dung chương này tập trung vào mục tiêu đánh giá, môi trường thực nghiệm,

case study OrangeHRM, kịch bản thử nghiệm, kết quả chạy test và mức độ đáp ứng các mục tiêu đã đặt ra.

Chương 7 trình bày cách tổ chức thực hiện đề tài, bao gồm phân chia công việc, quy trình làm việc, kế hoạch thực hiện, công cụ hỗ trợ và các khó khăn gặp phải trong quá trình phát triển.

Chương 8 trình bày kết luận và hướng phát triển. Chương này tóm tắt kết quả chính của khóa luận, nêu các hạn chế còn tồn tại, bài học kinh nghiệm và đề xuất các hướng mở rộng trong tương lai.

Chương 2

Nền tảng kỹ thuật và các giải pháp liên quan

2.1 Kiểm thử web và kiểm thử tự động

Kiểm thử phần mềm là hoạt động nhằm đánh giá chất lượng hệ thống, phát hiện lỗi và kiểm tra mức độ đáp ứng của phần mềm so với yêu cầu đã đặt ra. Theo ISTQB, kiểm thử không chỉ là chạy chương trình để tìm lỗi, mà còn bao gồm các hoạt động như phân tích yêu cầu, thiết kế ca kiểm thử, chuẩn bị dữ liệu, thực thi kiểm thử, ghi nhận kết quả và đánh giá kết quả thu được [6].

Đối với ứng dụng web, kiểm thử thường tập trung vào các chức năng mà người dùng thao tác trực tiếp trên trình duyệt, chẳng hạn như đăng nhập, đăng ký, nhập dữ liệu, tìm kiếm, điều hướng, gửi biểu mẫu và kiểm tra nội dung hiển thị. Trong phạm vi đề tài, hệ thống tập trung vào kiểm thử giao diện người dùng của ứng dụng web, đặc biệt là các luồng có thể thực hiện thông qua trình duyệt như mở trang, nhập dữ liệu, nhấn nút, điều hướng và xác nhận trạng thái sau thao tác.

Kiểm thử chức năng là loại kiểm thử nhằm xác định hệ thống có thực hiện đúng chức năng đã được đặc tả hay không. Trong khi đó, kiểm thử hồi quy là hoạt động kiểm tra lại các chức năng đã tồn tại sau khi hệ thống

có thay đổi, nhằm đảm bảo các chức năng cũ không bị ảnh hưởng bởi thay đổi mới. Hai loại kiểm thử này liên quan trực tiếp đến đề tài vì hệ thống hướng đến việc hỗ trợ tạo, chạy và lưu lại các kịch bản kiểm thử giao diện web để phục vụ các lần kiểm thử sau.

Kiểm thử tự động là việc sử dụng công cụ, script hoặc hệ thống phần mềm để thực hiện hoạt động kiểm thử thay cho thao tác thủ công. Đối với ứng dụng web, kiểm thử tự động thường dùng các framework có khả năng điều khiển trình duyệt, tìm phần tử trên trang và mô phỏng hành vi người dùng như nhấp chuột, nhập dữ liệu hoặc kiểm tra nội dung hiển thị. Cách tiếp cận này giúp giảm công sức lặp lại, tăng tính nhất quán và phù hợp với kiểm thử hồi quy.

Tuy nhiên, kiểm thử tự động truyền thống vẫn có một số khó khăn. Người dùng thường phải biết cách viết script, khai báo selector hoặc XPath, xử lý trạng thái bất đồng bộ và tổ chức dữ liệu kiểm thử. Khi giao diện thay đổi, selector cũ có thể không còn phù hợp, khiến test script bị lỗi dù chức năng nghiệp vụ vẫn hoạt động đúng. Những khó khăn này là cơ sở để đề tài kết hợp LLM, agent và replay script nhằm hỗ trợ quá trình tạo, thực thi và bảo trì kiểm thử tự động.

2.2 Một số công cụ kiểm thử web liên quan

Hiện nay có nhiều công cụ hỗ trợ kiểm thử tự động ứng dụng web, trong đó có thể kể đến Selenium, Playwright, Cypress và Katalon. Việc khảo sát các công cụ này giúp xác định các khái niệm nền tảng như browser automation, test script, locator, assertion, record-and-replay và bảo trì kịch bản kiểm thử.

Selenium là một framework kiểm thử web tự động được sử dụng rộng rãi. Thành phần quan trọng nhất của Selenium là Selenium WebDriver, cho phép điều khiển trình duyệt thông qua API lập trình. WebDriver có thể điều khiển trình duyệt tương tự cách người dùng tương tác với trình duyệt và được xem là một khuyến nghị của W3C [7]. Selenium giúp làm

rõ các khái niệm như browser driver, locator, thao tác trên DOM và cơ chế chờ phần tử.

Playwright là framework hỗ trợ kiểm thử end-to-end cho ứng dụng web. Playwright hỗ trợ các trình duyệt Chromium, WebKit và Firefox, đồng thời cung cấp các cơ chế như test runner, assertion, chạy song song, cấu hình trình duyệt và hỗ trợ môi trường CI [8]. Playwright cũng có cơ chế auto-waiting, hỗ trợ nhiều cách định vị phần tử, chụp screenshot, lưu trace và sinh report. Trong đề tài này, Playwright có vai trò quan trọng vì được sử dụng làm lớp điều khiển trình duyệt và chạy replay script.

Cypress là công cụ kiểm thử frontend và end-to-end cho ứng dụng web. Cypress cho phép người dùng viết các bài kiểm thử có khả năng truy cập trang web, tìm phần tử, thao tác với giao diện và kiểm tra trạng thái ứng dụng thông qua các lệnh như `cy.visit()`, `cy.contains()`, `.click()` và `.should()` [9]. Công cụ này cho thấy tầm quan trọng của cú pháp dễ đọc, assertion rõ ràng và cơ chế tự động chờ trong kiểm thử web.

Katalon và các công cụ low-code hướng đến việc giảm rào cản lập trình khi xây dựng kịch bản kiểm thử tự động [10]. Các công cụ này thường cung cấp giao diện trực quan để tạo test case, ghi lại thao tác, quản lý đối tượng giao diện và chạy lại kịch bản. Tuy nhiên, các kịch bản ghi lại vẫn có thể phụ thuộc vào selector, trạng thái giao diện hoặc luồng thao tác tại thời điểm ghi, nên việc bảo trì script vẫn là một vấn đề quan trọng.

Khảo sát các công cụ trên cho thấy hệ thống của đề tài cần có lớp điều khiển trình duyệt ổn định, khả năng ghi nhận log và screenshot, khả năng lưu lại script sau phiên chạy thành công, khả năng chạy lại script cho kiểm thử hồi quy và giao diện giúp người dùng thao tác dễ hơn. Bên cạnh đó, hệ thống cần tận dụng LLM để hỗ trợ những bước người dùng thường gặp khó khăn, chẳng hạn như sinh test case, sinh dữ liệu kiểm thử, hỗ trợ agent khám phá website và phân tích lỗi.

2.3 Mô hình ngôn ngữ lớn trong hệ thống kiểm thử web

Mô hình ngôn ngữ lớn, hay Large Language Model (LLM), là mô hình trí tuệ nhân tạo có khả năng xử lý ngôn ngữ tự nhiên và sinh đầu ra dưới nhiều dạng khác nhau như văn bản, mã nguồn hoặc dữ liệu có cấu trúc. Theo tài liệu của OpenAI, mô hình ngôn ngữ lớn có thể sinh phản hồi văn bản, mã nguồn, dữ liệu JSON có cấu trúc và các nội dung gần với cách con người viết [11].

Trong quy trình kiểm thử của hệ thống, LLM được sử dụng ở nhiều bước khác nhau. Thứ nhất, LLM hỗ trợ chuyển prompt hoặc goal của người dùng thành test case có cấu trúc, bao gồm tiêu đề, mục tiêu, các bước dự kiến, dữ liệu liên quan và kết quả mong đợi. Thứ hai, LLM hỗ trợ sinh test data như dữ liệu hợp lệ, không hợp lệ, thiếu trường hoặc dữ liệu biên. Thứ ba, LLM hỗ trợ agent quan sát trạng thái hiện tại của website và xác định hành động tiếp theo trong quá trình chạy kiểm thử. Thứ tư, LLM hỗ trợ gợi ý nguyên nhân khi test thất bại dựa trên log, screenshot metadata, action history, error message và trạng thái cuối cùng của test run.

Prompt là đầu vào quan trọng trong các bước sử dụng LLM. Nếu người dùng cung cấp prompt có mục tiêu, dữ liệu, điều kiện kiểm tra và bối cảnh rõ ràng, hệ thống sẽ có nhiều thông tin hơn để sinh test case và hỗ trợ agent chạy đúng hướng. Ngoài prompt của người dùng, hệ thống còn có thể sử dụng thông tin từ project, base URL, scan result, runtime config và trạng thái giao diện hiện tại làm ngữ cảnh bổ sung. Tuy nhiên, chất lượng sinh test case và khả năng agent chạy đúng mục tiêu vẫn phụ thuộc vào độ rõ của prompt và mức đầy đủ của ngữ cảnh đầu vào.

Trong hệ thống kiểm thử tự động, dữ liệu có cấu trúc giúp các thành phần trao đổi dữ liệu nhất quán. Các dữ liệu như test case, test step, replay script, assertion, test data, action history và error analysis cần được

biểu diễn rõ ràng để hệ thống có thể lưu trữ, kiểm tra và thực thi. Vì vậy, khi sử dụng LLM, hệ thống cần yêu cầu mô hình sinh đầu ra theo định dạng có cấu trúc, chẳng hạn như JSON. Theo tài liệu về Structured Outputs, mô hình có thể được ràng buộc để sinh phản hồi bám theo một JSON Schema đã định nghĩa [12].

Tuy vậy, dữ liệu đúng schema chưa chắc đã đúng về mặt nghiệp vụ. Một test case có thể đúng định dạng nhưng vẫn thiếu điều kiện kiểm tra, dữ liệu kiểm thử có thể hợp lệ về cấu trúc nhưng chưa phù hợp với website, hoặc một action có thể đúng schema nhưng chọn sai phần tử. Vì vậy, hệ thống vẫn cần bước chạy thử, ghi evidence và cho phép người dùng xem lại kết quả.

2.4 AI agent và browser automation

Trong hệ thống, AI agent là thành phần tiếp nhận mục tiêu kiểm thử, quan sát trạng thái giao diện, sử dụng LLM để suy luận hành động tiếp theo và thực thi thao tác thông qua công cụ điều khiển trình duyệt. Khác với test script truyền thống vốn thường chạy theo danh sách bước cố định, agent hoạt động theo vòng lặp quan sát – suy luận – hành động. Cách tiếp cận này giúp agent linh hoạt hơn khi cần khám phá giao diện hoặc xử lý một số thay đổi nhỏ trong quá trình thực thi.

Browser-use được sử dụng để tổ chức quá trình chạy agent trên trình duyệt, còn Playwright được sử dụng làm lớp browser automation. Khi nhận test case làm đầu vào, agent gọi LLM để phân tích mục tiêu kiểm thử, quan sát giao diện hiện tại và xác định thao tác phù hợp. Sau đó, hệ thống thực hiện các thao tác như mở trang, nhập dữ liệu, nhấn nút, điều hướng hoặc kiểm tra trạng thái. Khi phiên chạy bằng agent thành công, hệ thống chuyển lịch sử hành động thành replay script và lưu lại để chạy bằng Playwright trong các lần kiểm thử sau.

Browser automation là lớp kỹ thuật giúp hệ thống thao tác với trình duyệt. Lớp này chịu trách nhiệm mở trang web, tìm phần tử, nhập dữ

liệu, nhấp chuột, kiểm tra nội dung, chụp screenshot và ghi nhận thông tin phục vụ báo cáo. Trong hệ thống đề xuất, browser automation phục vụ cả hai chế độ thực thi: chạy linh hoạt bằng agent và chạy lại bằng replay script.

Việc kết hợp AI agent và browser automation tạo ra hai khả năng bổ sung cho nhau. Agent hỗ trợ khám phá website và chuyển test case thành chuỗi hành động phù hợp trong lần chạy ban đầu. Replay script giúp chạy lại chuỗi hành động đã ghi một cách ổn định hơn trong kiểm thử hồi quy. Tuy nhiên, do agent phụ thuộc vào trạng thái giao diện và kết quả suy luận của LLM, quá trình chạy bằng agent có thể không hoàn toàn xác định giữa các lần thực thi. Vì vậy, hệ thống cần chuẩn hóa lịch sử hành động thành replay script sau phiên chạy thành công.

2.5 Định vị giải pháp đề xuất

Trên cơ sở các nền tảng kỹ thuật đã trình bày, giải pháp của đề tài hướng đến một nền tảng web hỗ trợ quy trình kiểm thử tự động ứng dụng web từ đầu đến cuối. Người dùng có thể tạo project, nhập mục tiêu kiểm thử, sinh test case, chạy test bằng agent, lưu replay script, chạy lại script, quản lý test data và xem report.

Hệ thống kết hợp các hướng tiếp cận chính gồm kiểm thử tự động ứng dụng web, LLM, AI agent và browser automation. LLM hỗ trợ sinh test case, sinh dữ liệu kiểm thử, hỗ trợ agent trong quá trình thực thi và phân tích lỗi. Browser-use agent hỗ trợ khám phá website và thực thi test case ở lần chạy linh hoạt ban đầu. Playwright đảm nhiệm vai trò điều khiển trình duyệt và chạy lại replay script.

Điểm khác biệt chính của hệ thống là workflow kết hợp giữa agent-based execution và replay-based execution. Agent được sử dụng để khám phá web từ test case và tạo ra lịch sử hành động phù hợp. Sau đó, lịch sử này được chuẩn hóa thành replay script để lưu lại và chạy trong các lần kiểm thử sau. Ngoài ra, hệ thống còn lưu log, screenshot, evidence, object

repository và report để hỗ trợ truy vết kết quả và bảo trì test case.

Như vậy, chương này đã trình bày các nền tảng kỹ thuật cần thiết cho đề tài, bao gồm kiểm thử web, kiểm thử tự động, công cụ kiểm thử web, LLM, prompt, dữ liệu có cấu trúc, AI agent và browser automation. Các nội dung này là cơ sở để phân tích yêu cầu, thiết kế kiến trúc và hiện thực hệ thống ở các chương tiếp theo.

Chương 3

Phân tích yêu cầu hệ thống

3.1 Tổng quan hệ thống

Hệ thống được xây dựng là một nền tảng web hỗ trợ kiểm thử tự động ứng dụng web. Mục tiêu chính của hệ thống là giúp người dùng tạo, quản lý, thực thi và xem lại kết quả các ca kiểm thử cho website mục tiêu mà không cần trực tiếp viết mã kiểm thử phức tạp ngay từ đầu. Thay vì yêu cầu người dùng tự viết script bằng các framework kiểm thử truyền thống, hệ thống cho phép người dùng mô tả mục tiêu kiểm thử bằng ngôn ngữ tự nhiên, sau đó sử dụng AI agent kết hợp với browser automation để thực hiện kiểm thử trên trình duyệt.

Về mặt sử dụng, hệ thống hỗ trợ người dùng tạo project kiểm thử, khai báo website mục tiêu, scan/crawl website, tạo test case từ prompt hoặc goal, sinh dữ liệu kiểm thử, chạy test case bằng agent, lưu lại lịch sử thực thi, sinh replay script và chạy lại các kịch bản đã ghi nhận. Trong quá trình chạy bằng agent, hệ thống có thể hỗ trợ xử lý một số thay đổi nhỏ của giao diện ở mức locator fallback. Sau mỗi lần chạy, hệ thống lưu log, screenshot, action history, verdict và evidence để người dùng có thể xem lại quá trình thực thi, đánh giá kết quả và phân tích nguyên nhân khi ca kiểm thử thất bại.

Ở mức phân tích yêu cầu, hệ thống gồm các thành phần logic chính

như Bảng 3.1. Các công nghệ cụ thể dùng để hiện thực từng thành phần sẽ được trình bày ở Chương 4 và Chương 5.

Bảng 3.1: Các thành phần logic chính của hệ thống

Thành phần	Vai trò
Giao diện người dùng	Cung cấp giao diện để người dùng thao tác với project, test case, test run, replay script, test data, test suite, object repository, evidence và báo cáo kết quả.
Bộ xử lý nghiệp vụ	Tiếp nhận yêu cầu từ giao diện, kiểm tra dữ liệu đầu vào, kiểm tra quyền truy cập, quản lý dữ liệu nghiệp vụ và điều phối yêu cầu thực thi kiểm thử.
Kho dữ liệu	Lưu trữ thông tin người dùng, project, scan result, test case, test data, replay script, test run, evidence, report và object repository.
Thành phần thực thi kiểm thử	Thực hiện các tác vụ liên quan đến trình duyệt như chạy test bằng agent, chạy replay script, có thể hỗ trợ locator fallback ở mức agent run, ghi nhận log, screenshot, action history và trả kết quả về hệ thống.
Thành phần AI/LLM	Hỗ trợ sinh test case, sinh dữ liệu kiểm thử, hỗ trợ agent suy luận hành động và gợi ý nguyên nhân khi test thất bại.

Hệ thống không được thiết kế như một công cụ chỉ chạy script có sẵn, mà là một nền tảng hỗ trợ quy trình kiểm thử tự động từ giai đoạn chuẩn bị, thực thi đến phân tích kết quả. Người dùng có thể bắt đầu từ một mục tiêu kiểm thử ở mức nghiệp vụ, sau đó hệ thống hỗ trợ chuyển mục tiêu này thành test case, thực thi trên trình duyệt và ghi nhận kết quả để tái sử dụng trong các lần kiểm thử sau.

3.2 Nhóm người dùng

Trong phạm vi hiện tại, hệ thống tập trung vào nhóm người dùng đã xác thực, bao gồm QA/tester, developer hoặc người dùng nghiệp vụ có nhu cầu tạo và chạy kiểm thử tự động cho website. Hệ thống chưa phân tách vai trò quản trị viên riêng; do đó, các chức năng chính được thiết kế xoay quanh người dùng trực tiếp thao tác với project, test case, test run và báo cáo kết quả.

Người dùng hệ thống có thể thực hiện các thao tác chính sau:

- Đăng ký, đăng nhập và đăng xuất khỏi hệ thống.
- Tạo và quản lý project kiểm thử.
- Khai báo website mục tiêu thông qua base URL.
- Scan/crawl website để ghi nhận các trang và luồng điều hướng có thể truy cập.
- Tạo test case từ mô tả bằng ngôn ngữ tự nhiên.
- Sinh và quản lý dữ liệu kiểm thử.
- Chạy test case bằng AI agent hoặc replay script.
- Chạy test case với nhiều bộ dữ liệu.
- Quản lý test suite để gom nhóm nhiều test case và chạy theo lô.
- Quản lý object repository để tái sử dụng thông tin định vị phần tử.
- Xem log, screenshot, evidence, báo cáo và phân tích nguyên nhân thất bại.

Việc tập trung vào một nhóm người dùng đã xác thực giúp hệ thống đơn giản hơn về mặt phân quyền trong giai đoạn hiện tại. Thay vì xây

dựng nhiều vai trò phức tạp, hệ thống ưu tiên hoàn thiện luồng nghiệp vụ cốt lõi của kiểm thử tự động: tạo project, tạo test case, chạy test, ghi nhận kết quả, phát lại kịch bản và hỗ trợ phân tích lỗi.

3.3 Yêu cầu chức năng

Yêu cầu chức năng được nhóm theo các nhóm chức năng chính của hệ thống. Mức ưu tiên được phân loại thành hai nhóm: *Must* là yêu cầu bắt buộc cần có để hệ thống vận hành luồng kiểm thử cốt lõi; *Should* là yêu cầu nên có nhằm mở rộng khả năng sử dụng, bảo trì và hỗ trợ kiểm thử hồi quy.

Bảng 3.2: Danh sách yêu cầu chức năng của hệ thống

Mã	Nhóm chức năng	Mô tả yêu cầu	Ưu tiên
FR01	Quản lý người dùng và xác thực	Cho phép người dùng đăng ký, đăng nhập, đăng xuất; kiểm tra dữ liệu đầu vào; lưu mật khẩu ở dạng đã băm; duy trì phiên đăng nhập để truy cập hệ thống.	Must
FR02	Quản lý project kiểm thử	Cho phép người dùng tạo, xem, cập nhật project; khai báo tên project, mô tả, base URL và cấu hình liên quan.	Must
FR03	Scan/Crawl website	Cho phép khởi tạo scan/crawl từ base URL; ghi nhận trạng thái scan, sitemap, interaction map và lỗi nếu có; cung cấp ngữ cảnh cho việc tạo test case và chạy agent.	Should
FR04	Quản lý test case	Cho phép tạo test case từ prompt hoặc goal bằng ngôn ngữ tự nhiên; sinh tiêu đề, mục tiêu và test plan; lưu, chỉnh sửa và xem lịch sử chạy.	Must
FR05	Điều kiện kiểm tra kết quả	Hỗ trợ thiết lập assertion hoặc state anchor để xác định test case pass/fail, ví dụ kiểm tra URL, văn bản, phần tử hiển thị hoặc trạng thái lỗi.	Must

Mã	Nhóm chức năng	Mô tả yêu cầu	Ưu tiên
FR06	Sinh và quản lý test data	Cho phép tạo, lưu, sinh và quản lý dữ liệu kiểm thử; hỗ trợ dữ liệu hợp lệ, không hợp lệ, thiếu trường và dữ liệu biên.	Should
FR07	Quản lý test suite	Cho phép tạo test suite, thêm hoặc bỏ test case, sắp xếp thứ tự chạy, chạy suite theo lô và tổng hợp kết quả.	Should
FR08	Thực thi test bằng agent	Cho phép chạy test case bằng Browser-use agent; agent quan sát giao diện, suy luận hành động tiếp theo, thực hiện thao tác kiểm thử trên trình duyệt, đồng thời ghi nhận log, screenshot, action history và trả kết quả về hệ thống.	Must
FR09	Sinh và quản lý replay script	Tự động sinh replay script từ phiên chạy agent thành công; lưu script dạng JSON; cho phép xem, chỉnh sửa và quản lý script.	Must
FR10	Chạy replay script	Cho phép chạy lại replay script mà không cần gọi LLM ở từng bước; hỗ trợ các thao tác như navigate, fill, click, assertion và ghi nhận evidence.	Must
FR11	Chạy test với nhiều bộ dữ liệu	Cho phép chạy cùng một test case hoặc replay script với nhiều dòng dữ liệu và lưu kết quả riêng cho từng dataset run.	Should
FR12	Object repository	Lưu trữ các object giao diện theo project, gồm selector collection và attribute snapshot để hỗ trợ việc định vị phần tử khi agent thực thi kiểm thử và khi hệ thống cần tái sử dụng thông tin phần tử trong các lần chạy sau.	Should
FR13	Self-healing ở mức locator fallback	Khi chạy test bằng agent, nếu locator hoặc cách nhận diện phần tử ban đầu không còn phù hợp, agent có thể quan sát lại giao diện và thử cách định vị khác để tiếp tục thực thi. Cơ chế này chỉ hỗ trợ một số thay đổi nhỏ của giao diện và không thay thế việc sửa lại test case khi luồng nghiệp vụ thay đổi lớn.	Should

Mã	Nhóm chức năng	Mô tả yêu cầu	Ưu tiên
FR14	Log, evidence, báo cáo và phân tích lỗi	Ghi nhận log, screenshot, action history, evidence, verdict và error message; hiển thị báo cáo kết quả và hỗ trợ gợi ý nguyên nhân khi test thất bại.	Must

Nhìn chung, nhóm yêu cầu bắt buộc tập trung vào luồng kiểm thử cốt lõi: quản lý project, tạo test case, chạy test bằng agent, sinh replay script, chạy replay và xem báo cáo. Các yêu cầu còn lại đóng vai trò mở rộng, giúp hệ thống phù hợp hơn với kiểm thử hồi quy và bảo trì kịch bản kiểm thử.

3.4 Yêu cầu phi chức năng

Bên cạnh yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng để đảm bảo khả năng sử dụng, vận hành ổn định, truy vết kết quả và kiểm soát an toàn dữ liệu trong quá trình kiểm thử.

Bảng 3.3: Danh sách yêu cầu phi chức năng của hệ thống

Mã	Nhóm yêu cầu	Mô tả yêu cầu
NFR01	Tính dễ sử dụng	Giao diện cần giúp người dùng tạo project, nhập mục tiêu kiểm thử, tạo test case, chạy test và xem báo cáo trong một luồng thao tác rõ ràng.
NFR02	Độ ổn định	Hệ thống cần xử lý các lỗi như trang không tải được, trình duyệt timeout, locator thất bại, LLM phản hồi không hợp lệ hoặc thành phần thực thi gặp lỗi; dữ liệu test run và evidence đã ghi nhận không được bị mất khi một bước chạy thất bại.
NFR03	Hiệu năng vận hành	Mỗi test run cần có timeout cấu hình được. Thành phần thực thi cần giới hạn số phiên trình duyệt chạy đồng thời. Replay script nên được dùng để giảm thời gian chạy so với chế độ agent trong các lần kiểm thử hồi quy.

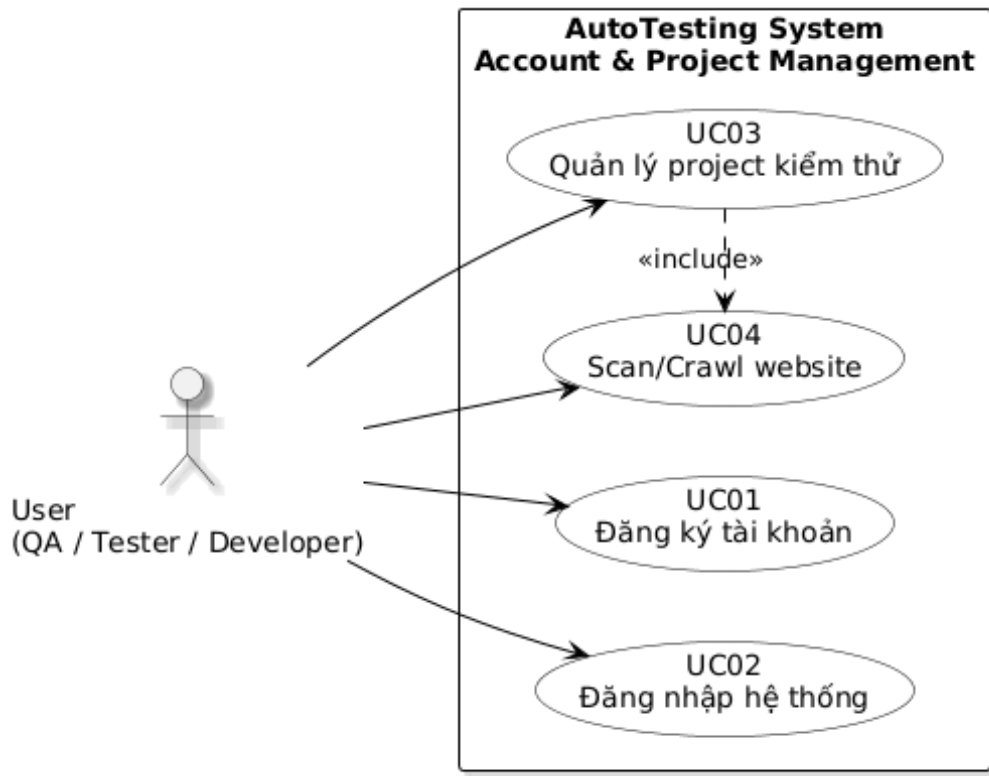
Mã	Nhóm yêu cầu	Mô tả yêu cầu
NFR04	Tối ưu chi phí vận hành	Hệ thống nên dùng agent cho giai đoạn tạo hoặc khám phá luồng kiểm thử ban đầu, sau đó dùng replay script để giảm số lần gọi LLM trong các lần kiểm thử hồi quy.
NFR05	Bảo mật	Hệ thống không lưu mật khẩu thô, không để lộ API key ở frontend hoặc log, kiểm soát quyền truy cập project, dataset, evidence và replay script.
NFR06	Khả năng mở rộng	Kiến trúc cần cho phép mở rộng runtime config, browser type, model LLM, loại assertion, dataset run, test suite, object repository và số lượng thành phần thực thi khi nhu cầu chạy test tăng lên.
NFR07	Khả năng truy vết	Mỗi lần chạy test cần lưu được trạng thái, verdict, log, screenshot, action history, error message và evidence liên quan để người dùng có thể xem lại quá trình thực thi và xác định bước xảy ra lỗi.
NFR08	Kiểm soát agent và ngữ cảnh LLM	Hệ thống cần xem nội dung website là dữ liệu không tin cậy, giới hạn domain được phép truy cập khi chạy test, không đưa token hoặc API key vào prompt và không cho phép nội dung website ghi đè quy tắc vận hành của agent.

Các yêu cầu phi chức năng trên đóng vai trò định hướng cho thiết kế kiến trúc ở Chương 4. Đặc biệt, việc tách các thành phần giao diện, xử lý nghiệp vụ, lưu trữ dữ liệu và thực thi kiểm thử giúp hệ thống dễ kiểm soát tài nguyên, giới hạn phạm vi chạy test, truy vết kết quả và mở rộng khi cần thiết.

3.5 Sơ đồ use case

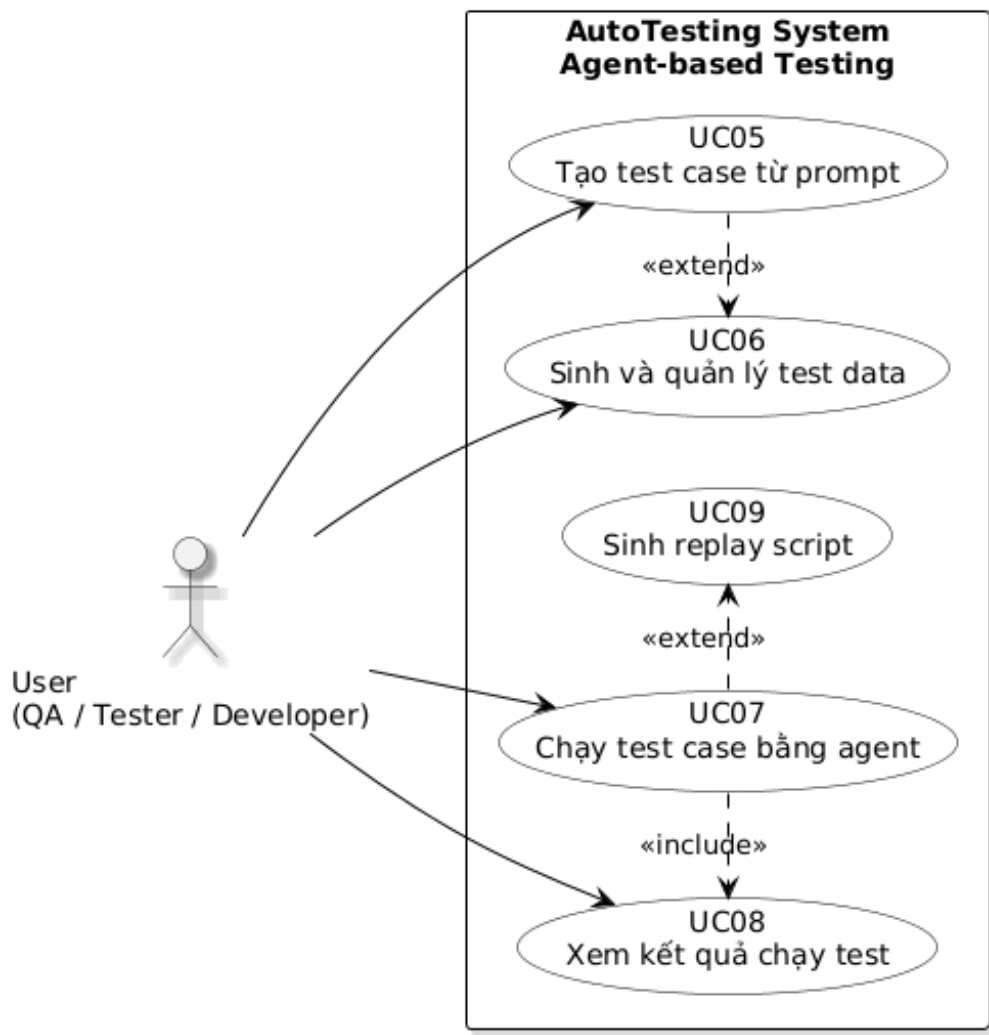
Do hệ thống bao gồm nhiều nhóm chức năng khác nhau, sơ đồ use case được chia thành nhiều hình nhỏ để đảm bảo tính dễ đọc. Các sơ đồ use case được tổ chức theo từng nhóm nghiệp vụ chính, bao gồm quản lý tài khoản

và project, tạo và thực thi test case bằng agent, chạy lại replay script với dữ liệu kiểm thử và test suite, cũng như nhóm chức năng evidence, báo cáo và phân tích lỗi.



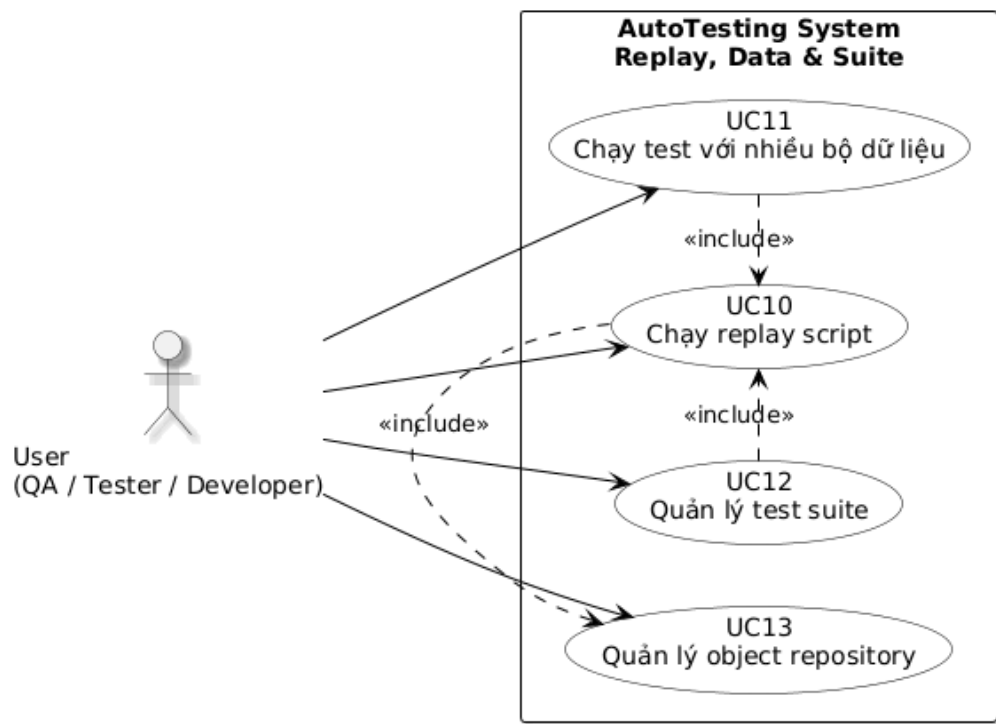
Hình 3.1: Use case quản lý tài khoản và project

Nhóm use case quản lý tài khoản và project mô tả các thao tác nền tảng đầu tiên của người dùng khi sử dụng hệ thống. Người dùng có thể đăng ký tài khoản, đăng nhập, tạo và quản lý project kiểm thử, sau đó thực hiện scan/crawl website để ghi nhận các trang và luồng điều hướng có thể truy cập.



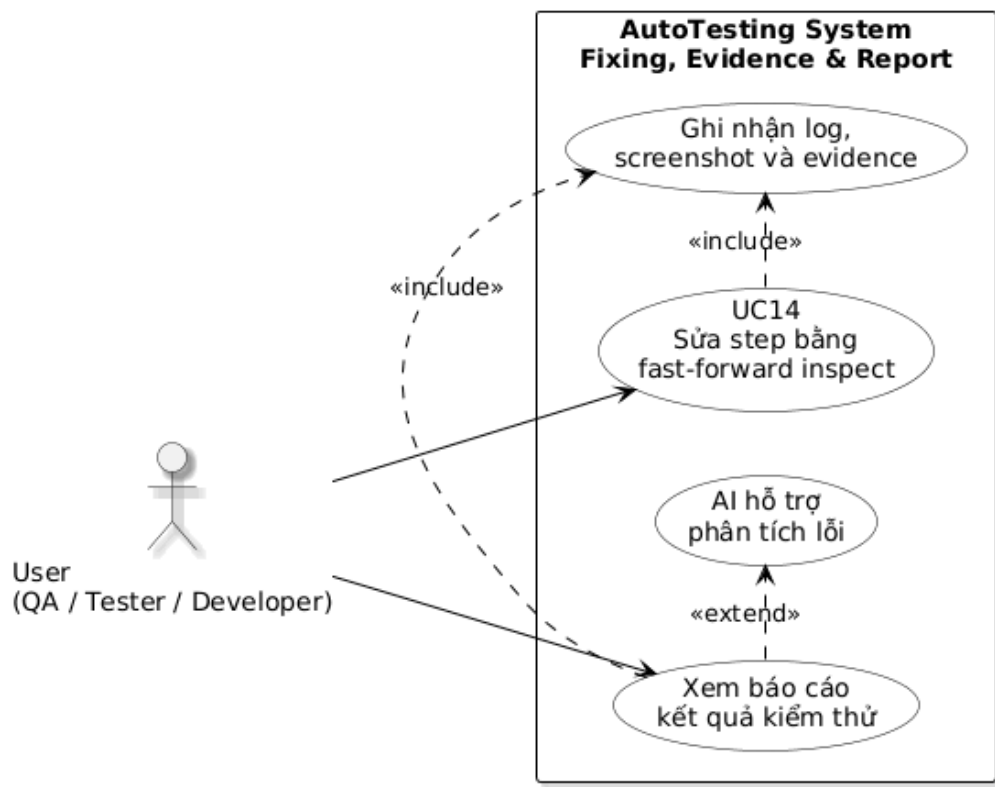
Hình 3.2: Use case tạo test case và chạy bằng agent

Nhóm use case tạo test case và chạy bằng agent thể hiện luồng kiểm thử chính của hệ thống. Người dùng nhập mục tiêu kiểm thử bằng ngôn ngữ tự nhiên để hệ thống sinh test case, có thể sinh hoặc quản lý test data, sau đó chạy test case bằng Browser-use agent. Trong quá trình thực thi, agent có thể quan sát giao diện, suy luận hành động tiếp theo và xử lý một số thay đổi nhỏ của giao diện ở mức locator fallback. Sau khi agent thực thi thành công, hệ thống có thể sinh replay script để phục vụ các lần chạy lại sau.



Hình 3.3: Use case replay script, test data và test suite

Nhóm use case replay script, test data và test suite mô tả khả năng tái sử dụng kịch bản kiểm thử sau khi agent đã chạy thành công. Người dùng có thể chạy replay script, chạy cùng một test case với nhiều bộ dữ liệu và gom nhiều test case vào test suite để phục vụ kiểm thử hồi quy.



Hình 3.4: Use case evidence, báo cáo và phân tích lỗi

Nhóm use case evidence, báo cáo và phân tích lỗi tập trung vào giai đoạn sau khi test được thực thi. Hệ thống ghi nhận log, screenshot, action history, verdict và evidence để người dùng xem lại quá trình chạy test. Khi test thất bại, hệ thống hiển thị thông tin lỗi và có thể hỗ trợ gợi ý nguyên nhân để người dùng cập nhật test case, replay script hoặc object repository nếu cần.

3.6 Đặc tả use case chính

Bảng 3.4 trình bày đặc tả rút gọn của các use case chính. Bảng tập trung vào mục tiêu và kết quả đầu ra để giữ chương ở mức phân tích yêu cầu, còn luồng xử lý chi tiết sẽ được trình bày ở Chương 4.

Bảng 3.4: Đặc tả rút gọn các use case chính

Mã	Use case	Mục tiêu	Kết quả đầu ra
UC01	Đăng ký tài khoản	Cho phép người dùng tạo tài khoản bằng email, họ tên và mật khẩu.	Tài khoản mới được tạo nếu dữ liệu hợp lệ.
UC02	Đăng nhập hệ thống	Xác thực người dùng để truy cập dashboard và các chức năng kiểm thử.	Phiên đăng nhập hợp lệ được tạo.
UC03	Quản lý project kiểm thử	Tạo và quản lý project kiểm thử gắn với base URL của website mục tiêu.	Project được lưu và có thể dùng để tạo test case, scan và chạy test.
UC04	Scan/Crawl website	Ghi nhận các trang và luồng điều hướng có thể truy cập từ base URL.	Sitemap, interaction map và trạng thái scan được lưu.
UC05	Tạo test case từ prompt	Chuyển mô tả ngôn ngữ tự nhiên thành test case có title, goal và test plan.	Test case được sinh và lưu vào project.
UC06	Sinh và quản lý test data	Tạo hoặc sinh dữ liệu kiểm thử để dùng trong test case và replay script.	Dataset được lưu và có thể gắn với test case hoặc replay script.
UC07	Chạy test case bằng agent	Thực thi test case bằng Browser-use agent và browser automation trên website mục tiêu; trong một số trường hợp, agent có thể hỗ trợ xử lý thay đổi nhỏ của giao diện ở mức locator fallback.	Test run, log, screenshot, action history, verdict và evidence được lưu.
UC08	Xem kết quả chạy test	Cho phép người dùng xem trạng thái, verdict, timeline, log và screenshot của test run.	Báo cáo chi tiết của test run được hiển thị.
UC09	Sinh replay script	Tạo replay script từ lịch sử hành động sau khi agent chạy thành công.	Replay script dạng JSON được lưu cho lần chạy sau.

Mã	Use case	Mục tiêu	Kết quả đầu ra
UC10	Chạy replay script	Chạy lại kịch bản kiểm thử mà không cần agent suy luận lại toàn bộ luồng.	Replay run được lưu với trạng thái pass, fail hoặc error.
UC11	Chạy test với nhiều bộ dữ liệu	Chạy một test case hoặc replay script với nhiều dòng dữ liệu khác nhau.	Kết quả được lưu riêng cho từng bộ dữ liệu.
UC12	Quản lý test suite	Gom nhiều test case vào một suite và chạy theo lô.	Kết quả chạy suite được tổng hợp.
UC13	Quản lý object repository	Lưu và tái sử dụng thông tin định vị phần tử giao diện để hỗ trợ agent thực thi kiểm thử và xử lý locator fallback khi cần.	Object, selector collection và attribute snapshot được lưu theo project.
UC14	Xem báo cáo và phân tích lỗi	Hiển thị kết quả kiểm thử, evidence và gợi ý nguyên nhân thất bại.	Người dùng có cơ sở để sửa test case, cập nhật object repository hoặc chạy lại test.

Các use case trên bao phủ vòng đời kiểm thử mà hệ thống hướng đến: từ khởi tạo tài khoản và project, tạo test case, chạy test bằng agent, sinh replay script, chạy lại với dữ liệu kiểm thử, đến phân tích kết quả và hỗ trợ bảo trì kịch bản kiểm thử.

3.7 Kết chương

Chương này đã trình bày phân tích yêu cầu của hệ thống kiểm thử tự động ứng dụng web. Nội dung chương bao gồm tổng quan hệ thống, nhóm người dùng, yêu cầu chức năng, yêu cầu phi chức năng, sơ đồ use case và đặc tả rút gọn các use case chính. Qua đó, chương làm rõ phạm vi chức năng mà hệ thống cần đáp ứng, từ đăng ký và đăng nhập, quản lý project, scan/crawl website, tạo test case từ prompt, chạy test bằng agent, sinh replay script, chạy với dataset, quản lý object repository, đến phân tích lỗi và báo cáo kết quả.

Các yêu cầu được phân tích trong chương này là cơ sở để thiết kế kiến trúc, mô hình dữ liệu và các luồng xử lý kỹ thuật ở Chương 4.

Chương 4

Kiến trúc và thiết kế hệ thống

4.1 Kiến trúc tổng thể hệ thống

Từ các yêu cầu đã phân tích ở Chương 3, hệ thống được thiết kế theo hướng tách biệt giữa giao diện người dùng, backend quản lý nghiệp vụ, cơ sở dữ liệu và agent-worker thực thi kiểm thử trên trình duyệt. Mục tiêu của kiến trúc là hỗ trợ quy trình kiểm thử tự động ứng dụng web, bao gồm tạo project, tạo test case từ mô tả tự nhiên, chạy test bằng agent, lưu evidence, sinh replay script và chạy lại kịch bản kiểm thử.

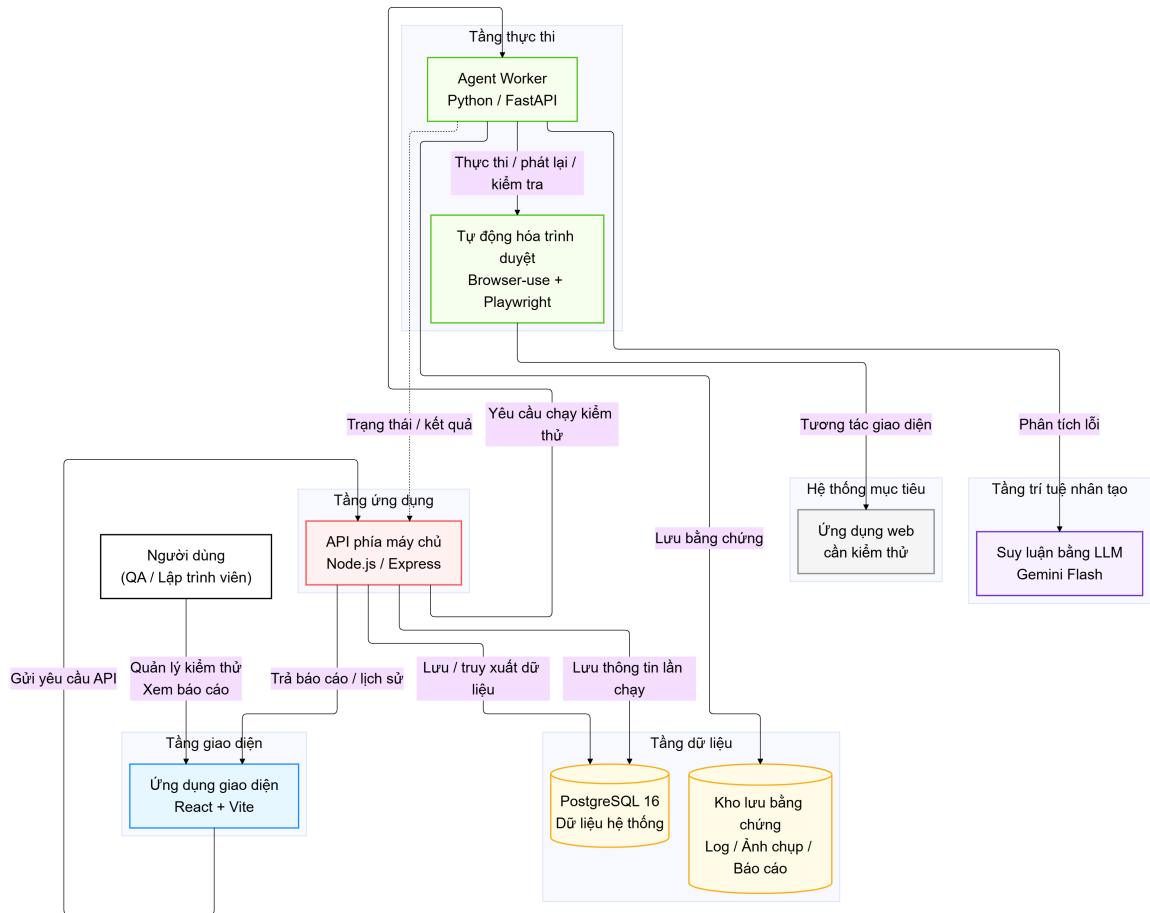
Các mục tiêu thiết kế chính gồm:

- **Tách biệt trách nhiệm:** frontend xử lý giao diện, backend xử lý nghiệp vụ và điều phối, agent-worker xử lý browser automation.
- **Hỗ trợ kiểm thử bằng ngôn ngữ tự nhiên:** người dùng có thể mô tả mục tiêu kiểm thử bằng prompt hoặc goal.
- **Kết hợp agent và replay script:** agent dùng cho lần chạy linh hoạt ban đầu, replay script dùng cho các lần kiểm thử hồi quy.
- **Lưu vết đầy đủ:** mỗi lần chạy cần có trạng thái, verdict, log, screenshot, evidence và thông tin lỗi.

- **Tăng khả năng bảo trì:** hệ thống hỗ trợ object repository, state anchors và self-healing ở mức locator fallback trong quá trình agent thực thi.
- **Kiểm soát chi phí và tài nguyên:** giảm số lần gọi LLM khi đã có replay script ổn định và giới hạn số phiên trình duyệt chạy đồng thời.

4.1.1 Kiến trúc component cấp hệ thống

Kiến trúc tổng thể của hệ thống gồm các thành phần chính: frontend, backend API, PostgreSQL database, agent-worker, Browser-use, Playwright, LLM và website mục tiêu. Các thành phần phối hợp với nhau thông qua API và callback để hoàn thành quy trình kiểm thử.



Hình 4.1: Kiến trúc tổng thể của hệ thống kiểm thử tự động ứng dụng web

Người dùng thao tác trên frontend để quản lý project, test case, dữ liệu kiểm thử và báo cáo. Frontend gửi yêu cầu đến backend API. Backend kiểm tra quyền truy cập, lưu dữ liệu nghiệp vụ vào PostgreSQL và gửi yêu cầu thực thi sang agent-worker. Agent-worker sử dụng Browser-use, Playwright và LLM để chạy test trên trình duyệt, sau đó callback kết quả về backend.

4.1.2 Công nghệ sử dụng

Bảng 4.1 tóm tắt các công nghệ chính được sử dụng trong hệ thống.

Bảng 4.1: Công nghệ sử dụng theo thành phần

Thành phần	Công nghệ	Vai trò
Frontend	ReactJS, Vite, shadcn/ui, Tailwind CSS	Cung cấp giao diện quản lý project, test case, test suite, test data, object repository và report.
Backend API	Node.js, Express.js	Xử lý API, quản lý dữ liệu nghiệp vụ, điều phối yêu cầu chạy test và nhận callback từ worker.
Database	PostgreSQL 16	Lưu trữ user, project, test case, replay script, test run, evidence, report, dataset, test suite và các dữ liệu hỗ trợ quá trình kiểm thử.
Agent-worker	Python, FastAPI	Nhận yêu cầu thực thi, khởi tạo trình duyệt, chạy agent hoặc replay script và trả kết quả về backend.
Browser automation	Browser-use, Playwright	Browser-use hỗ trợ goal-based agent; Playwright điều khiển trình duyệt và chạy replay script.
AI/LLM	Google Gemini Flash	Hỗ trợ xử lý mô tả tự nhiên, sinh test case, hỗ trợ agent suy luận hành động và gợi ý phân tích lỗi.

4.1.3 Luồng giao tiếp tổng quan

Về tổng thể, frontend gửi yêu cầu nghiệp vụ đến backend. Backend lưu dữ liệu vào database và gửi yêu cầu chạy test sang worker khi cần thực thi. Worker chạy test trên trình duyệt, ghi nhận log, screenshot, action history và callback kết quả về backend. Backend cập nhật test run, evidence summary, replay script nếu có và trả dữ liệu để frontend hiển thị

báo cáo.

4.2 Thiết kế dữ liệu và lưu trữ

Cơ sở dữ liệu được thiết kế để phục vụ vòng đời kiểm thử tự động, từ tạo project, tạo test case, chạy test, lưu evidence, sinh replay script đến phân tích lỗi. PostgreSQL 16 được sử dụng làm cơ sở dữ liệu chính.

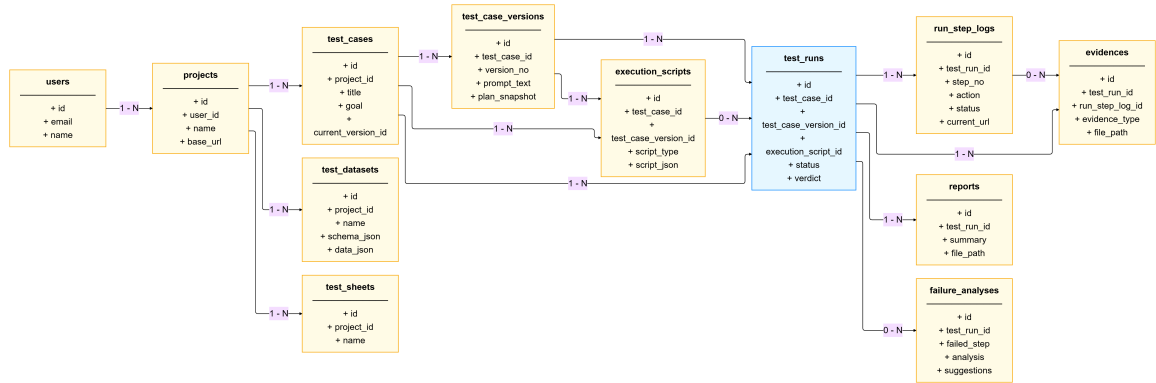
Trong chương này, phần thiết kế dữ liệu không trình bày toàn bộ schema vật lý của hệ thống. Thay vào đó, nội dung tập trung vào các bảng cốt lõi giúp người đọc hiểu dữ liệu chính được tổ chức và liên kết như thế nào trong vòng đời kiểm thử. Các bảng phụ trợ như cấu hình runtime, phiên đăng nhập, dữ liệu phân loại, lịch sử sinh dữ liệu bằng AI, test suite hoặc các bảng phục vụ vận hành được mô tả ở mức tổng quan và sẽ được trình bày chi tiết hơn ở Chương 5 nếu cần.

Các nguyên tắc thiết kế dữ liệu gồm:

- **Truy vết đầy đủ:** mỗi test run cần liên kết được với project, test case, replay script, evidence và người kích hoạt.
- **Tách dữ liệu theo vòng đời kiểm thử:** dữ liệu được tổ chức xoay quanh project, test case, replay script, test run, step log, evidence và object repository.
- **Hỗ trợ dữ liệu bán cấu trúc:** replay script, action history, evidence summary, runtime config hoặc thông tin hỗ trợ định vị phần tử có thể được lưu dạng JSON/JSONB để dễ mở rộng.
- **Hỗ trợ kiểm thử hồi quy:** hệ thống lưu lịch sử chạy, replay script và evidence để phục vụ các lần kiểm thử sau.
- **Hỗ trợ phân tích lỗi:** test run, step log và evidence được tách riêng để người dùng có thể xem lại quá trình thực thi.

4.2.1 Mô hình dữ liệu cốt lõi

Hình 4.2 trình bày mô hình dữ liệu cốt lõi của hệ thống. Sơ đồ này không nhằm thay thế toàn bộ schema vật lý, mà chỉ thể hiện các bảng quan trọng nhất giúp hình dung dữ liệu đi qua vòng đời kiểm thử như thế nào.



Hình 4.2: Mô hình dữ liệu cốt lõi phục vụ vòng đời kiểm thử

Trong mô hình dữ liệu cốt lõi, các bảng được chọn theo tiêu chí đại diện trực tiếp cho vòng đời kiểm thử: người dùng tạo project, tạo test case, chạy kiểm thử, lưu lịch sử chạy, lưu replay script và ghi nhận bằng chứng thực thi. Các bảng này giúp người đọc hiểu mối quan hệ chính của hệ thống mà không cần đi vào toàn bộ schema vật lý.

Bảng 4.2 mô tả các bảng cốt lõi trong thiết kế dữ liệu.

Bảng 4.2: Các bảng cốt lõi trong thiết kế dữ liệu

Bảng	Vai trò chính	Lý do đưa vào mô hình cốt lõi
users	Lưu thông tin người dùng đã xác thực.	Là cơ sở để gắn dữ liệu project, test case và test run với người sử dụng hệ thống.
projects	Lưu thông tin project kiểm thử, base URL và cấu hình liên quan.	Project là đơn vị tổ chức chính, liên kết hầu hết dữ liệu kiểm thử trong hệ thống.
test_cases	Lưu mục tiêu kiểm thử, tên test case, trạng thái và thông tin tổng quát.	Đây là thực thể trung tâm mô tả nghiệp vụ cần kiểm thử.
test_case_versions	Lưu nội dung test case theo từng phiên bản, gồm prompt, goal, plan snapshot hoặc thông tin liên quan.	Giúp truy vết nội dung test case tại thời điểm chạy, đặc biệt khi test case được chỉnh sửa nhiều lần.
execution_scripts	Lưu replay script được sinh từ phiên chạy agent thành công.	Là cơ sở để chạy lại kiểm thử hồi quy mà không cần agent suy luận lại toàn bộ luồng.
test_runs	Lưu mỗi lần thực thi test case hoặc replay script, gồm trạng thái, verdict, thời điểm chạy và lỗi tổng quát.	Đại diện cho kết quả của một lần chạy cụ thể và liên kết với evidence, log, report.
run_step_logs	Lưu log hoặc trạng thái theo từng bước thực thi.	Giúp truy vết test thất bại ở bước nào và hỗ trợ phân tích lỗi sau khi chạy.
evidences	Lưu các bằng chứng như screenshot, log file, artifact hoặc đường dẫn bằng chứng.	Giúp người dùng xem lại quá trình thực thi và kiểm chứng kết quả test run.

Trong các bảng trên, **projects** đóng vai trò gốc để gom nhóm dữ liệu kiểm thử. **test_cases** và **test_case_versions** mô tả nội dung cần kiểm thử và lịch sử thay đổi của nội dung đó. **execution_scripts** lưu kịch bản phát lại để hỗ trợ kiểm thử hồi quy. **test_runs**, **run_step_logs** và **evidences** lưu quá trình thực thi, kết quả và bằng chứng đi kèm. Nhờ cách tách này, hệ thống có thể phân biệt rõ giữa một test case ở mức thiết kế và một lần chạy cụ thể của test case đó.

4.2.2 Các bảng hỗ trợ và phạm vi trình bày

Ngoài các bảng cốt lõi, hệ thống còn có một số bảng hỗ trợ phục vụ chức năng mở rộng, vận hành và bảo trì kiểm thử. Các bảng này không được phân tích chi tiết trong ERD chính của Chương 4 để tránh làm mô hình dữ liệu quá phức tạp. Thay vào đó, Chương 4 chỉ trình bày vai trò tổng quan của từng nhóm bảng, còn chi tiết hiện thực sẽ được trình bày ở Chương 5 hoặc phần phụ lục schema nếu cần.

Bảng 4.3: Các nhóm bảng hỗ trợ trong hệ thống

Nhóm bảng hỗ trợ	Ví dụ bảng	Vai trò tổng quan
Báo cáo và phân tích lỗi	<code>reports</code> , <code>failure_analyses</code>	Tổng hợp kết quả chạy test và lưu gợi ý nguyên nhân khi test thất bại.
Dữ liệu kiểm thử	<code>test_datasets</code> , các bảng dataset run nếu có	Hỗ trợ chạy cùng một test case hoặc replay script với nhiều bộ dữ liệu đầu vào.
Test suite hoặc test sheet	<code>test_sheets</code> , <code>test_sheet_runs</code>	Gom nhiều test case thành nhóm chạy và tổng hợp kết quả theo lô.
Object repository	Các bảng lưu object, selector collection và attribute snapshot	Lưu thông tin nhận diện phần tử giao diện để hỗ trợ locator fallback và bảo trì test case.
Cấu hình runtime	<code>agent_runtime_configs</code> , <code>browser_profiles</code>	Lưu cấu hình chạy agent, loại trình duyệt, timeout, allowed domains hoặc thông tin runtime liên quan.
Bảo mật và phiên làm việc	<code>sessions</code> , <code>project_secrets</code>	Quản lý phiên đăng nhập, dữ liệu nhạy cảm và quyền truy cập theo project.
Lịch sử sinh bằng AI	Các bảng generation log hoặc candidate nếu có	Lưu lịch sử sinh test case, dữ liệu kiểm thử hoặc phản hồi từ mô hình ngôn ngữ lớn.

Cách tách giữa bảng cốt lõi và bảng hỗ trợ giúp Chương 4 tập trung vào thiết kế dữ liệu ở mức kiến trúc. Các chi tiết như kiểu dữ liệu cụ thể, migration, ràng buộc khóa ngoại, chỉ mục, cấu trúc JSON/JSONB và các

bảng phụ trợ sẽ phù hợp hơn khi trình bày trong phần hiện thực cơ sở dữ liệu ở Chương 5.

4.2.3 Các quyết định schema đáng chú ý

Bảng 4.4: Các quyết định schema đáng chú ý

Quyết định	Lý do
Chỉ trình bày các bảng cốt lõi trong ERD chính	Chương 4 tập trung vào thiết kế ở mức kiến trúc, nên ERD chính chỉ cần thể hiện các bảng đại diện trực tiếp cho vòng đời kiểm thử. Các bảng phụ trợ được mô tả tổng quan để tránh làm sơ đồ quá rối.
Tách test case và test run	Test case mô tả mục tiêu kiểm thử, còn test run mô tả một lần thực thi cụ thể. Cách tách này giúp một test case có thể được chạy nhiều lần và có nhiều kết quả khác nhau.
Quản lý test case theo version	Test case có thể thay đổi theo thời gian, nên cần lưu version để truy vết nội dung test case tại thời điểm chạy.
Lưu replay script dạng JSON	Replay script có cấu trúc linh hoạt, có thể chứa nhiều loại action, assertion, params và meta-data.
Tách evidence khỏi test run	Một test run có thể có nhiều evidence như screenshot, log, DOM snapshot, video hoặc trace. Tách evidence giúp hệ thống mở rộng loại bằng chứng mà không làm bảng test run quá lớn.
Lưu snapshot khi chạy	Runtime config, browser profile hoặc dataset có thể thay đổi, nên cần lưu snapshot để phân tích lại test run chính xác.
Lưu dữ liệu bán cấu trúc bằng JSON/JSONB	Các dữ liệu như replay script, evidence summary, action history hoặc selector collection có cấu trúc linh hoạt, nên lưu dạng JSON/JSONB để dễ mở rộng mà không phải thay đổi schema quá thường xuyên.

Ngoài cấu trúc bảng, hệ thống cần quản lý rõ trạng thái của các tiến trình bất đồng bộ như scan, test run, replay run và suite run. Một số trạng

thái chính gồm pending, running, passed, failed, error và cancelled. Việc chuẩn hóa trạng thái giúp frontend hiển thị tiến độ, backend cập nhật kết quả và người dùng dễ truy vết quá trình thực thi.

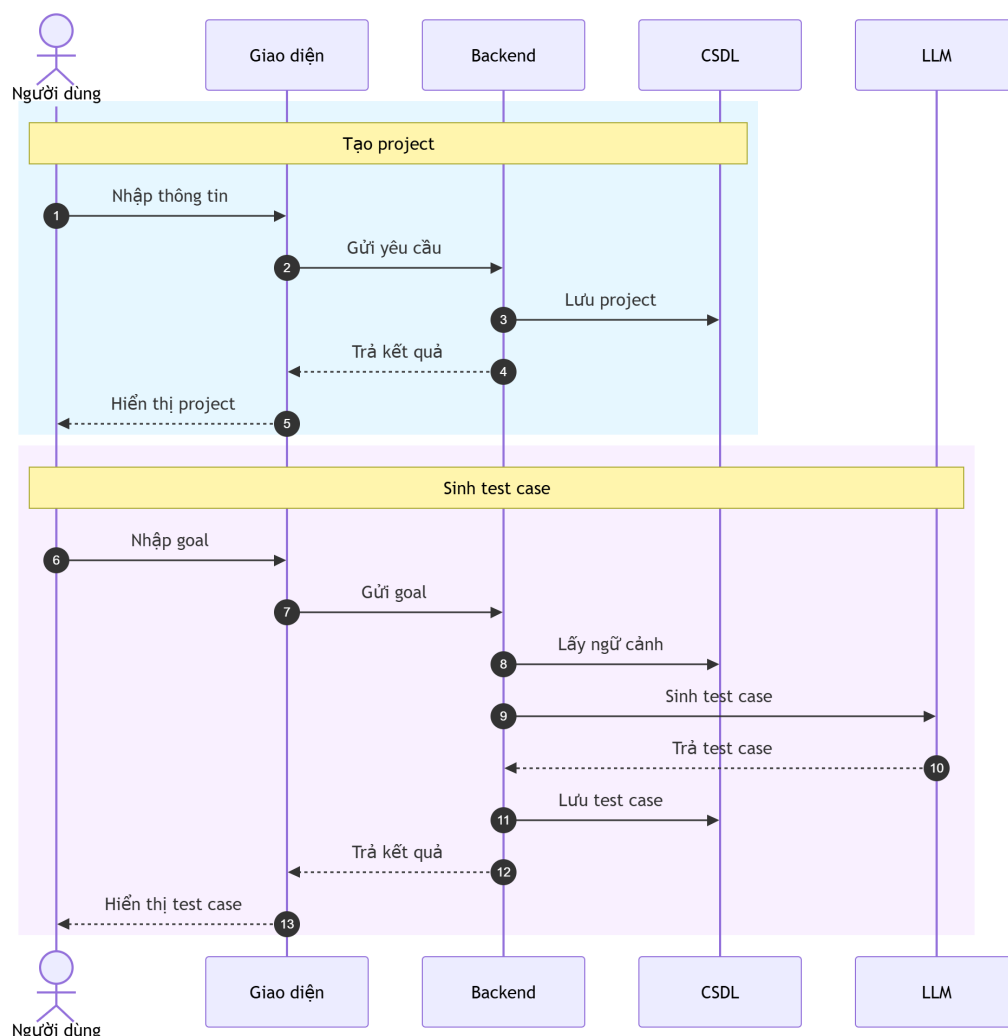
4.3 Thiết kế các luồng xử lý cốt lõi

Phần này mô tả các luồng xử lý chính của hệ thống. Các luồng được trình bày theo vòng đời kiểm thử: chuẩn bị project, tạo test case, chạy kiểm thử bằng agent, sinh replay script, chạy replay và ghi nhận kết quả.

4.3.1 Luồng tạo project, scan/crawl website và sinh test case

Luồng scan/crawl website được dùng để xây dựng thông tin nền ban đầu về website trong phạm vi project, bao gồm các URL có thể truy cập, sitemap và interaction map. Chức năng này không phân tích sâu toàn bộ logic giao diện mà chỉ cung cấp ngữ cảnh ban đầu cho các bước tạo test case và chạy agent.

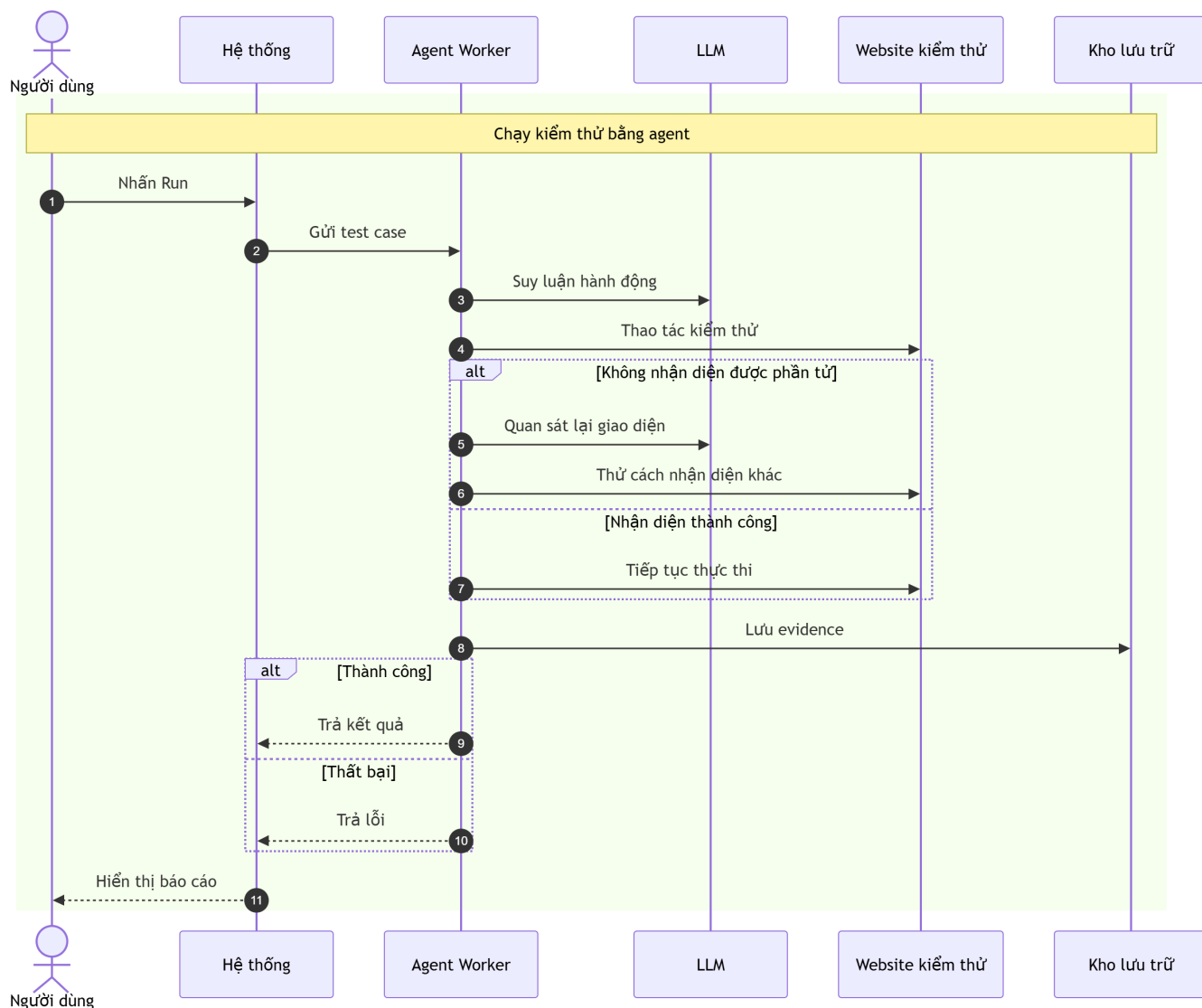
Luồng sinh test case bắt đầu từ prompt hoặc goal do người dùng nhập vào. Backend kết hợp prompt với thông tin project hoặc scan result nếu có, sau đó gọi AI để sinh test case gồm title, goal và test plan. Test case được lưu để người dùng xem lại, chỉnh sửa hoặc chạy thử.



Hình 4.3: Luồng tạo project và sinh test case

4.3.2 Luồng chạy bằng goal-based agent

Goal-based agent là chế độ chạy linh hoạt nhất của hệ thống. Agent đọc mục tiêu kiểm thử, quan sát giao diện hiện tại và quyết định hành động tiếp theo dựa trên trạng thái thực tế của website. Khi agent gặp khó khăn trong việc nhận diện phần tử giao diện, cơ chế locator fallback được xử lý như một nhánh trong chính luồng agent execution, thay vì tách thành một luồng riêng.



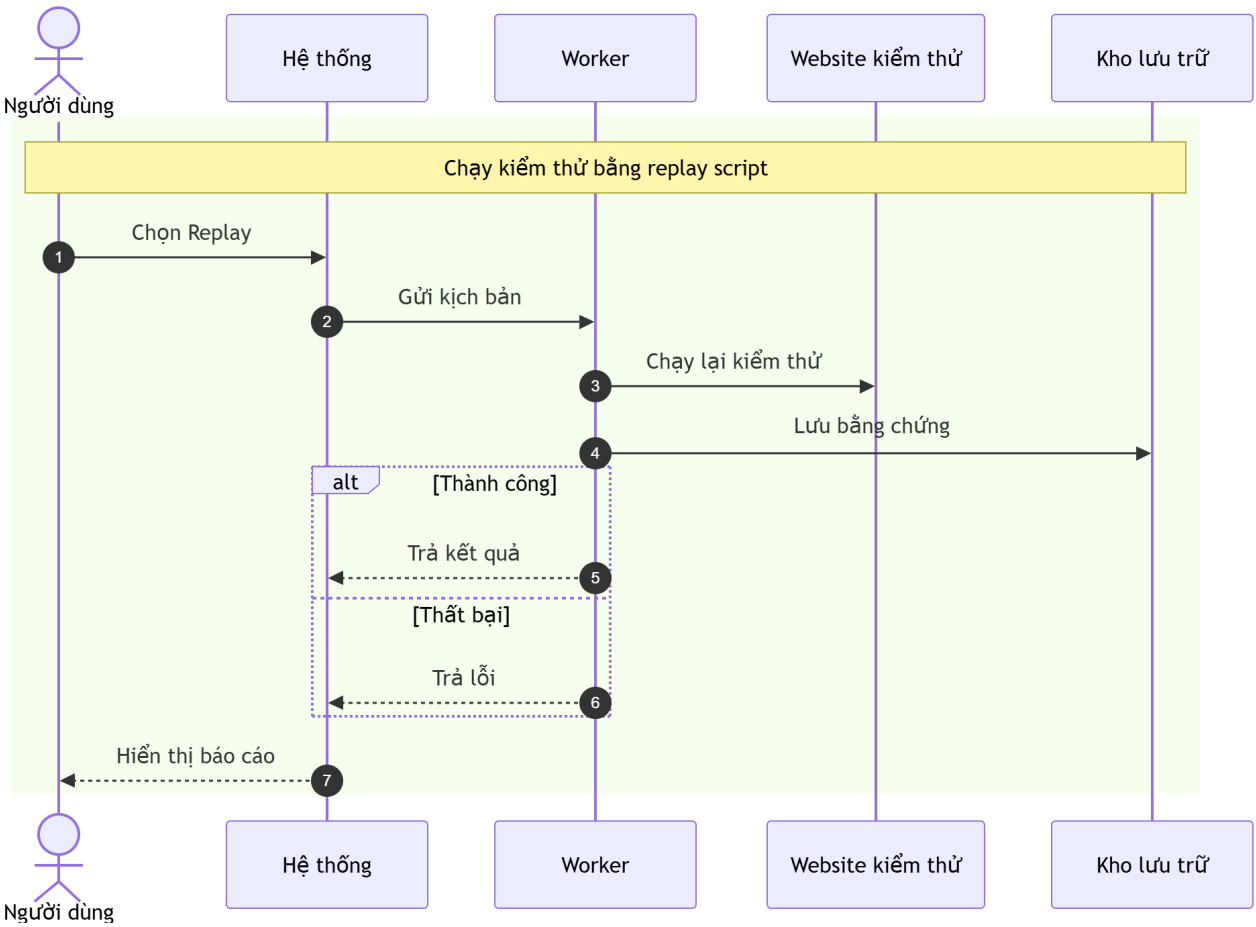
Hình 4.4: Luồng chạy test bằng goal-based agent

Trong luồng này, người dùng chọn test case và yêu cầu chạy bằng agent. Backend tạo test run và gửi yêu cầu sang worker. Worker sử dụng Browser-use, Playwright và LLM để quan sát giao diện, suy luận hành động và thao tác trên website mục tiêu. Nếu locator hoặc cách nhận diện phần tử ban đầu không còn phù hợp, agent có thể quan sát lại giao diện, sử dụng ngữ cảnh hoặc thông tin phần tử có sẵn để thử cách nhận diện khác và tiếp tục thực thi. Trong quá trình chạy, hệ thống ghi nhận log, screenshot, action history và evidence. Khi test kết thúc, worker trả kết quả về backend. Nếu

phiên chạy thành công, hệ thống có thể sinh replay script từ lịch sử hành động để phục vụ các lần chạy lại sau.

4.3.3 Luồng chạy bằng replay script

Replay script được sử dụng khi test case đã có kịch bản phát lại. Đây là chế độ phù hợp cho kiểm thử hồi quy vì không cần agent suy luận lại toàn bộ luồng kiểm thử.



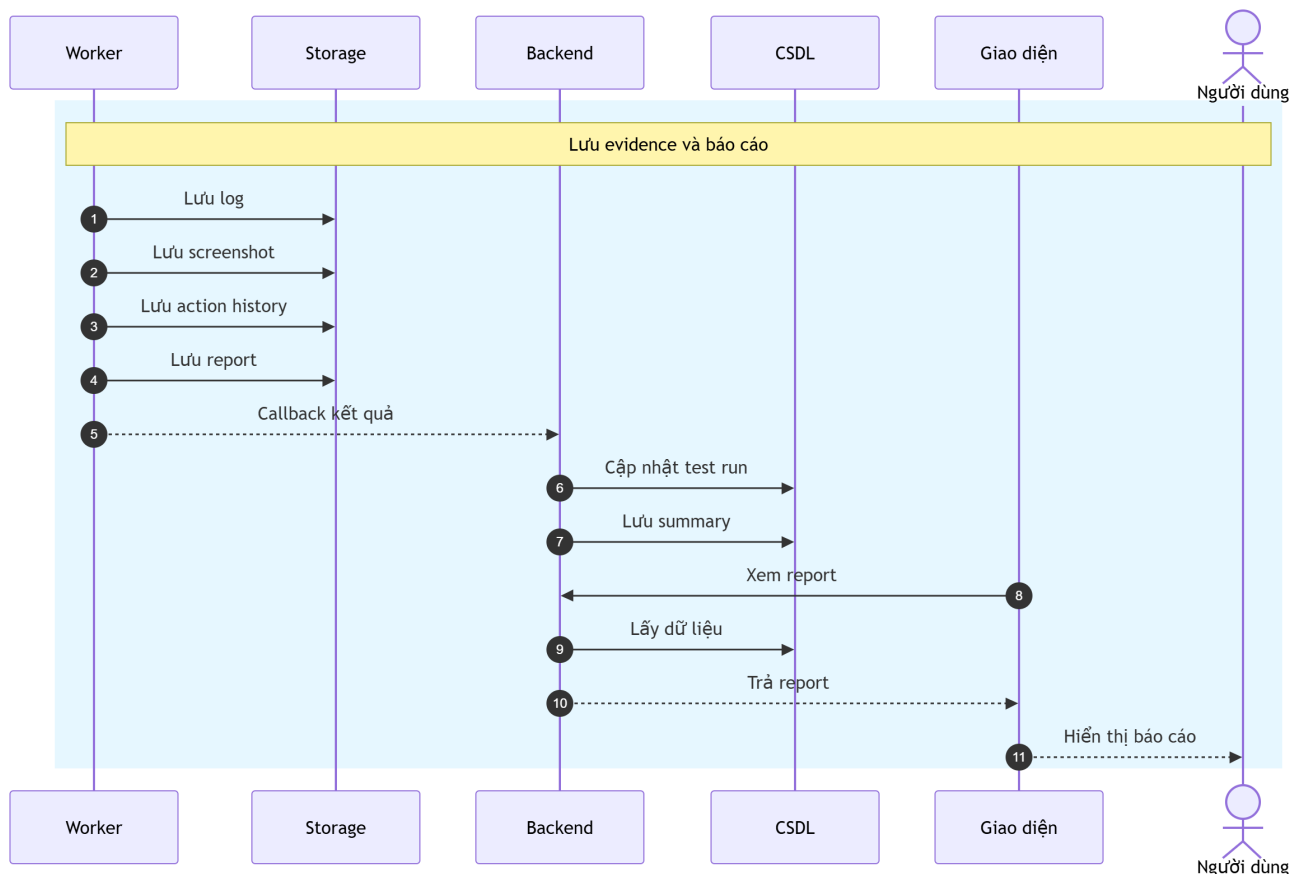
Hình 4.5: Luồng chạy test bằng replay script

Trong chế độ replay, worker đọc script JSON đã lưu và sử dụng Playwright để chạy các step theo thứ tự. Replay script ưu tiên tính ổn định và khả năng lặp lại, nên không gọi LLM để tự suy luận lại toàn bộ luồng kiểm

thử. Nếu locator, action hoặc assertion trong script không còn phù hợp, test run được ghi nhận là failed hoặc error, đồng thời hệ thống lưu evidence để người dùng phân tích và cập nhật lại test case, object repository hoặc replay script khi cần.

4.3.4 Luồng sinh report và evidence

Trong quá trình chạy test, worker ghi nhận log, screenshot, action history và kết quả từng bước. Khi test kết thúc, worker tổng hợp verdict, error message và evidence summary rồi callback về backend. Backend cập nhật test run, lưu evidence và cung cấp dữ liệu để frontend hiển thị report.



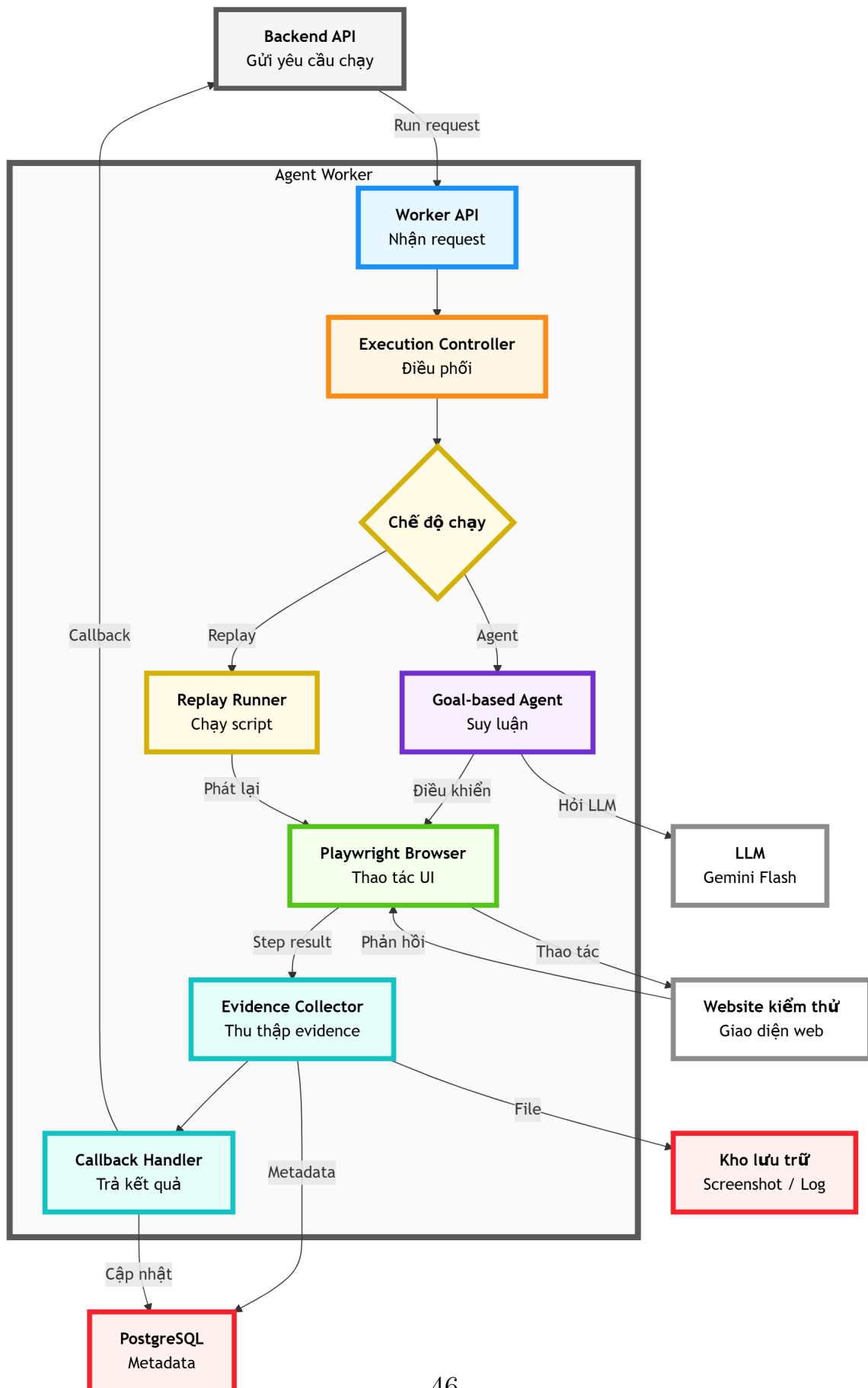
Hình 4.6: Luồng lưu evidence và sinh báo cáo

Report cần thể hiện rõ test pass/fail, lỗi xảy ra ở bước nào và evidence

liên quan. Với người dùng kỹ thuật, hệ thống có thể hiển thị thêm log chi tiết, action history, assertion failure hoặc thông tin locator để phục vụ quá trình sửa lỗi.

4.4 Thiết kế agent-worker và cơ chế thực thi

Agent-worker là thành phần kỹ thuật lõi của hệ thống vì đây là nơi trực tiếp khởi tạo trình duyệt, điều phối Browser-use agent, chạy replay script bằng Playwright và ghi nhận evidence. Việc tách agent-worker khỏi backend giúp backend tập trung vào nghiệp vụ và API, trong khi worker xử lý các tác vụ nặng hơn như browser automation, gọi LLM và lưu bằng chứng thực thi.



Hình 4.7: Thiết kế bên trong agent-worker

Khi chạy bằng goal-based agent, worker nhận test case và runtime config từ backend, sau đó khởi tạo browser context trong phạm vi website được phép truy cập. Worker sử dụng Browser-use agent để quan sát giao diện và gọi LLM nhằm suy luận hành động tiếp theo. Các thao tác trên trình duyệt được thực hiện thông qua Playwright. Trong quá trình này, worker ghi nhận action history, log, screenshot và các evidence liên quan.

Khi chạy bằng replay script, worker không cần gọi LLM ở từng bước mà đọc script JSON đã lưu và dùng Playwright để phát lại các action theo thứ tự. Cách này phù hợp với kiểm thử hồi quy vì giảm chi phí gọi LLM và giúp kết quả ổn định hơn so với chế độ agent.

Agent-worker cũng cần có các cơ chế kiểm soát vận hành như timeout, giới hạn số phiên trình duyệt chạy đồng thời, allowed domains và xử lý lỗi khi website không tải được, locator thất bại hoặc assertion không đạt. Các lỗi này được ghi nhận vào test run và evidence để người dùng có thể phân tích sau khi chạy.

4.5 Thiết kế cơ chế hỗ trợ bảo trì kiểm thử

4.5.1 Object Repository và State Anchors

Object Repository được dùng để lưu thông tin định vị phần tử giao diện theo hướng có thể tái sử dụng. Mỗi object có thể chứa tên object, page key, mô tả, selector collection, attribute snapshot và nguồn tạo object. Thông tin này giúp hệ thống có thêm dữ liệu tham chiếu khi agent cần nhận diện phần tử giao diện trong quá trình thực thi, đồng thời hỗ trợ tái sử dụng thông tin phần tử cho các lần chạy sau.

State Anchors là các điều kiện xác nhận trạng thái quan trọng của giao diện, chẳng hạn như URL thay đổi, văn bản xuất hiện, phần tử hiển thị, giá trị input đúng hoặc thông báo lỗi xuất hiện. Khi chạy test, hệ thống không chỉ thực hiện action mà còn cần kiểm tra các anchor/assertion này để xác định test case đạt hay không đạt.

Self-healing ở mức locator fallback không được tách thành một quy trình độc lập trong thiết kế, mà được xem là một phần của luồng chạy bằng agent. Khi agent không nhận diện được phần tử theo cách ban đầu, hệ thống có thể sử dụng lại ngữ cảnh giao diện, thuộc tính phần tử hoặc thông tin trong object repository để hỗ trợ agent thử cách nhận diện khác. Cơ chế này chỉ hỗ trợ một số thay đổi nhỏ của giao diện và không thay thế việc cập nhật lại test case khi luồng nghiệp vụ thay đổi lớn.

4.5.2 Quản lý test suite và dữ liệu kiểm thử

Test suite giúp gom nhóm nhiều test case theo một mục tiêu kiểm thử lớn hơn, ví dụ nhóm kiểm thử đăng nhập, quản lý dữ liệu hoặc kiểm thử hồi quy cho một module. Khi chạy test suite, hệ thống lần lượt chạy các test case trong danh sách và tổng hợp kết quả pass, failed hoặc error.

Dữ liệu kiểm thử giúp chạy cùng một test case với nhiều bộ input khác nhau. Ví dụ, một test case đăng nhập có thể được chạy với tài khoản hợp lệ, sai mật khẩu, thiếu username hoặc dữ liệu sai định dạng. Cách thiết kế này giúp người dùng không cần tạo nhiều test case gần giống nhau chỉ để thay đổi dữ liệu đầu vào.

4.6 Thiết kế bảo mật và kiểm soát vận hành

Do hệ thống có khả năng tự động mở trình duyệt, gọi LLM và thao tác trên website mục tiêu, các cơ chế bảo mật và kiểm soát vận hành cần được thiết kế ngay từ đầu. Bảng 4.5 tóm tắt các nhóm kiểm soát chính.

Bảng 4.5: Các nhóm kiểm soát bảo mật và vận hành

Nhóm kiểm soát	Nội dung thiết kế
Xác thực và phân quyền	Người dùng phải đăng nhập trước khi truy cập hệ thống. Mỗi project, test case, dataset, evidence và replay script cần gắn với người dùng hoặc phạm vi truy cập tương ứng.
Bảo vệ backend API	Backend kiểm tra request, kiểm tra dữ liệu đầu vào, kiểm tra quyền truy cập project và không để lộ API key, token hoặc cấu hình LLM ở frontend.
Bảo vệ agent-worker	Worker chỉ nhận request từ backend hoặc nguồn tin cậy, có timeout, giới hạn số phiên browser đồng thời và giới hạn domain được phép truy cập.
Bảo vệ dữ liệu kiểm thử	Screenshot, log, dataset và evidence có thể chứa thông tin nhạy cảm, nên cần được lưu ở vị trí có kiểm soát truy cập và tránh ghi token, API key hoặc mật khẩu vào log.
Kiểm soát phạm vi website	Mỗi test run cần có allowed domains hoặc cơ chế giới hạn domain để tránh agent hoặc replay script điều hướng ra ngoài phạm vi kiểm thử.
Nội dung website và prompt injection	Nội dung trên website mục tiêu được xem là dữ liệu không tin cậy. Worker không cho phép nội dung website ghi đè quy tắc vận hành của agent, không đưa API key hoặc token vào prompt và cần giới hạn hành động trong phạm vi kiểm thử đã cấu hình.

Đối với các website cần đăng nhập, hệ thống nên khuyến khích sử dụng tài khoản staging hoặc tài khoản test có quyền hạn giới hạn. Không nên sử dụng tài khoản thật trên môi trường production nếu chưa có cơ chế kiểm soát phù hợp.

4.7 Cơ sở lựa chọn thiết kế và các đánh đổi

Các quyết định thiết kế chính của hệ thống được tổng hợp trong Bảng 4.6. Bảng này giúp làm rõ không chỉ hệ thống được thiết kế như thế nào, mà còn vì sao nhóm chọn hướng thiết kế đó.

Bảng 4.6: Tổng hợp các quyết định kiến trúc lớn của Chương 4

Quyết định	Lý do chọn	Đánh đổi
Tách backend và agent-worker	Backend tập trung vào nghiệp vụ, worker tập trung vào chạy trình duyệt và LLM.	Cần callback, đồng bộ trạng thái và xử lý lỗi giữa hai service.
Sử dụng LLM và agent trong lần chạy ban đầu	Giúp người dùng tạo và thực thi kiểm thử từ mô tả tự nhiên, giảm rào cản viết script; đồng thời hỗ trợ self-healing ở mức locator fallback khi giao diện thay đổi nhỏ.	Kết quả có thể không hoàn toàn xác định, cần lưu evidence và chuyển phiên chạy thành replay script khi thành công.
Kết hợp agent và replay script	Agent phù hợp cho lần chạy linh hoạt ban đầu; replay phù hợp cho kiểm thử hồi quy và giảm chi phí LLM.	Cần chuyển lịch sử agent thành script và bảo trì script khi giao diện thay đổi.
Lưu replay script dạng JSON	Dễ mở rộng action, assertion, params và metadata.	Cần validate script trước khi chạy để tránh lỗi runtime.
Chỉ trình bày bảng cốt lõi trong ERD chính	Giữ mô hình dữ liệu ở Chương 4 gọn, tập trung vào vòng đời kiểm thử chính.	Các bảng phụ trợ cần được mô tả tổng quan và trình bày chi tiết hơn ở Chương 5 hoặc phụ lục.
Tách evidence khỏi test run	Cho phép lưu nhiều loại bằng chứng cho một lần chạy.	Cần quản lý lưu trữ và quyền truy cập evidence chặt chẽ.
Dùng object repository và state anchors	Cung cấp thêm thông tin nhận diện phần tử và điều kiện xác nhận trạng thái, giúp agent thực thi ổn định hơn khi giao diện thay đổi nhỏ.	Cần logic lưu, cập nhật và lựa chọn selector/attribute phù hợp.
Giới hạn domain khi chạy worker	Giảm rủi ro agent truy cập ngoài phạm vi kiểm thử.	Người dùng cần cấu hình project và allowed domains đúng.

Nhìn chung, các quyết định trên đều xoay quanh một mục tiêu chung: giữ cho hệ thống đủ linh hoạt để tận dụng AI agent, nhưng vẫn có cơ chế xác định, truy vết và kiểm soát khi chạy kiểm thử hồi quy.

4.8 Kết chương

Chương này đã trình bày kiến trúc và thiết kế hệ thống kiểm thử tự động ứng dụng web. Hệ thống được thiết kế theo hướng tách biệt giữa frontend, backend API, cơ sở dữ liệu và agent-worker, giúp phân tách rõ phần giao diện, nghiệp vụ, lưu trữ và thực thi kiểm thử.

Chương cũng trình bày thiết kế dữ liệu ở mức cốt lõi, các luồng xử lý chính, thiết kế agent-worker, cơ chế hỗ trợ bảo trì kiểm thử, nguyên tắc bảo mật và các đánh đổi quan trọng. Đối với thiết kế dữ liệu, chương chỉ tập trung vào các bảng cốt lõi phục vụ vòng đời kiểm thử, đồng thời mô tả tổng quan các nhóm bảng hỗ trợ để tránh làm mô hình dữ liệu quá phức tạp. Các chi tiết hiện thực cụ thể của schema vật lý và các bảng phụ trợ sẽ được trình bày ở Chương 5.

Đặc biệt, chương đã làm rõ vai trò của agent-worker trong việc điều phối Browser-use, Playwright và LLM để thực thi kiểm thử, đồng thời phân biệt giữa chế độ chạy linh hoạt bằng agent và chế độ chạy ổn định hơn bằng replay script. Đây là cơ sở để trình bày quá trình hiện thực và tích hợp hệ thống ở chương tiếp theo.

Chương 5

Xây dựng và tích hợp hệ thống

5.1 Môi trường và công nghệ sử dụng

Dựa trên kiến trúc đã trình bày ở Chương 4, hệ thống được xây dựng theo mô hình nhiều thành phần, gồm frontend, backend API, cơ sở dữ liệu và agent-worker. Cách tách này giúp hệ thống dễ phát triển, dễ bảo trì và phù hợp với bài toán kiểm thử tự động ứng dụng web, trong đó thành phần chạy trình duyệt có thể tiêu tốn nhiều tài nguyên hơn các thành phần còn lại.

Bảng 5.1 trình bày các công nghệ chính được sử dụng trong quá trình hiện thực hệ thống.

Bảng 5.1: Môi trường và công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Frontend	ReactJS, Vite, shadcn/ui, Tailwind CSS	Xây dựng giao diện quản lý project, test case, test data, test suite, object repository và report.
Backend API	Node.js, Express.js	Cung cấp API, xử lý nghiệp vụ, quản lý dữ liệu và điều phối yêu cầu chạy test.
Database	PostgreSQL 16	Lưu trữ user, project, test case, replay script, test run, evidence, report, dataset và các dữ liệu hỗ trợ quá trình kiểm thử.
Agent-worker	Python, FastAPI	Thực thi test trên trình duyệt, chạy agent, chạy replay script, ghi nhận evidence và callback kết quả.
Browser automation	Browser-use, Playwright	Browser-use hỗ trợ agent chạy theo mục tiêu tự nhiên; Playwright điều khiển trình duyệt và chạy replay script.
AI/LLM	Google Gemini Flash	Hỗ trợ xử lý prompt, sinh test case, hỗ trợ agent suy luận hành động và gợi ý phân tích lỗi.

Trong quá trình xây dựng, frontend không trực tiếp chạy kiểm thử trên trình duyệt, backend không trực tiếp điều khiển browser, còn agent-worker không quản lý dữ liệu nghiệp vụ dài hạn. Mỗi thành phần đảm nhiệm một vai trò riêng để hệ thống dễ mở rộng, dễ kiểm soát lỗi và dễ thay thế khi cần thiết.

5.2 Xây dựng các thành phần chính

5.2.1 Giao diện người dùng

Giao diện người dùng được xây dựng bằng ReactJS, Vite, shadcn/ui và Tailwind CSS. Frontend là nơi người dùng thao tác với toàn bộ quy trình kiểm thử, từ tạo project, tạo test case, chạy test, xem lịch sử thực thi đến phân tích kết quả.

Các nhóm màn hình chính gồm màn hình xác thực, dashboard, project detail, test case detail, test data management, test suite, object repository, replay script và test run report. Giao diện được thiết kế theo hướng giúp người dùng thao tác với project, prompt, test case và báo cáo mà không cần trực tiếp viết mã kiểm thử. Các thông tin kỹ thuật như replay script, selector hoặc assertion chỉ được hiển thị ở những khu vực cần chỉnh sửa nâng cao.

5.2.2 Backend API

Backend API được xây dựng bằng Node.js và Express.js. Thành phần này giữ vai trò trung tâm trong việc xử lý nghiệp vụ, quản lý dữ liệu và điều phối các yêu cầu chạy test sang agent-worker.

Các nhóm API chính gồm authentication, project, scan, test case, execution, replay script, test data, test suite, object repository và report. Khi người dùng yêu cầu chạy test, backend tạo bản ghi test run, chuẩn bị payload gồm thông tin test case, input params, runtime config, browser profile và callback URL, sau đó gửi sang agent-worker. Khi worker hoàn tất, backend nhận callback, cập nhật trạng thái test run, lưu evidence summary và trả kết quả để frontend hiển thị.

Backend cũng chịu trách nhiệm kiểm tra quyền truy cập project, validate dữ liệu đầu vào, chuẩn hóa trạng thái thực thi và đảm bảo các dữ liệu như replay script, evidence hoặc dataset chỉ được truy cập trong phạm vi

hợp lệ.

5.2.3 Agent-worker

Agent-worker được xây dựng bằng Python và FastAPI. Đây là thành phần chịu trách nhiệm thực thi kiểm thử trên trình duyệt, bao gồm chạy test bằng Browser-use agent, chạy replay script bằng Playwright, ghi nhận evidence và callback kết quả về backend.

Agent-worker hỗ trợ hai chế độ thực thi chính:

- **Goal-based agent:** worker nhận goal, prompt, plan snapshot và input params, sau đó tạo task cho Browser-use agent để chạy kiểm thử trên trình duyệt.
- **Replay script:** worker đọc script JSON đã lưu và dùng Playwright để chạy lại các bước như navigate, fill, click và assertion.

Trong quá trình chạy, worker ghi nhận action history, current URL, screenshot, log, trạng thái từng bước, verdict cuối cùng và error message nếu có. Sau khi chạy xong, worker callback kết quả về backend. Nếu phiên chạy bằng agent thành công, hệ thống có thể chuyển lịch sử hành động thành replay script để sử dụng cho các lần chạy sau.

Locator fallback được xử lý trong luồng agent execution. Khi agent không nhận diện được phần tử theo cách ban đầu, worker có thể hỗ trợ agent quan sát lại giao diện, sử dụng thông tin ngữ cảnh hoặc thông tin định vị phần tử đã lưu để thử cách nhận diện khác. Cơ chế này chỉ hỗ trợ một số thay đổi nhỏ của giao diện và không thay thế việc cập nhật lại test case khi luồng nghiệp vụ thay đổi lớn.

5.2.4 Cơ sở dữ liệu

Cơ sở dữ liệu sử dụng PostgreSQL 16 để lưu trữ dữ liệu nghiệp vụ và dữ liệu thực thi của hệ thống. Các nhóm dữ liệu chính gồm người dùng và

project, test case và replay script, test run và evidence, report, test data, test suite và các bảng hỗ trợ vận hành.

Một số dữ liệu có cấu trúc linh hoạt như replay script, plan snapshot, variables schema, runtime config snapshot, selector collection và evidence summary được lưu dưới dạng JSON/JSONB. Cách lưu này giúp hệ thống dễ mở rộng khi cần bổ sung loại action, assertion, metadata hoặc evidence mới.

Trong quá trình hiện thực, nhóm không đưa toàn bộ schema vào phần thiết kế chính của Chương 4, mà chỉ trình bày các bảng cốt lõi phục vụ vòng đời kiểm thử. Các bảng phụ trợ như cấu hình chạy, phiên đăng nhập, bảng phân loại hoặc lịch sử sinh dữ liệu được giữ trong schema vật lý nhưng không làm trọng tâm trình bày.

5.3 Tích hợp các công nghệ lõi

5.3.1 Tích hợp Browser-use

Browser-use được tích hợp trong agent-worker để hỗ trợ chế độ goal-based agent. Khi nhận yêu cầu chạy test, worker xây dựng task text từ goal, prompt, plan snapshot, input params, base URL và allowed domains. Task này được truyền cho Browser-use agent để agent quan sát giao diện, suy luận hành động và thao tác trên website mục tiêu.

Quá trình chạy bằng Browser-use được tổ chức theo vòng lặp quan sát – suy luận – hành động. Sau mỗi bước, worker ghi nhận log, screenshot và action history để phục vụ báo cáo và sinh replay script. Khi agent gặp khó khăn trong việc nhận diện phần tử, hệ thống có thể cung cấp thêm ngữ cảnh giao diện hoặc thông tin định vị phần tử đã lưu để hỗ trợ agent thử cách nhận diện khác.

5.3.2 Tích hợp Playwright

Playwright được sử dụng làm lớp điều khiển trình duyệt trong hệ thống. Trong chế độ agent, Playwright hỗ trợ mở trình duyệt và thực hiện các hành động do agent yêu cầu. Trong chế độ replay, Playwright trực tiếp chạy script JSON đã lưu.

Các thao tác chính gồm mở trang web, tìm phần tử, thực hiện click/fill/navigate, kiểm tra assertion, chụp screenshot và ghi nhận thông tin phục vụ evidence. Việc sử dụng Playwright giúp các lần chạy replay ổn định hơn vì không cần LLM suy luận lại toàn bộ luồng kiểm thử.

Trong chế độ replay, hệ thống ưu tiên tính xác định. Nếu locator, action hoặc assertion trong script không còn phù hợp, test run được ghi nhận là failed hoặc error và evidence được lưu lại để người dùng phân tích. Hệ thống không gọi LLM để tự sửa toàn bộ kịch bản replay trong lúc chạy.

5.3.3 Tích hợp Google Gemini Flash

Google Gemini Flash được sử dụng trong các tác vụ liên quan đến ngôn ngữ tự nhiên và phân tích ngữ cảnh. Trong hệ thống, mô hình hỗ trợ phân tích prompt, sinh test case, hỗ trợ agent suy luận hành động và gợi ý nguyên nhân khi test thất bại.

Tuy nhiên, hệ thống không phụ thuộc hoàn toàn vào LLM trong mọi lần chạy. Sau khi agent chạy thành công và replay script được tạo, các lần kiểm thử hồi quy có thể dùng Playwright để chạy lại script mà không cần gọi LLM ở từng bước. Cách kết hợp này giúp giảm chi phí vận hành và tăng tính xác định của quá trình kiểm thử.

5.4 Xây dựng các chức năng kiểm thử

5.4.1 Scan/crawl website

Chức năng scan/crawl website được xây dựng để ghi nhận thông tin nền ban đầu về website trong phạm vi project. Quá trình scan bắt đầu từ base URL và duyệt qua các trang có thể truy cập trong phạm vi cấu hình. Kết quả scan được lưu dưới dạng sitemap, interaction map, trạng thái scan, thời điểm chạy và thông tin lỗi nếu có.

Thông tin scan không thay thế khả năng quan sát của agent, mà đóng vai trò cung cấp ngữ cảnh ban đầu cho việc tạo test case và thực thi kiểm thử.

5.4.2 Sinh test case

Chức năng sinh test case cho phép người dùng nhập prompt hoặc goal bằng ngôn ngữ tự nhiên. Hệ thống sử dụng AI để phân tích nội dung này và tạo ra test case có cấu trúc gồm tiêu đề, mục tiêu kiểm thử, test plan và các thông tin liên quan.

Sau khi test case được sinh, người dùng có thể xem lại, chỉnh sửa hoặc lưu vào project. Test case được lưu cùng prompt gốc, execution mode, plan snapshot, variables schema và AI model nếu có để hỗ trợ truy vết và chỉnh sửa sau này.

5.4.3 Chạy test bằng agent

Chức năng chạy test bằng agent là phần thực thi linh hoạt của hệ thống. Người dùng chọn một test case và yêu cầu chạy bằng agent. Backend tạo test run, gửi thông tin test case và cấu hình chạy sang agent-worker. Worker khởi tạo trình duyệt, truyền mục tiêu kiểm thử cho Browser-use agent và sử dụng Playwright để thao tác trên website.

Trong quá trình chạy, agent quan sát giao diện, gọi LLM để suy luận hành động tiếp theo và thực hiện các thao tác như mở trang, nhập dữ liệu, nhấn nút, điều hướng hoặc kiểm tra trạng thái. Nếu gặp vấn đề về nhận diện phần tử, agent có thể quan sát lại giao diện hoặc sử dụng thông tin phần tử đã lưu để thử cách nhận diện khác. Sau khi hoàn tất, hệ thống lưu log, screenshot, action history, evidence và verdict của test run.

5.4.4 Replay script

Replay script là kịch bản phát lại được sinh từ lịch sử hành động sau khi agent chạy thành công. Script được lưu dưới dạng JSON, bao gồm danh sách step, action type, params, assertion và metadata liên quan.

Khi chạy replay, worker sử dụng Playwright để thực thi từng step theo thứ tự. Replay script không dùng LLM để suy luận lại toàn bộ luồng nên phù hợp với kiểm thử hồi quy. Nếu script không còn phù hợp với giao diện hiện tại, hệ thống ghi nhận lỗi, lưu evidence và để người dùng cập nhật lại test case, replay script hoặc thông tin định vị phần tử khi cần.

5.4.5 Quản lý test data và test suite

Chức năng test data cho phép người dùng tạo và quản lý các bộ dữ liệu đầu vào cho test case. Dataset có thể được tạo thủ công hoặc sinh tự động bằng AI dựa trên mục tiêu kiểm thử. Khi chạy test với dataset, hệ thống sử dụng dữ liệu tương ứng làm input và lưu kết quả riêng cho từng lần chạy.

Test suite được xây dựng để gom nhóm nhiều test case và chạy theo lô. Người dùng có thể tạo suite, thêm test case vào suite, sắp xếp thứ tự chạy và khởi tạo suite run. Sau khi chạy, hệ thống tổng hợp kết quả từng test case trong suite để phục vụ kiểm thử hồi quy.

5.4.6 Report và evidence

Sau mỗi lần chạy test, hệ thống tổng hợp kết quả thành report. Report bao gồm trạng thái cuối cùng, verdict, thời gian chạy, danh sách bước đã thực hiện, screenshot, log, evidence và error message nếu có.

Evidence được ghi nhận để hỗ trợ truy vết và phân tích lỗi. Với mỗi test run, hệ thống có thể lưu screenshot, action history, step log, URL hiện tại hoặc artifact khác tùy theo cấu hình. Khi test thất bại, các evidence này giúp người dùng xác định bước xảy ra lỗi và đưa ra hướng sửa phù hợp.

5.5 Xây dựng cơ chế hỗ trợ

5.5.1 Object repository và locator fallback

Object repository được xây dựng để lưu trữ thông tin định vị phần tử giao diện theo project. Mỗi object đại diện cho một phần tử có ý nghĩa trong kiểm thử, chẳng hạn như ô nhập username, ô nhập password, nút login hoặc một trường dữ liệu trong form.

Thay vì chỉ lưu một selector duy nhất, hệ thống có thể lưu nhiều thuộc tính định vị khác nhau như test id, id ổn định, name attribute, CSS selector, accessible name, placeholder, title hoặc XPath. Các thông tin này cung cấp thêm ngữ cảnh để agent nhận diện phần tử khi giao diện thay đổi nhỏ.

Locator fallback trong phạm vi hệ thống được hiểu là cơ chế hỗ trợ trong quá trình agent thực thi. Khi cách nhận diện ban đầu không còn phù hợp, agent có thể quan sát lại giao diện và thử cách định vị khác dựa trên ngữ cảnh hoặc thông tin đã lưu. Cơ chế này không nhằm tự động sửa mọi thay đổi giao diện và không thay thế việc cập nhật lại test case khi website thay đổi lớn về luồng nghiệp vụ.

5.5.2 AI analysis và report

Khi test thất bại, hệ thống có thể sử dụng AI để hỗ trợ gợi ý nguyên nhân lỗi, chẳng hạn do ứng dụng không thỏa điều kiện kiểm tra, locator không tìm thấy phần tử, timeout, dữ liệu đầu vào không phù hợp hoặc agent chọn sai hành động. Kết quả phân tích này chỉ đóng vai trò hỗ trợ, không thay thế việc kiểm tra lại của người dùng.

Thông tin đưa vào quá trình phân tích có thể gồm error message, action history, screenshot metadata, URL cuối cùng, trạng thái step và verdict. Dựa trên đó, hệ thống hiển thị gợi ý để người dùng cập nhật test case, dữ liệu kiểm thử, replay script hoặc thông tin định vị phần tử.

5.6 Bảo mật và xử lý dữ liệu nhạy cảm

Trong quá trình xây dựng hệ thống, một số nhóm dữ liệu cần được xem là nhạy cảm, bao gồm tài khoản người dùng, mật khẩu, API key, thông tin xác thực website, dữ liệu test account, screenshot và log. Do đó, hệ thống cần áp dụng các nguyên tắc bảo vệ dữ liệu trong quá trình hiện thực.

Các cơ chế chính gồm:

- Không lưu mật khẩu người dùng ở dạng thô.
- Không để lộ API key hoặc cấu hình LLM ở frontend.
- Kiểm tra quyền truy cập project trước khi cho phép xem hoặc chỉnh sửa test case, dataset, replay script và evidence.
- Giới hạn domain được phép truy cập khi chạy agent hoặc replay script.
- Không ghi token, API key hoặc mật khẩu thật vào log nếu không cần thiết.

- Xem nội dung website là dữ liệu không tin cậy để hạn chế rủi ro prompt injection.
- Khuyến khích sử dụng tài khoản test hoặc tài khoản staging khi chạy kiểm thử.

Do hệ thống có khả năng tự động thao tác trên website, việc giới hạn phạm vi kiểm thử là đặc biệt quan trọng. Mỗi project cần xác định base URL và allowed domains để hạn chế trường hợp agent hoặc replay script điều hướng ra ngoài phạm vi mong muốn.

5.7 Khó khăn trong quá trình xây dựng

Trong quá trình xây dựng hệ thống, nhóm gặp một số khó khăn chính sau:

- **Tích hợp nhiều công nghệ:** hệ thống kết hợp ReactJS, Node.js, PostgreSQL, Python FastAPI, Browser-use, Playwright và LLM, nên cần cơ chế giao tiếp rõ ràng giữa các thành phần.
- **Quản lý trạng thái chạy test:** một test run có thể kéo dài, timeout hoặc phát sinh lỗi, nên cần lưu trạng thái và callback kết quả chính xác.
- **Chuyển lịch sử agent thành replay script:** hành động do agent thực hiện cần được chuẩn hóa thành các step có cấu trúc để có thể chạy lại bằng Playwright.
- **Xử lý locator không ổn định:** giao diện web có thể thay đổi, khiến selector cũ không còn hoạt động, nên cần object repository và locator fallback ở mức agent run.
- **Kiểm soát chi phí khi dùng LLM:** replay script được dùng để giảm số lần gọi LLM trong kiểm thử hồi quy.

- **Ghi nhận evidence hợp lý:** screenshot, log và action history cần đủ để phân tích lỗi nhưng không nên làm hệ thống quá tải.

Những khó khăn trên ảnh hưởng trực tiếp đến cách hiện thực hệ thống. Vì vậy, nhóm ưu tiên tách thành phần, lưu vết đầy đủ, dùng replay script để giảm chi phí LLM và bổ sung object repository để hỗ trợ bảo trì kịch bản kiểm thử.

5.8 Kết chương

Chương này đã trình bày quá trình xây dựng và tích hợp hệ thống kiểm thử tự động ứng dụng web. Nội dung chương bao gồm môi trường công nghệ, các thành phần chính, cách tích hợp Browser-use, Playwright và Google Gemini Flash, cũng như các chức năng kiểm thử như scan/crawl website, sinh test case, chạy test bằng agent, replay script, test data, test suite, report và evidence.

Chương cũng trình bày các cơ chế hỗ trợ như object repository, locator fallback, AI analysis và report, đồng thời nêu các vấn đề bảo mật và khó khăn trong quá trình xây dựng. Đây là cơ sở để đánh giá kết quả đạt được của hệ thống ở chương tiếp theo.

Chương 6

Thực nghiệm và đánh giá hệ thống

6.1 Mục tiêu đánh giá

Sau khi hoàn thành các thành phần chính của hệ thống, quá trình thực nghiệm được thực hiện nhằm đánh giá mức độ đáp ứng các mục tiêu đã đặt ra trong phạm vi khóa luận. Việc đánh giá không chỉ xem xét một ca kiểm thử có chạy thành công hay không, mà còn quan sát toàn bộ quy trình kiểm thử từ khâu chuẩn bị project, sinh test case, chạy test trên trình duyệt, ghi nhận evidence, sinh replay script, chạy lại script và tổng hợp kết quả.

Các mục tiêu đánh giá chính gồm:

- Đánh giá khả năng sinh test case từ mô tả ngôn ngữ tự nhiên, bao gồm tiêu đề, mục tiêu kiểm thử, các bước thực hiện và điều kiện kỳ vọng.
- Đánh giá khả năng thực thi test case bằng agent trên trình duyệt thật.
- Đánh giá khả năng tái sử dụng kết quả chạy thành công thông qua replay script.

- Kiểm tra khả năng sinh dữ liệu kiểm thử phục vụ các kịch bản có nhiều bộ dữ liệu đầu vào.
- Phân tích các lỗi thường gặp trong quá trình chạy test như lỗi locator, lỗi assertion, lỗi môi trường hoặc lỗi do prompt chưa rõ ràng.
- Đánh giá vai trò của các thành phần hỗ trợ như evidence, log, timeline, report và phân tích lỗi.
- Thu thập phản hồi người dùng để đánh giá mức độ dễ sử dụng, mức độ hữu ích và các điểm cần cải thiện của hệ thống.

Trong phạm vi khóa luận, đánh giá được thực hiện theo hai hướng. Thứ nhất là đánh giá định lượng dựa trên log thực nghiệm được lưu trong hệ thống. Thứ hai là đánh giá định tính dựa trên phản hồi người dùng sau khi trải nghiệm hệ thống. Cách kết hợp này giúp đánh giá hệ thống từ cả góc nhìn kỹ thuật và góc nhìn sử dụng thực tế.

6.2 Môi trường và dữ liệu thực nghiệm

6.2.1 Môi trường thực nghiệm

Môi trường thực nghiệm được thiết lập theo kiến trúc nhiều thành phần của hệ thống. Các thành phần frontend, backend, database và agent-worker được tách riêng để dễ theo dõi log, kiểm tra API và phân tích lỗi khi phiên chạy test thất bại.

Bảng 6.1: Môi trường thực nghiệm hệ thống

Thành phần	Công nghệ sử dụng	Vai trò trong thực nghiệm
Frontend	ReactJS, Vite, Tailwind CSS, shadcn/ui	Thao tác tạo project, nhập prompt, tạo test case, chạy test và xem report.
Backend API	Node.js, Express.js	Lưu dữ liệu nghiệp vụ, tạo test run, điều phối worker và nhận callback kết quả.
Database	PostgreSQL	Lưu user, project, test case, generation log, test run, evidence, dataset và replay script.
Agent-worker	Python, FastAPI	Chạy agent, chạy replay script, chụp screenshot, ghi log và gửi kết quả về back-end.
Browser automation	Browser-use, Playwright, Chromium	Điều khiển trình duyệt, quan sát giao diện, thao tác với website và phát lại script.
LLM	Gemini Flash theo cấu hình hệ thống	Hỗ trợ sinh test case, sinh dataset, điều phối agent và phân tích lỗi.
Website mục tiêu	OrangeHRM demo	Ứng dụng Web được dùng làm case study để đánh giá các luồng kiểm thử giao diện.

Trong quá trình thực nghiệm, hệ thống được cấu hình để giới hạn phạm vi website mục tiêu thông qua base URL và allowed domain. Điều này giúp tránh trường hợp agent hoặc replay script điều hướng ra ngoài phạm vi kiểm thử.

6.2.2 Nguồn dữ liệu đánh giá

Dữ liệu đánh giá được tổng hợp từ hai nhóm nguồn chính. Nhóm thứ nhất là log hệ thống được ghi nhận trong quá trình người dùng thao tác và chạy kiểm thử, bao gồm log sinh test case, log sinh dataset, log thực thi test case, log lỗi từng bước và thông tin liên quan đến thời gian xử lý. Nhóm thứ hai là dữ liệu khảo sát trải nghiệm người dùng sau khi dùng thử hệ thống.

Các nguồn dữ liệu này được sử dụng nhằm đánh giá hệ thống từ hai góc nhìn khác nhau. Log hệ thống phản ánh khả năng vận hành thực tế của các chức năng như sinh test case, chạy agent, replay script và ghi nhận lỗi. Trong khi đó, dữ liệu khảo sát giúp bổ sung góc nhìn người dùng về mức độ dễ sử dụng, mức độ hữu ích và các điểm cần cải thiện của hệ thống.

Bảng 6.2: Nguồn dữ liệu sử dụng trong đánh giá

Nguồn dữ liệu	Nội dung ghi nhận	Vai trò trong đánh giá
Log thực thi test case	Kết quả chạy, thời gian thực thi, chế độ chạy, trạng thái pass/fail và khả năng tái sử dụng.	Đánh giá khả năng thực thi kiểm thử trên trình duyệt và mức độ ổn định của hệ thống.
Log sinh test case	Prompt đầu vào, test case được sinh ra, thời gian sinh và thông tin token sử dụng.	Đánh giá khả năng hỗ trợ tạo test case từ mô tả ngôn ngữ tự nhiên.
Log sinh dữ liệu kiểm thử	Prompt sinh dữ liệu, số dòng dữ liệu sinh ra, trạng thái sinh dữ liệu và thời gian xử lý.	Đánh giá khả năng hỗ trợ chuẩn bị dữ liệu kiểm thử cho các kịch bản khác nhau.
Log lỗi từng bước	Hành động bị lỗi, loại lỗi, thông báo lỗi, URL tại thời điểm lỗi và thông tin bước thực thi.	Phân tích nguyên nhân thất bại và các yếu tố ảnh hưởng đến kết quả kiểm thử.
Khảo sát người dùng	Đánh giá theo thang điểm và phản hồi mở sau khi người dùng trải nghiệm hệ thống.	Bổ sung góc nhìn về trải nghiệm sử dụng, mức độ hữu ích và các điểm cần cải thiện.

Dữ liệu log được lọc theo các project có website mục tiêu liên quan đến OrangeHRM để phục vụ đánh giá case study.

6.2.3 Quy trình thực nghiệm

Quy trình thực nghiệm được thực hiện theo các bước sau:

1. Tạo project kiểm thử và cấu hình website mục tiêu là OrangeHRM.
2. Nhập prompt mô tả yêu cầu kiểm thử bằng ngôn ngữ tự nhiên để sinh test case.

3. Kiểm tra lại test case được sinh ra, bao gồm mục tiêu kiểm thử, các bước thực hiện và điều kiện kỳ vọng.
4. Chạy test case lần đầu bằng agent trên trình duyệt thật.
5. Với các phiên chạy thành công, hệ thống ghi nhận lịch sử thao tác và sinh replay script để có thể chạy lại.
6. Chạy lại các kịch bản bằng replay script nhằm đánh giá khả năng tái sử dụng trong kiểm thử hồi quy.
7. Ghi nhận kết quả thực thi, thời gian chạy, trạng thái pass/fail, lỗi từng bước và thông tin token sử dụng.
8. Tổng hợp log hệ thống và phản hồi khảo sát người dùng để phân tích kết quả.

Quy trình này giúp đánh giá hệ thống theo đúng workflow sử dụng thực tế: từ prompt ban đầu, sinh test case, chạy bằng agent, tạo replay script, chạy lại kịch bản và phân tích kết quả.

6.3 Case study: OrangeHRM

OrangeHRM được chọn làm case study vì đây là một ứng dụng Web quản lý nhân sự có cấu trúc giao diện và nghiệp vụ tương đối điển hình. Ứng dụng có màn hình đăng nhập, dashboard, thanh điều hướng, nhiều phân hệ chức năng và nhiều biểu mẫu nhập liệu. Những đặc điểm này phù hợp để đánh giá hệ thống kiểm thử tự động ứng dụng Web ở mức giao diện người dùng.

Các lý do chọn OrangeHRM gồm:

- Có luồng đăng nhập rõ ràng, phù hợp để đánh giá khả năng thao tác form và xác nhận trạng thái sau đăng nhập.

- Có nhiều phân hệ và menu điều hướng, phù hợp để đánh giá khả năng agent tìm và chuyển đến đúng khu vực chức năng.
- Có các màn hình danh sách, tìm kiếm và bộ lọc, phù hợp để đánh giá kịch bản nhập dữ liệu và kiểm tra kết quả hiển thị.
- Có giao diện Web động, phù hợp để quan sát ảnh hưởng của tốc độ tải trang, trạng thái đăng nhập và locator đến kết quả kiểm thử.
- Có thể xây dựng các kịch bản kiểm thử không phá hủy dữ liệu, chẳng hạn như đăng nhập, điều hướng, tìm kiếm và kiểm tra thông báo lỗi.

Phạm vi case study tập trung vào các luồng giao diện phổ biến thay vì kiểm thử toàn bộ nghiệp vụ nhân sự. Các thao tác có khả năng thay đổi dữ liệu quan trọng được hạn chế hoặc chỉ thực hiện với tài khoản kiểm thử nhằm tránh ảnh hưởng đến môi trường mục tiêu.

6.4 Thiết kế kịch bản và chỉ số đánh giá

6.4.1 Các kịch bản đánh giá

Các kịch bản được thiết kế để bao phủ những năng lực quan trọng của hệ thống, từ giai đoạn tạo test case đến giai đoạn chạy lại và phân tích kết quả.

Bảng 6.3: Các kịch bản đánh giá trên OrangeHRM

Mã	Kịch bản	Mục tiêu đánh giá	Chức năng liên quan
SC01	Đăng nhập với tài khoản hợp lệ	Kiểm tra khả năng nhập form, nhấn nút đăng nhập và xác nhận vào dashboard.	Agent, replay, report
SC02	Đăng nhập với mật khẩu sai	Kiểm tra khả năng nhận diện thông báo lỗi và phân biệt expected failure.	Dataset, assertion, report
SC03	Đăng nhập thiếu thông tin	Kiểm tra xử lý validation message khi username hoặc password bị bỏ trống.	Dataset, replay, evidence
SC04	Điều hướng đến phân hệ quản lý nhân sự	Kiểm tra khả năng agent tìm menu và chuyển đến đúng màn hình chức năng.	Agent, object repository
SC05	Tìm kiếm hoặc lọc dữ liệu trong danh sách	Kiểm tra thao tác nhập điều kiện tìm kiếm và kiểm tra kết quả hiển thị.	Agent, replay, assertion
SC06	Sinh replay script từ phiên chạy thành công	Kiểm tra khả năng chuẩn hóa lịch sử agent thành script có thể chạy lại.	Replay script
SC07	Chạy lại replay script	Kiểm tra tính ổn định của script trong kịch bản hồi quy.	Playwright, locator fallback
SC08	Sinh và sử dụng nhiều bộ dữ liệu kiểm thử	Kiểm tra khả năng sinh dataset phục vụ các trường hợp đầu vào khác nhau.	Dataset generation
SC09	Ghi nhận report, screenshot và log	Kiểm tra khả năng lưu lại evidence và hiển thị kết quả sau khi chạy.	Evidence, report

Các kịch bản trên được chia thành ba nhóm. Nhóm thứ nhất kiểm tra luồng agent trên giao diện thật. Nhóm thứ hai kiểm tra khả năng tái sử dụng kết quả agent thông qua replay script. Nhóm thứ ba kiểm tra các chức năng hỗ trợ như dataset, evidence, report và phân tích lỗi.

6.4.2 Các chỉ số đánh giá

Để đánh giá hệ thống một cách nhất quán, khóa luận sử dụng kết hợp chỉ số định lượng và nhận xét định tính. Các chỉ số định lượng phù hợp với dữ liệu có thể đo được như số lần chạy thành công, thời gian sinh test case, thời gian chạy test hoặc số token sử dụng. Các nhận xét định tính được dùng cho những khía cạnh khó lượng hóa hơn như mức hữu ích của report hoặc khả năng hỗ trợ phân tích lỗi.

Bảng 6.4: Các chỉ số đánh giá hệ thống

Chỉ số	Ý nghĩa	Cách ghi nhận
Tỷ lệ thực thi thành công	Tỷ lệ test run có kết quả Pass trên tổng số lần chạy.	Log thực thi test case
Thời gian sinh test case	Thời gian LLM sinh test case từ prompt.	Log sinh test case
Thời gian thực thi	Thời gian agent hoặc replay script chạy trên trình duyệt.	Log thực thi test case
Tỷ lệ tái sử dụng	Tỷ lệ test run có thể tái sử dụng dưới dạng script hoặc kết quả chạy lại.	Log thực thi test case
Số token sử dụng	Số token đầu vào và đầu ra khi sinh test case hoặc khi agent thực thi.	Log sinh test case và log thực thi agent
Phân loại lỗi	Nhóm lỗi thường gặp như bad locator, prompt misunderstanding hoặc app error.	Log lỗi từng bước
Mức độ hài lòng người dùng	Mức độ người dùng đánh giá hệ thống dễ dùng, hữu ích và có tiềm năng hỗ trợ kiểm thử.	Khảo sát

Với các chỉ số dạng tỷ lệ, công thức tổng quát được sử dụng như sau:

$$\text{Tỷ lệ thành công} = \frac{\text{Số trường hợp đạt}}{\text{Tổng số trường hợp đánh giá}} \times 100\%$$

6.5 Kết quả thực nghiệm từ log hệ thống

6.5.1 Tổng quan kết quả thực nghiệm

Bảng 6.5 trình bày kết quả tổng hợp từ 167 lần thực thi trên case study OrangeHRM.

Bảng 6.5: Tổng hợp kết quả thực nghiệm trên OrangeHRM

Chỉ số	Giá trị
Tổng số test case/task	76
Tổng số lần thực thi	167
Tỷ lệ Pass tổng thể	83.2%
Tỷ lệ Fail tổng thể	16.8%
Tỷ lệ tái sử dụng	65.9%
Thời gian thực thi trung bình	70.12 giây

Các thông tin liên quan đến token và thời gian sinh test case được trình bày trong các bảng chi tiết của từng chức năng nhằm tránh trộn lẫn giữa dữ liệu tổng hợp theo test run và dữ liệu theo batch sinh test case.

Kết quả thực nghiệm cho thấy hệ thống đạt tỷ lệ Pass tổng thể 83.2% trên 167 lần thực thi. Tỷ lệ Fail là 16.8%, phản ánh các trường hợp hệ thống gặp lỗi trong quá trình thao tác với giao diện Web, kiểm tra assertion hoặc xử lý locator.

Tỷ lệ tái sử dụng đạt 65.9% nếu tính trên tổng số lần thực thi. Chỉ số này phản ánh tỷ lệ các lần chạy có thể tạo ra hoặc gắn với script có khả năng sử dụng lại cho các lần kiểm thử sau. Tuy nhiên, chỉ số này không đồng nghĩa với tỷ lệ replay thành công, vì replay script còn phụ thuộc vào locator, assertion và trạng thái giao diện tại thời điểm chạy lại.

Thời gian thực thi trung bình trên trình duyệt là 70.12 giây. Khi đối chiếu với kết quả sinh test case được trình bày riêng ở phần sau, có thể thấy phần tổn thời gian chính của hệ thống không nằm ở bước sinh test case, mà nằm ở quá trình agent hoặc replay script thao tác với trình duyệt, chờ trang tải, tìm phần tử và kiểm tra điều kiện kỳ vọng.

6.5.2 Kết quả theo nhóm kịch bản

Bảng 6.6 trình bày kết quả thực nghiệm theo nhóm kịch bản chính.

Bảng 6.6: Tỷ lệ thực thi thành công theo nhóm kịch bản chính

Nhóm kịch bản	Tỷ lệ Pass	Nhận xét
Đăng nhập và xác thực người dùng	90.09%	Đây là nhóm kịch bản đại diện cho các thao tác cơ bản của kiểm thử Web như nhập dữ liệu vào form, nhấn nút và kiểm tra trạng thái sau đăng nhập.
Các kịch bản mở rộng ngoài đăng nhập	65.31%	Bao gồm các kịch bản điều hướng, kiểm tra giao diện hoặc các prompt khó phân loại theo module cụ thể.

Kết quả theo nhóm kịch bản cho thấy nhóm đăng nhập và xác thực người dùng đạt tỷ lệ Pass 90.09%. Điều này cho thấy hệ thống xử lý khá tốt các thao tác cơ bản của kiểm thử Web như nhập dữ liệu vào form, nhấn nút và kiểm tra trạng thái sau đăng nhập.

Đối với các kịch bản mở rộng ngoài đăng nhập, tỷ lệ Pass đạt 65.31%. Nhóm này bao gồm nhiều loại yêu cầu khác nhau như điều hướng, kiểm tra giao diện hoặc các prompt khó phân loại theo module cụ thể. Tỷ lệ Pass thấp hơn cho thấy khi phạm vi kịch bản mở rộng, hệ thống dễ bị ảnh hưởng bởi locator, trạng thái giao diện, dữ liệu đầu vào và độ rõ ràng của prompt.

Một số module như Admin, PIM, Leave hoặc Time & Attendance có xuất hiện trong log thực nghiệm, tuy nhiên số lần chạy còn thấp nên không được tách riêng để đưa ra nhận xét định lượng. Các kết quả này chỉ được dùng như quan sát bổ sung, chưa đủ cơ sở để kết luận về độ ổn định của hệ thống trên từng module.

6.5.3 Kết quả sinh test case

Chức năng sinh test case được đánh giá thông qua 64 batch sinh test case. Mỗi batch tương ứng với một lần người dùng nhập prompt và hệ

thống gọi LLM để sinh ra test case hoặc danh sách ứng viên test case.

Bảng 6.7: Kết quả sinh test case từ prompt

Chỉ số	Giá trị
Số batch sinh test case	64
Thời gian sinh thấp nhất	2.00 giây
Thời gian sinh cao nhất	5.63 giây
Thời gian sinh trung bình	3.66 giây
Tổng token sử dụng	56,614
Token trung bình mỗi batch	884.59
Tổng input tokens	26,910
Tổng output tokens	29,704

Kết quả cho thấy thời gian sinh test case tương đối thấp, trung bình khoảng 3.66 giây mỗi batch. So với thời gian thực thi test trên trình duyệt, bước sinh test case không phải là nút thắt chính về thời gian. Vai trò chính của chức năng này là giảm công sức chuẩn bị kịch bản ban đầu, đặc biệt đối với người dùng chưa quen viết test case hoặc test script tự động.

Tuy nhiên, test case sinh ra từ LLM vẫn cần được người dùng kiểm tra lại trước khi chạy chính thức. Các thông tin cần kiểm tra gồm dữ liệu đầu vào, điều kiện kỳ vọng, bước thao tác và phạm vi tác động lên website mục tiêu.

6.5.4 Kết quả sinh dataset

Bên cạnh sinh test case, hệ thống còn hỗ trợ sinh dataset để phục vụ các kịch bản có nhiều bộ dữ liệu đầu vào, chẳng hạn như đăng nhập hợp lệ, sai mật khẩu hoặc thiếu thông tin.

Bảng 6.8: Kết quả sinh dataset

Chỉ số	Giá trị
Kết quả sinh dataset trong các lần ghi nhận	Thành công
Tổng số dòng dữ liệu sinh ra	65
Thời gian sinh thấp nhất	1.71 giây
Thời gian sinh cao nhất	4.59 giây
Thời gian sinh trung bình	2.57 giây
Tổng token sử dụng	13,075
Token trung bình mỗi lần sinh	1,089.58

Trong các lần ghi nhận thuộc phạm vi thực nghiệm, chức năng sinh dataset đều hoàn thành thành công. Kết quả này cho thấy chức năng có thể hỗ trợ người dùng chuẩn bị dữ liệu kiểm thử với chi phí thời gian thấp, với thời gian sinh trung bình 2.57 giây. Tuy nhiên, dữ liệu hiện tại mới phản ánh quá trình sinh dataset, chưa phản ánh đầy đủ kết quả chạy từng dòng dữ liệu. Do đó, dataset generation được xem là chức năng hỗ trợ chuẩn bị dữ liệu, còn hiệu quả của data-driven testing cần được đánh giá sâu hơn khi hệ thống lưu đầy đủ log theo từng dataset row.

6.6 So sánh chế độ agent và replay script

Hệ thống hỗ trợ hai chế độ chạy chính: chạy ban đầu bằng agent và chạy lại bằng replay script. Bảng 6.9 trình bày kết quả so sánh dựa trên log thực nghiệm.

Bảng 6.9: So sánh kết quả thực thi giữa agent và replay script

Chế độ chạy	Số lần chạy	Pass	Fail	Tỷ lệ Pass
Agent initial run	121	110	11	90.91%
Replay script	46	29	17	63.04%

Kết quả cho thấy agent initial run đạt tỷ lệ Pass 90.91%, cao hơn replay script với tỷ lệ Pass 63.04%. Điều này phản ánh sự khác biệt giữa hai chế độ chạy. Agent có khả năng quan sát giao diện hiện tại và suy luận hành động tiếp theo, do đó linh hoạt hơn trong các tình huống giao diện thay đổi hoặc trạng thái trang chưa ổn định. Ngược lại, replay script phụ thuộc nhiều hơn vào các bước và locator đã lưu trước đó, nên dễ bị ảnh hưởng khi giao diện thay đổi, phần tử chưa sẵn sàng hoặc assertion không còn phù hợp.

Bên cạnh tỷ lệ Pass, thời gian thực thi cần được phân tích riêng theo kết quả chạy để tránh diễn giải sai. Bảng 6.10 trình bày thời gian trung bình của từng nhóm pass/fail.

Bảng 6.10: So sánh thời gian thực thi theo kết quả chạy

Nhóm thực thi	Điều kiện lọc	Tỷ lệ	Thời gian TB
Agent thành công	Initial run có kết quả Pass	90.91%	63.96 giây
Agent thất bại	Initial run có kết quả Fail	9.09%	135.10 giây
Replay thành công	Replay script có kết quả Pass	63.04%	54.59 giây
Replay thất bại	Replay script có kết quả Fail	36.96%	94.43 giây

Khi xét riêng các lần chạy thành công, replay script có thời gian thực thi trung bình 54.59 giây, thấp hơn agent initial run thành công với thời gian trung bình 63.96 giây. Như vậy, đối với các kịch bản replay thành công, thời gian thực thi giảm khoảng 14.65% so với agent. Kết quả này phù hợp với mục tiêu của replay script là giảm phụ thuộc vào quá trình suy luận của agent và phát lại các bước đã được lưu trước đó.

Tuy nhiên, khi replay thất bại, thời gian trung bình tăng lên 94.43 giây. Nguyên nhân là các phiên thất bại thường phải chờ timeout ở bước tìm locator, điều hướng hoặc kiểm tra assertion. Vì vậy, nếu chỉ xét thời gian

trung bình tổng thể mà không tách pass/fail, kết quả có thể gây hiểu nhầm rằng replay không nhanh hơn agent. Kết quả này cho thấy replay có lợi thế rõ hơn về thời gian trong các kịch bản đã ổn định và chạy thành công, nhưng chưa thể thay thế hoàn toàn agent ở các kịch bản mới hoặc có giao diện biến động.

Bên cạnh kết quả trung bình, một kịch bản cụ thể cũng cho thấy lợi ích của replay script khi kịch bản đã ổn định. Bảng 6.11 trình bày ví dụ so sánh giữa lần chạy ban đầu bằng agent và lần chạy lại bằng replay script trên cùng test case.

Bảng 6.11: Ví dụ so sánh thời gian giữa lần chạy agent và replay thành công

Lần chạy	Run ID	Thời gian	Kết quả
Chạy ban đầu bằng agent	Run #84	87.6 giây	Pass
Chạy lại bằng replay script	Run #232	22.1 giây	Pass

Trong ví dụ này, lần chạy ban đầu bằng agent mất 87.6 giây, trong khi lần chạy lại bằng replay script chỉ mất 22.1 giây. Như vậy, thời gian thực thi giảm khoảng 74.77%, tương đương replay nhanh hơn khoảng 4.0 lần so với lần chạy ban đầu. Kết quả này minh họa lợi ích của replay script đối với các kịch bản đã chạy thành công và có thể tái sử dụng.

Tuy nhiên, mức cải thiện này phụ thuộc vào từng kịch bản cụ thể. Khi tính trung bình trên toàn bộ dữ liệu, kết quả có thể bị ảnh hưởng bởi các lần replay thất bại hoặc timeout.

Tổng thời gian thực thi của agent là 8,522.1 giây, trong khi tổng thời gian thực thi của replay script là 3,188.5 giây. Tuy nhiên, do số lần chạy của agent và replay script không bằng nhau, tổng thời gian thực thi của hai nhóm không được dùng để so sánh trực tiếp. Thay vào đó, khóa luận sử dụng thời gian trung bình theo từng nhóm kết quả pass/fail để phản ánh chính xác hơn đặc điểm vận hành của từng chế độ.

Từ kết quả trên có thể thấy agent và replay script có vai trò bổ sung cho nhau. Agent phù hợp với giai đoạn chạy ban đầu, khi hệ thống cần quan sát giao diện và suy luận hành động từ mục tiêu kiểm thử. Replay script phù hợp hơn với các kịch bản đã chạy thành công và có thể tái sử dụng, vì khi replay thành công, thời gian thực thi trung bình thấp hơn agent thành công khoảng 14.65%.

Tuy nhiên, replay script hiện vẫn có tỷ lệ thất bại cao hơn agent. Điều này cho thấy hệ thống cần tiếp tục cải thiện chất lượng locator, cơ chế chờ, assertion và khả năng tự phục hồi khi giao diện thay đổi. Vì vậy, hướng tiếp cận phù hợp là sử dụng agent để tạo và khám phá kịch bản ban đầu, sau đó dùng replay script để chạy lại các kịch bản đã ổn định trong kiểm thử hồi quy.

6.7 Phân tích lỗi trong quá trình thực nghiệm

6.7.1 Phân loại lỗi theo failure type

Trong các phiên thực thi thất bại, một phần lỗi được hệ thống phân loại rõ ràng trong failure analysis. Do số lượng lỗi đã phân loại còn thấp, Bảng 6.12 chỉ trình bày tỷ lệ theo loại lỗi để quan sát xu hướng thay vì nhấn mạnh vào số lượng tuyệt đối.

Bảng 6.12: Phân loại lỗi trong quá trình thực nghiệm

Loại lỗi	Tỷ lệ trong các lỗi đã phân loại
Bad locator	70.59%
Prompt misunderstanding	23.53%
App error	5.88%

Lỗi phổ biến nhất là Bad locator, chiếm 70.59% trong các lỗi đã được phân loại. Đây là nhóm lỗi xảy ra khi hệ thống không tìm thấy phần tử, phần tử chưa hiển thị hoặc locator không còn phù hợp tại thời điểm chạy.

Kết quả này phù hợp với đặc thù của kiểm thử giao diện Web: các phần tử có thể được render động, thay đổi cấu trúc DOM hoặc xuất hiện chậm hơn so với thời điểm script tìm kiếm.

Nhóm lỗi Prompt misunderstanding chiếm 23.53%. Lỗi này xảy ra khi prompt, expected result hoặc điều kiện kiểm tra chưa đủ rõ, khiến agent hoặc assertion đánh giá sai kết quả. Ví dụ, một kịch bản đăng nhập không hợp lệ có thể được xem là thành công nếu expected result là hiển thị thông báo lỗi, nhưng có thể bị đánh giá fail nếu hệ thống chỉ kiểm tra việc điều hướng sang dashboard.

Bên cạnh các lỗi đã được phân loại, vẫn còn một số lần thất bại chưa có failure type rõ ràng trong log. Điều này cho thấy hệ thống cần cải thiện việc gán nguyên nhân lỗi cho mọi phiên chạy thất bại, chẳng hạn bổ sung nhóm Unknown hoặc trích xuất nguyên nhân từ error message và step log.

6.7.2 Phân tích lỗi theo step thất bại

Bên cạnh phân loại lỗi theo run, hệ thống còn ghi nhận thông tin về các step thất bại. Do số lượng step lỗi được ghi nhận còn thấp, Bảng 6.13 chỉ trình bày tỷ lệ theo loại hành động để quan sát xu hướng lỗi thay vì nhấn mạnh vào số lượng tuyệt đối.

Bảng 6.13: Tỷ lệ step thất bại theo loại hành động

Loại hành động	Tỷ lệ trong các step thất bại
click	41.67%
assert_url_changed	29.17%
assert_visible	12.50%
input	8.33%
fill	4.17%
write_file	4.17%

Hành động click chiếm tỷ lệ cao nhất trong các step thất bại được ghi nhận. Điều này cho thấy thao tác click phụ thuộc nhiều vào khả

năng xác định đúng phần tử, trạng thái hiển thị của phần tử và thời điểm trang đã sẵn sàng để tương tác. Các lỗi liên quan đến assertion như `assert_url_changed` và `assert_visible` cũng chiếm tỷ lệ đáng kể, cho thấy việc thiết kế điều kiện kiểm tra kết quả là yếu tố quan trọng trong hệ thống kiểm thử tự động.

Từ kết quả trên, có thể rút ra rằng việc cải thiện độ ổn định của hệ thống cần tập trung vào ba hướng: tăng chất lượng locator, bổ sung cơ chế chờ thông minh hơn và thiết kế assertion rõ ràng hơn cho từng loại test case.

6.8 Đánh giá các cơ chế hỗ trợ

6.8.1 Object repository và self-healing

Object repository được thiết kế để lưu thông tin định vị phần tử và tái sử dụng trong replay script. Thay vì chỉ lưu một selector duy nhất, hệ thống có thể lưu nhiều thuộc tính như CSS selector, XPath, text, placeholder, accessible name hoặc các thông tin liên quan đến phần tử. Cách tiếp cận này giúp replay script có thêm lựa chọn khi locator chính không còn phù hợp.

Trong dữ liệu thực nghiệm hiện tại, lỗi Bad locator là nhóm lỗi phổ biến nhất. Điều này cho thấy object repository và locator fallback là hướng cải thiện cần thiết. Tuy nhiên, log export hiện tại chưa ghi nhận đầy đủ số lần primary locator thất bại, số lần fallback được dùng và số lần self-healing thành công. Vì vậy, trong phạm vi Chương này, object repository và self-healing được đánh giá chủ yếu theo hướng định tính và thông qua phân tích lỗi locator.

Kết quả thực nghiệm cho thấy self-healing nên được hiểu là cơ chế hỗ trợ phục hồi trước một số thay đổi nhỏ của locator, không phải khả năng tự động sửa mọi lỗi kiểm thử. Nếu giao diện thay đổi mạnh, phần tử bị xóa hoặc luồng nghiệp vụ thay đổi, người dùng vẫn cần cập nhật test case,

replay script hoặc object repository.

6.8.2 Evidence, log và report

Evidence đóng vai trò quan trọng trong việc giúp người dùng hiểu một phiên chạy test. Một report hữu ích cần cho biết test pass hay fail, lỗi xảy ra ở bước nào, URL tại thời điểm lỗi, screenshot liên quan, log thao tác và error message.

Trong dữ liệu thực nghiệm, hệ thống đã ghi nhận được execution result, execution time, run type, run note, failure type và step failure. Các thông tin này cho phép phân tích tổng quan tỷ lệ pass/fail, phân loại lỗi và xác định nhóm hành động thường thất bại. Tuy nhiên, để đánh giá đầy đủ hơn về evidence, hệ thống cần export thêm các chỉ số như số screenshot mỗi run, số step log mỗi run, số report có AI analysis và mức độ đầy đủ của timeline.

Dù vậy, kết quả khảo sát người dùng cho thấy các thành phần report, screenshot, log và timeline được đánh giá tương đối tích cực. Điều này được trình bày chi tiết ở phần đánh giá trải nghiệm người dùng.

6.8.3 Phân tích lỗi bằng mô hình ngôn ngữ lớn

Phân tích lỗi bằng mô hình ngôn ngữ lớn có vai trò hỗ trợ người dùng đọc log và đưa ra gợi ý nguyên nhân thất bại. Cơ chế này đặc biệt hữu ích khi người dùng không quen đọc error message kỹ thuật hoặc không biết phân biệt lỗi do dữ liệu, lỗi do locator, lỗi do timeout và lỗi do ứng dụng.

Trong phạm vi thực nghiệm hiện tại, hệ thống đã có dữ liệu failure type và notes để hỗ trợ phân tích lỗi. Tuy nhiên, chưa có bộ dữ liệu đánh giá riêng để chấm mức độ chính xác của AI analysis, ví dụ AI xác định đúng bước lỗi hay phân loại đúng nguyên nhân lỗi. Vì vậy, phần AI analysis được đánh giá chủ yếu thông qua phản hồi người dùng trong khảo sát. Trong các hướng phát triển tiếp theo, cần bổ sung bộ tiêu chí đánh giá AI

analysis như: xác định đúng step lỗi, phân loại đúng root cause, gợi ý xử lý hữu ích và không sinh thông tin không có trong evidence.

6.9 Đánh giá trải nghiệm người dùng

6.9.1 Mục tiêu khảo sát

Bên cạnh các số liệu thực nghiệm thu được từ log hệ thống, nhóm thực hiện thêm khảo sát nhỏ nhằm thu thập phản hồi của người dùng sau khi trải nghiệm hệ thống. Khảo sát tập trung vào các khía cạnh: giao diện và khả năng sử dụng, chức năng sinh test case, quá trình chạy test, report/evidence và mức độ hữu ích tổng thể.

Khảo sát được thực hiện ở quy mô nhỏ với nhóm người dùng thử hệ thống. Các câu hỏi đánh giá sử dụng thang điểm 1–5, trong đó 1 thể hiện mức độ không đồng ý và 5 thể hiện mức độ đồng ý cao.

Do đó, các kết quả khảo sát được sử dụng như nguồn tham khảo định tính nhằm bổ sung cho dữ liệu log thực nghiệm.

6.9.2 Đối tượng tham gia khảo sát

Bảng 6.14: Nhóm người tham gia khảo sát

Nhóm người dùng	Tỷ lệ
Sinh viên ngành Công nghệ thông tin	47.1%
Lập trình viên	17.6%
Người dùng chưa có nhiều kinh nghiệm về kiểm thử phần mềm	17.6%
Tester / QA	11.8%
Khác	5.9%

Nhóm người tham gia khảo sát tương đối phù hợp với phạm vi đề tài vì bao gồm cả người có nền tảng công nghệ thông tin, lập trình viên,

tester/QA và người chưa có nhiều kinh nghiệm kiểm thử phần mềm.

Bảng 6.15: Kinh nghiệm của người tham gia với kiểm thử tự động Web

Mức độ kinh nghiệm	Tỷ lệ
Chưa từng sử dụng công cụ kiểm thử tự động Web	35.3%
Đã từng sử dụng cơ bản	29.4%
Đã từng sử dụng thường xuyên	17.6%
Đã từng nghe qua nhưng chưa sử dụng	17.6%

Có 35.3% người tham gia chưa từng sử dụng công cụ kiểm thử tự động Web. Điều này phù hợp với mục tiêu của hệ thống là giúp người dùng giảm rào cản khi tiếp cận kiểm thử tự động thông qua việc nhập yêu cầu bằng ngôn ngữ tự nhiên thay vì phải viết script ngay từ đầu.

6.9.3 Kết quả khảo sát theo nhóm tiêu chí

Bảng 6.16 tổng hợp điểm khảo sát theo các nhóm tiêu chí chính.

Bảng 6.16: Tổng hợp kết quả khảo sát theo nhóm tiêu chí

Nhóm tiêu chí	Điểm trung bình	Tỷ lệ đánh giá 4–5
Giao diện và khả năng sử dụng	4.28/5	83.5%
Sinh test case từ prompt	4.24/5	80.9%
Quá trình chạy test	3.84/5	75.0%
Kết quả, evidence và report	4.13/5	80.4%
Giá trị tổng thể và mức độ hài lòng	4.04/5	76.5%

Kết quả khảo sát cho thấy người dùng đánh giá tích cực giao diện, khả năng nhập yêu cầu kiểm thử bằng ngôn ngữ tự nhiên và chức năng sinh test case. Nhóm tiêu chí giao diện và khả năng sử dụng đạt điểm trung bình 4.28/5, trong khi nhóm sinh test case từ prompt đạt 4.24/5. Điều

này cho thấy hệ thống có khả năng hỗ trợ người dùng tạo kịch bản kiểm thử ban đầu dễ dàng hơn so với việc viết test case hoặc test script hoàn toàn thủ công.

Nhóm kết quả, evidence và report đạt điểm trung bình 4.13/5. Các câu hỏi liên quan đến log, timeline và phân tích lỗi được đánh giá khá tốt, cho thấy việc lưu evidence và hiển thị lại quá trình thực thi giúp người dùng hiểu rõ hơn kết quả kiểm thử.

Tuy nhiên, nhóm tiêu chí liên quan đến quá trình chạy test có điểm trung bình thấp hơn, đạt 3.84/5. Trong đó, câu hỏi về việc người dùng có thể hiểu hệ thống đang thực hiện đến bước nào trong quá trình chạy test chỉ đạt 3.47/5. Đây là một điểm cần cải thiện, cho thấy hệ thống cần hiển thị trạng thái thực thi rõ ràng hơn theo thời gian thực, chẳng hạn bổ sung progress indicator, mô tả step đang chạy và phân biệt rõ các trạng thái đang chờ, đang thao tác, thất bại hoặc hoàn thành.

6.9.4 Mức độ chỉnh sửa test case sinh ra

Một nội dung quan trọng trong khảo sát là mức độ chỉnh sửa test case sinh ra trước khi sử dụng. Kết quả được trình bày trong Bảng 6.17.

Bảng 6.17: Mức độ chỉnh sửa test case sinh ra trước khi sử dụng

Mức độ chỉnh sửa	Số phản hồi	Tỷ lệ
Không cần chỉnh sửa	3	17.6%
Chỉnh sửa rất ít	5	29.4%
Chỉnh sửa một phần	9	52.9%

Kết quả cho thấy 52.9% người tham gia cho rằng test case cần chỉnh sửa một phần trước khi sử dụng, 29.4% cho rằng chỉ cần chỉnh sửa rất ít và 17.6% cho rằng không cần chỉnh sửa. Điều này cho thấy test case sinh ra từ mô hình ngôn ngữ lớn có giá trị như bản nháp ban đầu, giúp giảm công sức chuẩn bị kiểm thử, nhưng vẫn cần người dùng kiểm tra lại dữ liệu đầu vào, step và điều kiện kỳ vọng trước khi chạy chính thức.

6.10 Nhận xét tổng hợp

Dựa trên các kết quả định lượng từ log hệ thống và phản hồi khảo sát người dùng, có thể tổng hợp một số nhận xét chính về hai chế độ thực thi của hệ thống như sau. Bảng 6.18 so sánh hai chế độ chạy chính của hệ thống là agent và replay script.

Bảng 6.18: So sánh agent và replay script

Tiêu chí	Agent	Replay script
Đầu vào	Goal hoặc prompt bằng ngôn ngữ tự nhiên.	Script đã được sinh hoặc chỉnh sửa.
Mức độ phụ thuộc LLM	Cao hơn vì cần suy luận hành động trong quá trình chạy.	Thấp hơn vì chạy theo các step đã lưu.
Tính linh hoạt	Phù hợp khi giao diện cần được quan sát và diễn giải theo ngữ cảnh.	Phù hợp với luồng đã ổn định và cần chạy lặp lại.
Tính xác định	Có thể thay đổi theo trạng thái giao diện và phản hồi của LLM.	Ổn định hơn về mặt luồng thao tác, nhưng phụ thuộc locator và assertion.
Khả năng bảo trì	Ít yêu cầu selector ban đầu, nhưng khó kiểm soát nếu prompt mơ hồ.	Cần bảo trì locator, assertion và dữ liệu khi website thay đổi.
Chi phí vận hành	Cao hơn do gọi LLM trong quá trình chạy.	Thấp hơn trong các lần chạy lặp lại vì chủ yếu sử dụng Playwright.
Vai trò phù hợp	Khám phá luồng, tạo kịch bản ban đầu, xử lý tình huống cần suy luận.	Kiểm thử hồi quy, chạy dataset, chạy suite.

Tổng hợp các kết quả trên cho thấy agent và replay script đóng vai trò bổ sung cho nhau. Agent phù hợp với giai đoạn khám phá và chạy test ban đầu, trong khi replay script phù hợp với các kịch bản đã ổn định và

cần chạy lặp lại. Kết quả khảo sát cũng cho thấy người dùng đánh giá cao khả năng nhập yêu cầu kiểm thử bằng ngôn ngữ tự nhiên và sinh test case từ prompt, nhưng hệ thống vẫn cần cải thiện khả năng hiển thị trạng thái thực thi trong lúc chạy test.

Bảng 6.19: Tổng hợp mức độ đáp ứng các mục tiêu đánh giá

Mục tiêu đánh giá	Kết quả ghi nhận	Nhận xét
Sinh test case từ mô tả tự nhiên	Thời gian sinh trung bình 3.66 giây mỗi batch.	Đáp ứng mục tiêu hỗ trợ tạo test case ban đầu.
Thực thi test bằng agent	Agent initial run đạt tỷ lệ Pass 90.91%.	Phù hợp với giai đoạn chạy ban đầu và khám phá luồng.
Chạy lại bằng replay script	Replay script đạt tỷ lệ Pass 63.04%; khi replay thành công, thời gian trung bình thấp hơn agent thành công 14.65%.	Có tiềm năng cho kiểm thử hồi quy nhưng cần cải thiện locator và assertion.
Sinh dữ liệu kiểm thử	Chức năng sinh dataset hoàn thành trong các lần ghi nhận; thời gian sinh trung bình 2.57 giây.	Hỗ trợ chuẩn bị dữ liệu, nhưng cần đánh giá thêm theo từng dataset row.
Phân tích lỗi	Bad locator là lỗi phổ biến nhất trong các lỗi đã phân loại.	Giúp xác định hướng cải thiện object repository, locator fallback và timeout.
Trải nghiệm người dùng	Các nhóm tiêu chí chính đạt điểm trung bình từ 3.84/5 đến 4.28/5.	Người dùng đánh giá tích cực, nhưng cần cải thiện hiển thị tiến trình chạy test.

6.11 Các yếu tố ảnh hưởng và giới hạn thực nghiệm

Kết quả thực nghiệm chịu ảnh hưởng bởi nhiều yếu tố khác nhau. Các yếu tố quan trọng gồm:

- **Chất lượng prompt:** prompt càng rõ về mục tiêu, dữ liệu đầu vào và kết quả mong đợi thì test case và hành động agent càng dễ đúng.
- **Trạng thái website:** trạng thái đăng nhập, dữ liệu hiện có, quyền tài khoản và phiên làm việc ảnh hưởng trực tiếp đến kết quả test.
- **Tốc độ tải trang:** website tải chậm hoặc phản hồi bất đồng bộ có thể làm locator chưa sẵn sàng tại thời điểm agent hoặc replay thao tác.
- **Độ ổn định của locator:** selector thay đổi hoặc phần tử được render động có thể làm replay script thất bại.
- **Dữ liệu kiểm thử:** dữ liệu hết hạn, bị thay đổi hoặc không còn tồn tại có thể khiến test fail dù hệ thống kiểm thử vẫn hoạt động đúng.
- **Cấu hình runtime:** timeout, browser profile, allowed domains và số phiên chạy đồng thời ảnh hưởng đến độ ổn định của worker.
- **Phản hồi của LLM:** trong chế độ agent, kết quả có thể thay đổi theo cách mô hình diễn giải goal và trạng thái giao diện.
- **Thiết kế assertion:** assertion quá chung có thể bỏ sót lỗi, trong khi assertion quá chặt có thể làm test fail khi giao diện thay đổi nhỏ.

Bên cạnh đó, quá trình đánh giá hiện tại còn có một số giới hạn:

- Dữ liệu thực nghiệm tập trung nhiều vào nhóm Login, trong khi các module khác như PIM, Leave, Admin hoặc Time & Attendance có số lần chạy còn ít.

- Dữ liệu log được lọc theo các project có website mục tiêu liên quan đến OrangeHRM. Do đó, các bản ghi thuộc cùng website mục tiêu có thể đến từ nhiều project thử nghiệm khác nhau trong quá trình phát triển hệ thống. Vì vậy, kết quả cần được xem là dữ liệu thực nghiệm tổng hợp cho case study OrangeHRM, không phải benchmark tuyệt đối trên một project duy nhất.
- Chỉ số Generated Tree Correct chưa được đánh giá thủ công; toàn bộ bản ghi trong log hiện tại vẫn ở trạng thái chưa đánh giá. Vì vậy, chỉ số này chưa được dùng để kết luận định lượng.
- Dữ liệu về crawl website, test suite run, dataset row run, self-healing và AI analysis chưa được ghi nhận hoặc export đầy đủ thành các bảng định lượng riêng.
- Khảo sát người dùng có số lượng phản hồi còn hạn chế, phù hợp để tham khảo định tính nhưng chưa đủ lớn để đại diện cho toàn bộ nhóm người dùng mục tiêu.

Những giới hạn trên cho thấy kết quả đánh giá cần được hiểu trong phạm vi case study và điều kiện thực nghiệm hiện tại. Để có kết luận tổng quát hơn, hệ thống cần được chạy trên nhiều website hơn, nhiều module nghiệp vụ hơn và với số lần lặp lại lớn hơn.

6.12 Kết chương

Chương này đã trình bày quá trình thực nghiệm và đánh giá hệ thống kiểm thử tự động ứng dụng Web thông qua case study OrangeHRM. Kết quả từ log hệ thống cho thấy trong 167 lần thực thi, hệ thống đạt 139 lần Pass và 28 lần Fail, tương ứng tỷ lệ Pass 83.2%. Thời gian sinh test case trung bình theo batch là 3.66 giây, trong khi thời gian thực thi trung bình trên trình duyệt là 70.12 giây. Kết quả này cho thấy hệ thống có khả năng

hỗ trợ quy trình kiểm thử từ prompt ngôn ngữ tự nhiên đến thực thi trên trình duyệt và ghi nhận kết quả.

Khi so sánh hai chế độ chạy, agent initial run đạt tỷ lệ Pass 90.91%, cao hơn replay script với tỷ lệ 63.04%. Tuy nhiên, nếu chỉ xét các lần chạy thành công, replay script có thời gian trung bình 54.59 giây, thấp hơn agent thành công với 63.96 giây. Điều này cho thấy agent linh hoạt hơn trong giai đoạn khám phá và chạy test ban đầu, trong khi replay script phù hợp với các kịch bản đã ổn định và cần tiếp tục được cải thiện về locator, assertion và xử lý trạng thái để nâng cao tỷ lệ thành công.

Phân tích lỗi cho thấy Bad locator là nhóm lỗi phổ biến nhất trong các lỗi đã được phân loại, chiếm 70.59%. Điều này nhấn mạnh vai trò của object repository, locator fallback và cơ chế chờ phù hợp trong hệ thống kiểm thử giao diện Web. Ngoài ra, các lỗi liên quan đến prompt misunderstanding và assertion cũng cho thấy người dùng cần mô tả rõ mục tiêu kiểm thử, dữ liệu đầu vào và kết quả kỳ vọng.

Kết quả khảo sát người dùng cho thấy hệ thống được đánh giá tích cực về giao diện, khả năng sinh test case từ prompt, report/evidence và giá trị hỗ trợ kiểm thử. Tuy nhiên, người dùng vẫn gặp khó khăn trong việc theo dõi hệ thống đang thực hiện đến bước nào trong quá trình chạy test. Đây là một hướng cải thiện quan trọng trong các phiên bản tiếp theo.

Tổng hợp lại, kết quả thực nghiệm cho thấy hướng tiếp cận kết hợp mô hình ngôn ngữ lớn, agent, browser automation, replay script và evidence-based report là khả thi trong phạm vi case study OrangeHRM. Đồng thời, kết quả thực nghiệm cũng chỉ ra các giới hạn cần tiếp tục cải thiện như độ ổn định của replay script, khả năng tự phục hồi locator, đánh giá AI analysis và mở rộng thực nghiệm sang nhiều website khác.

Chương 7

Tổ chức thực hiện đề tài

7.1 Phân chia công việc trong nhóm

Đề tài được thực hiện bởi nhóm gồm hai thành viên: Nguyễn Chí Công và Nguyễn Hưng. Do hệ thống có nhiều thành phần như frontend, backend, cơ sở dữ liệu, agent-worker, browser automation và báo cáo thực nghiệm, nhóm đã phân chia công việc theo hướng mỗi thành viên phụ trách một số mảng chính nhưng vẫn phối hợp trong các phần có liên quan.

7.1.1 Vai trò của từng thành viên

Bảng 7.1 trình bày vai trò chính của từng thành viên trong quá trình thực hiện đề tài.

Bảng 7.1: Vai trò của từng thành viên trong nhóm

Thành viên	Vai trò chính
Nguyễn Chí Công	Tham gia phân tích yêu cầu, xây dựng giao diện người dùng, hỗ trợ thiết kế luồng chức năng, kiểm tra giao diện và hoàn thiện tài liệu báo cáo.
Nguyễn Hưng	Tham gia thiết kế kiến trúc hệ thống, xây dựng backend API, cơ sở dữ liệu, agent-worker, tích hợp Browser-use, Playwright, AI và hỗ trợ thực nghiệm hệ thống.

Việc phân chia trên giúp nhóm có định hướng rõ ràng trong quá trình phát triển. Tuy nhiên, các phần quan trọng như phân tích yêu cầu, kiểm thử chức năng, tích hợp hệ thống và viết báo cáo vẫn được hai thành viên cùng trao đổi để đảm bảo tính thống nhất.

7.1.2 Phạm vi công việc của từng thành viên

Phạm vi công việc được chia theo từng nhóm chức năng chính của hệ thống. Thành viên phụ trách chính sẽ đảm nhiệm việc xây dựng, chỉnh sửa và kiểm tra phần việc tương ứng, trong khi thành viên còn lại hỗ trợ góp ý, kiểm thử và tích hợp khi cần.

Bảng 7.2: Phân chia phạm vi công việc

Hạng mục	Người phụ trách chính	Nội dung thực hiện
Phân tích yêu cầu	Cả nhóm	Xác định chức năng hệ thống, nhóm người dùng, use case và yêu cầu phi chức năng.
Frontend	Nguyễn Chí Công	Xây dựng giao diện project, test case, dataset, test suite, object repository và report.
Backend API	Nguyễn Hưng	Xây dựng API, xử lý nghiệp vụ, quản lý dữ liệu và điều phối yêu cầu chạy test.
Cơ sở dữ liệu	Nguyễn Hưng	Thiết kế schema PostgreSQL, các bảng lưu project, test case, test run, evidence, replay script và object repository.
Agent-worker	Nguyễn Hưng	Xây dựng worker bằng Python FastAPI, tích hợp Browser-use, Playwright và cơ chế callback kết quả.
Kiểm thử và thực nghiệm	Cả nhóm	Chạy thử các luồng chức năng, ghi nhận lỗi, kiểm tra report và đánh giá kết quả.
Viết báo cáo	Cả nhóm	Tổng hợp nội dung, mô tả hệ thống, trình bày kiến trúc, thực nghiệm và kết luận.

7.1.3 Cách phối hợp giữa các thành viên

Trong quá trình thực hiện, nhóm phối hợp theo từng giai đoạn nhỏ. Trước khi phát triển một chức năng, nhóm trao đổi để thống nhất mục tiêu, luồng xử lý và dữ liệu cần lưu. Sau khi một thành viên hoàn thành phần việc của mình, thành viên còn lại kiểm tra, chạy thử và góp ý chỉnh sửa.

Đối với các chức năng cần tích hợp nhiều thành phần như chạy test bằng agent, sinh replay script hoặc hiển thị report, nhóm thực hiện theo hướng chia nhỏ luồng xử lý. Backend và worker được kiểm tra trước, sau đó frontend được kết nối để hoàn thiện luồng người dùng cuối. Cách làm này giúp hạn chế lỗi tích hợp và dễ xác định nguyên nhân khi hệ thống gặp vấn đề.

7.2 Quy trình làm việc

7.2.1 Quy trình phát triển hệ thống

Quy trình phát triển hệ thống được thực hiện theo hướng lặp. Nhóm không xây dựng toàn bộ hệ thống trong một lần, mà chia thành nhiều nhóm chức năng nhỏ để phát triển, kiểm tra và cải tiến dần.

Các bước chính trong quy trình phát triển gồm:

1. Phân tích yêu cầu và xác định phạm vi chức năng.
2. Thiết kế kiến trúc tổng thể và mô hình dữ liệu.
3. Xây dựng các thành phần nền tảng như xác thực, project, test case và database.
4. Tích hợp agent-worker, Browser-use và Playwright để chạy test.
5. Bổ sung replay script, test data, test suite, object repository và report.

6. Chạy thử hệ thống, ghi nhận lỗi và điều chỉnh.
7. Hoàn thiện tài liệu báo cáo và kết quả thực nghiệm.

Cách tiếp cận này giúp nhóm kiểm soát tiến độ tốt hơn, đồng thời phát hiện sớm các vấn đề về kiến trúc, dữ liệu và tích hợp.

7.2.2 Quy trình quản lý mã nguồn

Mã nguồn được quản lý bằng Git và lưu trữ trên GitHub. Mỗi thành viên làm việc trên phần chức năng được phân công, sau đó commit và push mã nguồn lên repository chung. Trước khi tích hợp các thay đổi lớn, nhóm kiểm tra lại trạng thái hệ thống để hạn chế xung đột hoặc lỗi phát sinh.

Quy trình quản lý mã nguồn gồm các thao tác chính:

- Pull phiên bản mới nhất trước khi bắt đầu chỉnh sửa.
- Commit theo từng nhóm thay đổi có ý nghĩa.
- Push mã nguồn lên repository chung.
- Kiểm tra lại hệ thống sau khi tích hợp thay đổi.
- Sửa lỗi phát sinh nếu có xung đột hoặc chức năng không hoạt động đúng.

7.2.3 Quy trình kiểm tra và tích hợp chức năng

Sau khi một chức năng được xây dựng, nhóm kiểm tra ở hai mức. Mức thứ nhất là kiểm tra riêng từng thành phần, ví dụ kiểm tra API, kiểm tra truy vấn database hoặc kiểm tra worker chạy đúng request. Mức thứ hai là kiểm tra luồng tích hợp từ frontend đến backend, từ backend sang worker và từ worker callback kết quả về backend.

Các chức năng quan trọng như chạy agent, replay script, dataset run và report được kiểm tra nhiều lần vì các chức năng này phụ thuộc đồng thời

vào frontend, backend, database, worker và trình duyệt. Khi phát hiện lỗi, nhóm dựa vào log, trạng thái test run và response API để xác định lỗi nằm ở thành phần nào.

7.3 Kế hoạch thực hiện

7.3.1 Các giai đoạn thực hiện đề tài

Quá trình thực hiện đề tài được chia thành các giai đoạn chính như sau:

Bảng 7.3: Các giai đoạn thực hiện đề tài

Giai đoạn	Nội dung thực hiện	Kết quả chính
Giai đoạn 1	Tìm hiểu bài toán, khảo sát công cụ kiểm thử Web, LLM, AI agent và browser automation.	Xác định hướng tiếp cận và phạm vi đề tài.
Giai đoạn 2	Phân tích yêu cầu, xác định use case, yêu cầu chức năng và phi chức năng.	Hoàn thành phân tích yêu cầu hệ thống.
Giai đoạn 3	Thiết kế kiến trúc, mô hình dữ liệu và các luồng xử lý chính.	Hoàn thành thiết kế hệ thống.
Giai đoạn 4	Xây dựng frontend, backend, database và agent-worker.	Có phiên bản hệ thống chạy được các luồng chính.
Giai đoạn 5	Tích hợp Browser-use, Playwright, Gemini Flash, replay script, object repository và report.	Hoàn thiện các chức năng cốt lõi.
Giai đoạn 6	Thực nghiệm, đánh giá, sửa lỗi và hoàn thiện báo cáo.	Có kết quả đánh giá và báo cáo khóa luận.

7.3.2 Tiến độ thực hiện

Tiến độ thực hiện được tổ chức theo từng nhóm công việc. Mỗi nhóm công việc có thể được điều chỉnh tùy theo mức độ khó và các lỗi phát sinh trong quá trình tích hợp.

Bảng 7.4: Tiến độ thực hiện đề tài

Thời gian	Công việc chính	Trạng thái
Giai đoạn đầu	Tìm hiểu đề tài, khảo sát công nghệ và xác định phạm vi.	Hoàn thành
Giai đoạn phân tích	Phân tích yêu cầu, xây dựng use case và mô hình chức năng.	Hoàn thành
Giai đoạn thiết kế	Thiết kế kiến trúc, database và các luồng xử lý chính.	Hoàn thành
Giai đoạn hiện thực	Xây dựng frontend, backend, worker và tích hợp công nghệ lõi.	Hoàn thành cơ bản
Giai đoạn thực nghiệm	Chạy thử hệ thống, ghi nhận kết quả và đánh giá.	Đang hoàn thiện
Giai đoạn báo cáo	Tổng hợp nội dung, chỉnh sửa báo cáo và hoàn thiện tài liệu.	Đang hoàn thiện

7.3.3 Kết quả đạt được theo từng giai đoạn

Sau từng giai đoạn, nhóm đạt được các kết quả tương ứng với mục tiêu đã đặt ra. Kết quả quan trọng nhất là hệ thống đã hình thành được luồng kiểm thử tự động từ tạo project, tạo test case, chạy test bằng agent, sinh replay script, chạy replay và xem báo cáo kết quả.

Các kết quả đạt được gồm:

- Hoàn thành phân tích bài toán và xác định phạm vi hệ thống.
- Xây dựng được kiến trúc gồm frontend, backend, database và agent-worker.

- Xây dựng được cơ sở dữ liệu phục vụ quản lý project, test case, test run, replay script, evidence và object repository.
- Tích hợp được Browser-use, Playwright và Google Gemini Flash.
- Xây dựng được các chức năng chính như tạo test case, chạy agent, sinh replay script, chạy replay, quản lý dataset, test suite và report.
- Hoàn thiện phần báo cáo mô tả quá trình xây dựng và đánh giá hệ thống.

7.4 Công cụ hỗ trợ thực hiện

7.4.1 Công cụ quản lý mã nguồn

Nhóm sử dụng Git và GitHub để quản lý mã nguồn. Git giúp theo dõi lịch sử thay đổi, còn GitHub được dùng để lưu trữ repository chung và hỗ trợ các thành viên đồng bộ mã nguồn trong quá trình làm việc.

7.4.2 Công cụ phát triển và kiểm thử

Các công cụ phát triển chính gồm Visual Studio Code, Node.js, npm, Python, PostgreSQL và trình duyệt Chromium. Ngoài ra, nhóm sử dụng Playwright để kiểm thử thao tác trình duyệt và Browser-use để chạy agent trên website mục tiêu.

Trong quá trình phát triển, các API được kiểm tra thông qua giao diện hệ thống hoặc công cụ gửi request. Các lỗi phát sinh được phân tích thông qua log backend, log worker và kết quả lưu trong database.

7.4.3 Công cụ triển khai và theo dõi hệ thống

Ở phạm vi khóa luận, hệ thống chủ yếu được chạy trên môi trường phát triển cục bộ. Các thành phần frontend, backend, database và worker được

khởi động riêng để dễ kiểm tra và sửa lỗi. Việc theo dõi hệ thống được thực hiện thông qua log terminal, trạng thái API, dữ liệu trong database và báo cáo hiển thị trên frontend.

7.4.4 Công cụ hỗ trợ trao đổi trong nhóm

Nhóm sử dụng các công cụ trao đổi trực tuyến để thảo luận yêu cầu, phân chia công việc, cập nhật tiến độ và xử lý lỗi. Các nội dung quan trọng như thay đổi kiến trúc, thay đổi schema hoặc thay đổi luồng chạy test đều được trao đổi trước khi cập nhật vào mã nguồn hoặc báo cáo.

7.5 Khó khăn và cách xử lý

7.5.1 Khó khăn về kỹ thuật

Khó khăn kỹ thuật lớn nhất là hệ thống phải kết hợp nhiều công nghệ khác nhau như ReactJS, Node.js, PostgreSQL, Python FastAPI, Browser-use, Playwright và LLM. Mỗi thành phần có cách cấu hình, cách xử lý lỗi và mô hình dữ liệu riêng, nên nhóm cần thiết kế giao tiếp rõ ràng giữa các service.

Cách xử lý là chia hệ thống thành các thành phần độc lập, kiểm tra từng phần trước khi tích hợp. Backend giữ vai trò điều phối, worker tập trung vào thực thi kiểm thử, còn frontend chỉ giao tiếp thông qua API.

7.5.2 Khó khăn về tích hợp hệ thống

Một khó khăn khác là quá trình tích hợp giữa backend và agent-worker. Các phiên chạy test có thể kéo dài, bị timeout hoặc thất bại do nhiều nguyên nhân khác nhau. Vì vậy, hệ thống cần có cơ chế tạo test run, cập nhật trạng thái, nhận callback và lưu error message đầy đủ.

Nhóm xử lý bằng cách thiết kế luồng thực thi bất đồng bộ. Backend tạo test run trước, worker thực thi test và callback kết quả sau khi hoàn

tất. Cách này giúp backend không bị phụ thuộc trực tiếp vào thời gian chạy của trình duyệt.

7.5.3 Khó khăn trong quá trình thực nghiệm

Trong quá trình thực nghiệm, kết quả chạy test có thể bị ảnh hưởng bởi trạng thái website mục tiêu, dữ liệu đầu vào, tốc độ tải trang, locator và phản hồi của mô hình AI. Một số ca kiểm thử có thể thất bại không phải do hệ thống lỗi, mà do website thay đổi trạng thái hoặc dữ liệu kiểm thử không còn phù hợp.

Cách xử lý là sử dụng môi trường kiểm thử có kiểm soát, chuẩn bị tài khoản test, ghi nhận evidence đầy đủ và phân biệt rõ các loại lỗi như lỗi ứng dụng, lỗi dữ liệu, lỗi locator, lỗi môi trường hoặc lỗi do agent suy luận sai.

7.5.4 Cách xử lý và bài học rút ra

Từ quá trình thực hiện đề tài, nhóm rút ra một số bài học chính:

- Cần thiết kế kiến trúc tách thành phần ngay từ đầu để dễ phát triển và sửa lỗi.
- Các chức năng liên quan đến browser automation cần có log, screenshot và evidence đầy đủ.
- Không nên phụ thuộc hoàn toàn vào LLM trong mọi lần chạy, mà cần replay script để giảm chi phí và tăng tính ổn định.
- Object repository và locator fallback là cần thiết để hỗ trợ bảo trì replay script.
- Việc viết báo cáo cần đi song song với quá trình phát triển để tránh thiếu mô tả về quyết định thiết kế và kết quả thực nghiệm.

7.6 Kết chương

Chương này đã trình bày cách nhóm tổ chức thực hiện đề tài, bao gồm phân chia công việc, quy trình làm việc, kế hoạch thực hiện, công cụ hỗ trợ và các khó khăn gặp phải. Nhìn chung, đề tài được thực hiện theo hướng chia nhỏ chức năng, phát triển từng thành phần, tích hợp dần và kiểm tra thường xuyên.

Việc tổ chức công việc theo từng giai đoạn giúp nhóm kiểm soát được phạm vi đề tài, đồng thời đảm bảo các thành phần chính như frontend, backend, database, agent-worker, Browser-use, Playwright và LLM có thể phối hợp với nhau trong một hệ thống thống nhất. Các khó khăn trong quá trình thực hiện cũng là cơ sở để nhóm rút ra kinh nghiệm và đề xuất hướng phát triển tiếp theo ở chương kết luận.

Chương 8

Kết luận và hướng phát triển

8.1 Kết quả đạt được

Khóa luận đã hoàn thành việc nghiên cứu, thiết kế và xây dựng hệ thống kiểm thử tự động ứng dụng Web theo hướng kết hợp giữa AI agent, browser automation và replay script. Hệ thống cho phép người dùng tạo project kiểm thử, mô tả mục tiêu kiểm thử bằng ngôn ngữ tự nhiên, chạy test trên trình duyệt, ghi nhận evidence và xem báo cáo kết quả sau mỗi lần thực thi.

8.1.1 Về mặt nghiên cứu và kiến thức nền tảng

Trong quá trình thực hiện đề tài, nhóm đã tìm hiểu các kiến thức nền tảng liên quan đến kiểm thử phần mềm, kiểm thử tự động ứng dụng Web, kiểm thử chức năng, kiểm thử hồi quy, browser automation, mô hình ngôn ngữ lớn và AI agent. Các kiến thức này là cơ sở để xác định phạm vi hệ thống, phân tích yêu cầu và lựa chọn hướng thiết kế phù hợp.

Nhóm cũng đã khảo sát các công cụ kiểm thử Web phổ biến như Selenium, Playwright, Cypress và Katalon để làm rõ ưu điểm, hạn chế và vị trí của hệ thống đề xuất. Qua đó, khóa luận xác định hướng tiếp cận chính là không thay thế hoàn toàn các framework kiểm thử truyền thống, mà bổ sung một lớp hỗ trợ giúp người dùng tạo và chạy kiểm thử dễ hơn thông

qua mô tả ngôn ngữ tự nhiên.

8.1.2 Về mặt xây dựng hệ thống

Về mặt hiện thực, nhóm đã xây dựng được hệ thống gồm các thành phần chính: frontend, backend API, cơ sở dữ liệu PostgreSQL và agent-worker. Frontend cung cấp giao diện để người dùng quản lý project, test case, dataset, test suite, object repository và report. Backend API xử lý nghiệp vụ, lưu trữ dữ liệu và điều phối yêu cầu chạy test. Agent-worker chịu trách nhiệm chạy test trên trình duyệt thông qua Browser-use và Playwright.

Hệ thống đã hỗ trợ các chức năng chính như:

- Đăng ký, đăng nhập và quản lý phiên làm việc của người dùng.
- Tạo và quản lý project kiểm thử.
- Scan/crawl website để ghi nhận thông tin nền ban đầu.
- Tạo test case từ prompt hoặc goal bằng ngôn ngữ tự nhiên.
- Chạy test case bằng goal-based agent.
- Ghi nhận log, screenshot, action history, evidence và verdict.
- Sinh replay script từ phiên chạy agent thành công.
- Chạy lại replay script bằng Playwright.
- Quản lý test data, dataset và chạy test với nhiều bộ dữ liệu.
- Quản lý test suite để chạy nhiều test case theo lô.
- Lưu object repository và hỗ trợ locator fallback.
- Hỗ trợ phân tích lỗi và hiển thị report sau khi chạy test.

Một điểm quan trọng của hệ thống là sự kết hợp giữa hai chế độ thực thi: chạy bằng agent và chạy bằng replay script. Agent giúp hệ thống linh hoạt hơn ở lần chạy ban đầu, trong khi replay script giúp các lần kiểm thử hồi quy có tính xác định hơn và giảm sự phụ thuộc vào LLM.

8.1.3 Về mặt thực nghiệm và đánh giá

Về mặt thực nghiệm, hệ thống được đánh giá thông qua case study trên website OrangeHRM. Các kịch bản thực nghiệm tập trung vào những luồng kiểm thử phổ biến của ứng dụng Web như đăng nhập, điều hướng, tìm kiếm, chạy test với nhiều bộ dữ liệu và chạy test suite.

Kết quả thực nghiệm cho thấy hệ thống có thể hỗ trợ toàn bộ quy trình kiểm thử từ giai đoạn chuẩn bị project, tạo test case, chạy test, sinh replay script, chạy lại kịch bản và xem báo cáo. Các bằng chứng như log, screenshot, action history và report giúp người dùng theo dõi quá trình thực thi và phân tích nguyên nhân khi test thất bại.

Tuy nhiên, kết quả thực nghiệm cũng cho thấy hiệu quả của hệ thống phụ thuộc vào chất lượng prompt, trạng thái website mục tiêu, dữ liệu kiểm thử, khả năng tìm lại phần tử giao diện và độ ổn định của mô hình AI. Đây là những yếu tố cần tiếp tục được cải thiện trong các phiên bản sau.

8.2 Hạn chế của hệ thống

Mặc dù hệ thống đã hoàn thành các chức năng chính, vẫn còn một số hạn chế cần được ghi nhận.

Thứ nhất, khả năng suy luận của agent còn phụ thuộc vào chất lượng prompt và trạng thái giao diện tại thời điểm chạy. Nếu prompt quá mơ hồ, thiếu điều kiện kiểm tra hoặc website có nhiều luồng hiển thị phức tạp, agent có thể chọn sai hành động hoặc không hoàn thành đúng mục tiêu kiểm thử.

Thứ hai, replay script vẫn có thể bị ảnh hưởng khi giao diện website thay đổi mạnh. Object repository và locator fallback chỉ hỗ trợ trong một số trường hợp phần tử vẫn còn tồn tại nhưng locator chính không hoạt động. Nếu website thay đổi luồng nghiệp vụ, xóa phần tử hoặc thay đổi logic xử lý, người dùng vẫn cần cập nhật test case hoặc replay script.

Thứ ba, hệ thống hiện tập trung chủ yếu vào kiểm thử giao diện Web trên trình duyệt máy tính để bàn. Các loại kiểm thử khác như kiểm thử hiệu năng, kiểm thử bảo mật, kiểm thử API độc lập hoặc kiểm thử ứng dụng di động chưa nằm trong phạm vi hiện tại.

Thứ tư, việc sử dụng LLM có thể phát sinh chi phí, độ trễ và sự không ổn định nhất định. Mặc dù replay script giúp giảm số lần gọi LLM trong các lần chạy sau, hệ thống vẫn cần cơ chế kiểm soát tốt hơn đối với prompt, kết quả sinh ra và các trường hợp mô hình phản hồi không phù hợp.

Thứ năm, cơ chế phân tích lỗi bằng AI mới dừng ở mức hỗ trợ giải thích và gợi ý nguyên nhân. Hệ thống chưa thể tự động sửa mọi lỗi hoặc tự quyết định cập nhật test case mà không cần người dùng xác nhận.

8.3 Bài học kinh nghiệm

Trong quá trình thực hiện đề tài, nhóm rút ra một số bài học kinh nghiệm quan trọng.

Trước hết, đối với một hệ thống có nhiều thành phần như frontend, backend, database, worker, browser automation và LLM, việc thiết kế kiến trúc tách biệt ngay từ đầu là rất cần thiết. Cách tách thành phần giúp quá trình phát triển, kiểm thử và sửa lỗi dễ kiểm soát hơn.

Thứ hai, các chức năng liên quan đến kiểm thử tự động trên trình duyệt cần có cơ chế ghi nhận log và evidence đầy đủ. Khi test thất bại, nếu không có screenshot, action history, URL hiện tại hoặc error message, việc xác định nguyên nhân lỗi sẽ rất khó khăn.

Thứ ba, không nên phụ thuộc hoàn toàn vào AI agent trong mọi lần chạy. Agent phù hợp với giai đoạn tạo và khám phá luồng kiểm thử ban

đầu, nhưng replay script cần thiết để hỗ trợ kiểm thử hồi quy với chi phí thấp hơn và tính xác định cao hơn.

Thứ tư, dữ liệu kiểm thử, trạng thái website và locator có ảnh hưởng lớn đến kết quả chạy test. Vì vậy, hệ thống cần có cơ chế quản lý dataset, object repository và fallback locator để hỗ trợ bảo trì kịch bản kiểm thử.

Cuối cùng, việc viết báo cáo cần đi song song với quá trình phát triển hệ thống. Khi các quyết định thiết kế, khó khăn kỹ thuật và kết quả thực nghiệm được ghi nhận kịp thời, nội dung báo cáo sẽ đầy đủ và phản ánh đúng quá trình thực hiện đề tài hơn.

8.4 Hướng phát triển trong tương lai

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng khác nhau.

Thứ nhất, hệ thống có thể cải thiện khả năng sinh test case bằng cách bổ sung nhiều ngữ cảnh hơn cho mô hình AI, chẳng hạn như kết quả scan/crawl, mô tả nghiệp vụ, trạng thái DOM, screenshot và lịch sử các lần chạy trước. Điều này có thể giúp test case được sinh ra rõ ràng và phù hợp hơn với website mục tiêu.

Thứ hai, cơ chế replay script có thể được mở rộng với nhiều loại action và assertion hơn. Ngoài các thao tác cơ bản như navigate, fill và click, hệ thống có thể hỗ trợ thêm các thao tác phức tạp như upload file, drag-and-drop, chọn ngày, xử lý modal, kiểm tra bảng dữ liệu hoặc kiểm tra trạng thái API phía sau.

Thứ ba, object repository và self-healing có thể được cải tiến để tăng khả năng tìm lại phần tử khi giao diện thay đổi. Hệ thống có thể bổ sung cơ chế chấm điểm locator, học từ các lần fallback thành công hoặc đề xuất cập nhật object sau khi replay script gặp lỗi.

Thứ tư, chức năng AI analysis có thể được phát triển sâu hơn. Thay vì chỉ gợi ý nguyên nhân lỗi, hệ thống có thể đề xuất bản sửa cụ thể cho step, assertion, selector hoặc dữ liệu kiểm thử, sau đó cho phép người dùng

xem lại và xác nhận trước khi lưu.

Thứ năm, hệ thống có thể mở rộng khả năng thực nghiệm và đánh giá trên nhiều website hơn. Việc kiểm thử trên nhiều loại ứng dụng Web khác nhau như thương mại điện tử, quản trị nội bộ, form nghiệp vụ hoặc dashboard dữ liệu sẽ giúp đánh giá rõ hơn độ ổn định và khả năng áp dụng thực tế của hệ thống.

Thứ sáu, hệ thống có thể được triển khai theo hướng hỗ trợ nhiều worker hoặc hàng đợi thực thi để chạy nhiều test song song. Điều này giúp hệ thống phù hợp hơn với nhu cầu kiểm thử hồi quy lớn hoặc chạy test suite định kỳ.

Cuối cùng, hệ thống có thể bổ sung các cơ chế bảo mật nâng cao hơn như mã hóa thông tin xác thực website, che dữ liệu nhạy cảm trong screenshot/log, phân quyền theo nhóm người dùng và tích hợp với các hệ thống CI/CD. Đây là những hướng cần thiết nếu hệ thống được phát triển thành một nền tảng kiểm thử tự động có khả năng ứng dụng trong môi trường thực tế.

8.5 Kết luận chung

Khóa luận đã đề xuất và xây dựng một hệ thống kiểm thử tự động ứng dụng Web theo hướng kết hợp giữa AI agent, browser automation và replay script. Hệ thống cho phép người dùng mô tả mục tiêu kiểm thử bằng ngôn ngữ tự nhiên, chạy test trên trình duyệt, ghi nhận bằng chứng, sinh replay script và hỗ trợ chạy lại kịch bản kiểm thử.

Kết quả đạt được cho thấy hướng tiếp cận này có tiềm năng trong việc giảm rào cản tạo test case, hỗ trợ kiểm thử hồi quy và cải thiện khả năng truy vết kết quả kiểm thử. Mặc dù hệ thống vẫn còn một số hạn chế về độ ổn định của agent, khả năng phục hồi khi giao diện thay đổi và phạm vi thực nghiệm, đề tài đã tạo được nền tảng ban đầu cho việc phát triển các hệ thống kiểm thử Web thông minh hơn trong tương lai.

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Tài liệu tham khảo

Tiếng Anh

- [1] Boehm, B. W. and Basili, V. R., “Software defect reduction top 10 list,” *Computer*, vol. 34, no. 1, pp. 135–137, 2001.
- [2] Selenium. “Selenium browser automation project,” Accessed: Jun. 29, 2026. [Online]. Available: <https://www.selenium.dev/>
- [3] Microsoft. “Playwright documentation,” Accessed: Jun. 29, 2026. [Online]. Available: <https://playwright.dev/>
- [4] Google AI for Developers. “Gemini API documentation,” Accessed: Jun. 29, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs>
- [5] Browser Use Contributors. “Browser-use: Make websites accessible for AI agents,” Accessed: Jun. 27, 2026. [Online]. Available: <https://github.com/browser-use/browser-use>
- [6] ISTQB. “ISTQB Glossary,” Accessed: Jul. 5, 2026. [Online]. Available: <https://glossary.istqb.org/>
- [7] Selenium. “Webdriver,” Accessed: Jul. 5, 2026. [Online]. Available: <https://www.selenium.dev/documentation/webdriver/>
- [8] Microsoft. “Playwright documentation,” Accessed: Jul. 5, 2026. [Online]. Available: <https://playwright.dev/docs/intro>

- [9] Cypress. “Writing your first end-to-end test,” Accessed: Jul. 5, 2026. [Online]. Available: <https://docs.cypress.io/app/end-to-end-testing/writing-your-first-end-to-end-test>
- [10] Katalon. “Katalon documentation,” Accessed: Jul. 5, 2026. [Online]. Available: <https://docs.katalon.com/>
- [11] OpenAI. “Text generation,” Accessed: Jul. 5, 2026. [Online]. Available: <https://platform.openai.com/docs/guides/text>
- [12] OpenAI. “Structured outputs,” Accessed: Jul. 5, 2026. [Online]. Available: <https://platform.openai.com/docs/guides/structured-outputs>

Phụ lục A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Phụ lục B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.